

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein

Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking

Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iii
Tabellenverzeichnis	iv
Abkürzungsverzeichnis	v
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum	2
1.2.2 Unidirektionalität	2
1.2.3 Herausforderungen	2
1.3 Zielsetzung	3
1.4 Abgrenzung und Umfang	4
1.5 Aufbau der Arbeit	4
1.6 KI-Einsatz	5
2 Grundlagen	6
2.1 IPv4	6
2.2 Das User Datagram Protocol (UDP)	10
2.2.1 Protokollstruktur	11
2.2.2 Eigenschaften	13
2.3 Erweiterungsmuster im Vergleich	14
2.4 Die Surplus Area	16
2.4.1 Lage und Kompatibilität	16
2.4.2 Aufbau und Ausrichtung	17
2.4.3 OCS und Fehlerbehandlung	19
2.4.4 TLV-Format und Empfangsentscheidung	20
2.4.5 Entwurfsprinzipien und Optionsklassen	22
2.5 Betriebssystem und Netzpfad	24
3 Methodik und KI-gestützte Entwicklung	30
3.1 Forschungsdesign	30
3.2 RFC-Auswertung und schrittweise Umsetzung	32
3.3 Beteiligte und ihre Rollen	35
3.4 Bekannte Schwächen der Sprachmodelle	36

3.5	Testarchitektur und Prüfprozess	37
3.6	Externe Referenzfälle	40
3.7	Reproduzierbarkeit und Grenzen	41
4	Analyse und Anforderungsmodell	42
4.1	Anwendung der Extraktionsmethode	42
4.2	Anforderungskatalog und Konformitätsmatrix	43
4.3	FF1-Bewertungskriterien	45
4.4	FF2-Pfad- und Fehlermodell	45
4.5	Technologieauswahl	49
4.5.1	Programmiersprache	49
4.5.2	Zugang zur Surplus Area	51
4.5.3	Mitschnitt und Auswertung	52
4.5.4	Fuzzing und formale Gegenprobe	52
5	Entwurf und Implementierung	54
5.1	Vorgehen der Implementierung	54
5.2	Gesamtarchitektur	55
5.3	Sendepfad	57
5.4	Empfangspfad	60
5.5	Optionsverarbeitung	65
5.6	Prüfarchitektur	68
5.7	Betriebs- und Sicherheitsgrenzen	71
5.8	Bewusste Grenzen	73
6	Evaluation und Diskussion	75
6.1	Test- und Messkonzept	75
6.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	75
6.3	Soll-Ist-Abgleich mit RFC 9868	75
7	Fazit und Ausblick	76
7.1	Beantwortung der Forschungsfragen	76
7.2	Ausblick	76
7.3	Schlusswort	76
	Literaturverzeichnis	77
	Eidesstattliche Erklärung	81

Abbildungsverzeichnis

2.1	Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.	7
2.2	Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente	10
2.3	Der UDP-Header nach RFC 768 [1]	11
2.4	Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]	12
2.5	IP Transport Payload und Surplus Area eines Datagramms mit Total Length 60	16
2.6	Anatomie eines Datagramms mit UDP Options	18
2.7	Paritätsentscheid für das Pad-Byte vor dem OCS	19
2.8	Empfangsentscheidung für die Surplus Area im Standardfall	22
2.9	Sende- und Empfangsweg eines UDP-Datagramms durch den Linux-Kernel (nach [4])	25
2.10	Die zwei Sendepfade durch den Linux-Kernel [4, 17]	26
2.11	Der Empfangspfad durch den Linux-Kernel [4, 17]	27
2.12	Messmodell des Datagrammwegs	28
3.1	Aufbau der Untersuchung und Zuordnung der Forschungsfragen . . .	30
3.2	Von den Normquellen zum geprüften Anforderungsbestand	33
3.3	Kernzyklus eines Implementierungsschritts bis Schritt 18 (Ausnahmen gestrichelt)	34
3.4	Rollen bei Erzeugung, Prüfung und Freigabe	35
3.5	Schwächen der Sprachmodelle und methodische Antworten	37
3.6	Prüfprozess im Sollzustand	39
4.1	Verteilung der markierten Absätze über die Abschnitte von RFC 9868 (eigene Zählung)	43
4.2	Kantenmodell eines Messpfads mit den erwarteten Mechanismenklassen	47
5.1	Komponenten der Bibliothek und ihre Abhängigkeiten	56
5.2	Entstehung des Beispieldatagramms in vier Schritten	58
5.3	Empfangspipeline der Bibliothek mit ihren Ergebnisarten	62
5.4	Entwicklungs- und Nachweisartefakte des Implementierungs-Repositorys	68

Tabellenverzeichnis

3.1	Operationalisierung der Forschungsfragen	31
4.1	Auszug aus der Konformitätsmatrix des Implementierungs-Repositorys (Stand f847895; normative Aussagen nach RFC 9868 [2])	44
4.2	Implementierungsumfang	48
4.3	Messinfrastruktur	48
4.4	Vergleich der Sprachkandidaten	50
5.1	Reichweite der Prüfmittel	70

Abkürzungsverzeichnis

APC Additional Payload Checksum.

AUTH Authentication Option.

DTLS Datagram Transport Layer Security.

EOL End of List.

FRAG Fragmentation Option.

MDS Maximum Datagram Size.

MRDS Maximum Reassembled Datagram Size.

NOP No Operation.

OCS Option Checksum.

PCAP Packet Capture.

PMTU Path Maximum Transmission Unit.

TIME Timestamp Option.

1. Einleitung

1.1 Motivation

Die ursprüngliche Spezifikation des User Datagram Protocol (UDP), RFC 768 aus dem Jahr 1980, umfasst 663 Wörter [1]. Die Erweiterung RFC 9868 wurde im Oktober 2025 veröffentlicht und führt unter dem Titel *Transport Options for UDP* erstmals einen allgemeinen Optionsmechanismus für UDP ein [2]. Zwischen beiden Dokumenten liegen 45 Jahre; die Klartextfassung des RFC 9868 ist mehr als 27-mal so umfangreich wie die des RFC 768.

Die Einfachheit von UDP ist dabei kein Mangel, sondern Entwurfsziel: Das Protokoll bietet lediglich Portnummern zur Demultiplexierung und eine Prüfsumme zur Fehlererkennung. Genau diese Einfachheit macht UDP zu einem der am häufigsten genutzten Transportprotokolle im Internet.

RFC 9868 greift Funktionen auf, die UDP ursprünglich der Anwendung oder einer darüberliegenden Schicht überließ: Transportfragmentierung, eine zusätzliche Nutzdatenprüfsumme sowie Optionen für Größen- und Zeitinformationen [2, Sec. 5, 11.3 und 11.8]. Für ihre Bereitstellung kommen grundsätzlich drei Wege in Betracht:

- Erweiterung innerhalb der Nutzdaten: Jede Anwendung definiert ein eigenes Rahmenformat in der UDP-Nutzlast.
- Ein neues Transportprotokoll: Alle gewünschten Mechanismen können so von Beginn an vorgesehen werden.
- Erweiterung des bestehenden Formats: UDP selbst erhält einen Optionsmechanismus, der für Bestandssysteme möglichst unsichtbar bleibt.

RFC 9868 wählt den dritten Weg (die Muster hinter diesen Wegen vergleicht Kapitel 2) und nutzt dafür die sogenannte *Surplus Area*: den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms. Da das UDP-Length-Feld die Länge des UDP-Datagramms angibt und diese kleiner sein kann als die *IP Transport Payload*, also alles, was im IP-Datagramm auf den IP-Header folgt, entsteht ein bisher ungenutzter Bereich für Erweiterungen (Abschnitt 2.4). Auf dieser Grundlage definiert RFC 9868 ein erweiterbares Rahmenwerk für Transportoptionen.

Damit steht das minimalistische UDP vor seiner bislang größten Erweiterung. Nun stellt sich die Frage, die diese Arbeit als Leitfrage begleitet:

Wie lässt sich UDP um Transportoptionen erweitern, ohne seine grundlegenden Eigenschaften aufzugeben?

Die Arbeit untersucht diese Frage anhand einer Referenzimplementierung und auf ausgewählten realen Netzpfeilen.

1.2 Problemstellung

Die Implementierung von UDP-Optionen gemäß RFC 9868 wirft drei Problemfelder auf, die diese Arbeit behandelt: die Beschaffenheit der Surplus Area selbst, die bewusste Unidirektionalität des Protokolls sowie die praktischen Implementierungsherausforderungen vom Zugriff aus dem Benutzerraum bis zum Verhalten von Zwischensystemen. Die folgenden Abschnitte behandeln sie in dieser Reihenfolge.

1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum

Anders als TCP besitzt der ursprüngliche UDP-Header kein explizites Optionsfeld. RFC 9868 platziert die Optionen deshalb außerhalb der durch das UDP-Length-Feld begrenzten Nutzdaten, aber noch innerhalb des IP-Datagramms. Damit bleibt der UDP-Header unverändert, die Implementierung muss jedoch zwei unterschiedliche Längengrenzen zuverlässig auswerten. Den genauen Aufbau der Surplus Area behandelt Abschnitt 2.4.

1.2.2 Unidirektionalität

RFC 9868 stellt explizit klar: „*UDP is unidirectional; UDP Options do not change that fact.*“ [2, Sec. 6].

Für Mechanismen, die Bidirektionalität voraussetzen, fordert RFC 9868 eine getrennte Spezifikation [2, Sec. 6]. RFC 9869 definiert einen solchen Einsatz von REQ und RES für die Pfad-MTU-Ermittlung [3, Sec. 3]. Diese Vorgabe schließt einen Datenaustausch in beiden Richtungen nicht aus. Sie bedeutet, dass die UDP-Optionsschicht keine Antwort selbstständig erzeugt. Nicht das Protokoll, sondern die Anwendung oder eine in ihrem Auftrag arbeitende Bibliothek löst eine Antwort aus.

1.2.3 Herausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP Optionen liegt im Zugriff auf die Surplus Area. Existierende Betriebssystem-Stacks liefern über Standard-Socket-APIs ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendungsschicht; die Surplus Area wird stillschweigend verworfen

[2, Sec. 18]. Den Weg eines UDP-Pakets durch den Linux-Kernel bis zu dieser Socket-Schnittstelle dokumentieren Stephan und Wüstrich [4].

Eine weitere Herausforderung stellt die Kompatibilität mit Zwischensystemen dar. RFC 9868 berichtet Interoperabilitätstests, in denen Pakete mit Surplus Area Geräte zur Netzadressübersetzung (NAT, *Network Address Translation*) passierten, nennt aber auch ein Intrusion Detection System, das die Diskrepanz zwischen UDP-Length und IP-Length in der Standardeinstellung als Angriff meldet [2, Sec. 18]. Ob reale Netzpfade solche Pakete tatsächlich durchlassen, ist eine empirische Frage dieser Arbeit (FF2).

1.3 Zielsetzung

Die Problemstellung verdichtet sich zu zwei getrennten Hürden: Ein UDP-Options-Paket muss normgerecht erzeugt und empfangen werden, und seine Surplus Area muss die untersuchten Netzpfade unverändert überstehen.

Eine normgerechte Implementierung sagt nichts darüber aus, ob Zwischensysteme Datagramme mit Surplus Area unverändert passieren lassen. Ebenso nützt ein durchlässiger Netzpfad wenig, wenn Erzeugung und Auswertung der Optionen fehlerhaft sind. Aus dieser Trennung ergeben sich zwei Forschungsfragen:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen?
- **FF2:** In welchem Umfang bleibt die *Surplus Area* auf den untersuchten realen Netzpfaden bis zur Gegenstelle unverändert erhalten?

Das Ziel dieser Arbeit ist es, beide Fragen zu beantworten. Dazu entsteht zunächst eine an RFC 9868 ausgerichtete und systematisch auf Konformität geprüfte Bibliothek für UDP-Transportoptionen in Rust. Die auf Linux und IPv4 begrenzte Implementierung arbeitet im Benutzerraum mit Raw Sockets. Sie umfasst Parser und Serialisierer für UDP-Optionen nach dem TLV-Schema (*Type-Length-Value*), die Validierung der Optionsprüfsumme (*Option Checksum*, OCS) gemäß Abschnitt 9 des RFC 9868 sowie die verpflichtend zu unterstützenden Optionen EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES [2, Sec. 10].

Die Verifikation erfolgt mehrstufig: Komponententests prüfen die einzelnen Bestandteile, Integrationstests den Loopback-Pfad, und für reine Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient als formale Gegenprobe für Prüfsummenarithmetik sowie Längen- und Anordnungsregeln; das Verhalten von Raw Sockets,

Betriebssystemen und Zwischensystemen bleibt Gegenstand empirischer Prüfungen. Dieses Prüfprogramm schafft die Grundlage für die Beantwortung von FF1.

Für FF2 vergleicht die Evaluation gewöhnliche UDP-Datagramme mit gleich großen Datagrammen, die eine Surplus Area enthalten, auf ausgewählten realen Netzpfeilen. Sie unterscheidet zwischen unveränderter Zustellung, Entfernung der Surplus Area und Paketverlust. Die Schlussfolgerungen bleiben auf die beobachteten Pfeile begrenzt.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf eine Umsetzung des RFC 9868 im Linux-Benutzerraum für IPv4. Gegenstand sind das TLV-Rahmenwerk, die OCS sowie die verpflichtend zu unterstützenden Optionen. Die Beschränkung auf den Benutzerraum ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Abschnitt 4.5 begründet die dafür gewählte Umsetzung mit Raw Sockets gegenüber kernelnahen Verfahren.

Außerhalb des Umfangs liegt der REQ/RES-Anwendungsfall zur Pfad-MTU-Ermittlung, den RFC 9869 definiert [3]. Die Implementierung unterstützt nur die in RFC 9868 festgelegten Formate von REQ und RES. Ebenfalls nicht implementiert wird die TIME-Option. RFC 9868 beschreibt zwar ihr Format und den Anwendungszweck, die Schätzung der Paketumlaufzeit (*Round-Trip Time*, RTT), legt aber kein nutzendes Protokoll fest. Die Kennungen für AUTH, UCMP und UENC sind in RFC 9868 lediglich reserviert und werden daher ebenfalls nicht implementiert (AUTH [2, Sec. 11.9], UCMP [2, Sec. 12.1], UENC [2, Sec. 12.2]). Kernel-Module werden nicht entwickelt.

IPv6 wird weder implementiert noch auf Konformität geprüft. Die hier untersuchten Mechanismen des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) lassen sich vollständig über IPv4 zeigen. Die Implementierung und ihre Konformitätsprüfung beziehen sich daher auf IPv4; in den Pfadmessungen erscheint IPv6 nur als Kontrollgröße der Messumgebung (Abschnitt 4.4).

1.5 Aufbau der Arbeit

Nach dieser Einleitung folgen sechs Kapitel, die drei Blöcke bilden:

- **Grundlagen und Methodik:** Kapitel 2 führt die technischen Grundlagen ein: das Internet Protocol, UDP, Erweiterungsmuster von Transportprotokollen sowie RFC 9868 mit der *Surplus Area* und seinen Entwurfsprinzipien. Kapi-

tel 3 legt die Forschungsmethodik offen, bevor mit ihr gearbeitet wird: das Forschungsdesign, die RFC-Auswertungsmethode und der Einsatz von KI.

- **Anwendungsteil:** Kapitel 4 überführt RFC 9868 mit dieser Methode in ein prüfbares Anforderungsmodell und begründet die Technologieauswahl. Kapitel 5 führt Entwurf und Implementierung der Bibliothek komponentenweise zusammen. Kapitel 6 berichtet die Ergebnisse zu beiden Forschungsfragen sowie eine deskriptive Auswertung der KI-gestützten Arbeitsweise.
- **Schluss:** Kapitel 7 wiederholt die Forschungsfragen, beantwortet sie einzeln und zieht die Bilanz zur Leitfrage.

Diese lineare Kapitelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ und agenten-gestützt, sodass Entwurf, Implementierung und Test einander fortlaufend bedingten.

1.6 KI-Einsatz

Diese Arbeit ist unter durchgehendem Einsatz künstlicher Intelligenz (KI) entstanden. Der Einsatz betrifft Text und Software. Auf der Textseite erstellten KI-Werkzeuge Entwürfe, überarbeiteten Formulierungen, übernahmen die LaTeX-Formatierung, setzten die Abbildungen als TikZ-Grafiken um und bedienten die LaTeX-Werkzeugkette. Zudem prüften sie den Text wiederholt gegen den Primärtext des RFC 9868. Auf der Softwareseite unterstützten sie Planung, Implementierung, Tests und Verifikation der Bibliothek. Der Autor entschied über die Übernahme der Ergebnisse und verantwortet sie.

Zum Einsatz kommen zwei KI-Werkzeuge (harness), die ein Sprachmodell mit Dateizugriff und Kommandoausführung verbinden: Claude Code von Anthropic und Codex von OpenAI. Ihre Rollenverteilung, die bekannten Schwächen dieser Arbeitsweise und die Prüfkette vor der Übernahme eines Ergebnisses behandelt Kapitel 3.

2. Grundlagen

Dieses Kapitel stellt die Grundlagen bereit, die zum Verständnis von RFC 9868 und zur Beurteilung der späteren Implementierung und Messungen nötig sind: IPv4 und UDP als die beiden beteiligten Protokolle (Abschnitt 2.1, Abschnitt 2.2), die Erweiterungsmuster im Vergleich (Abschnitt 2.3), die Surplus Area samt Entwurfsprinzipien (Abschnitt 2.4) sowie Betriebssystem und Netzpfad der Messungen (Abschnitt 2.5).

2.1 IPv4

Das Internet Protocol (IP) vermittelt Datagramme über die Grenzen einzelner Netze hinweg zwischen Endsystemen und bildet damit die Vermittlungsschicht, auf der UDP aufsetzt [5]. Dass diese Arbeit eine Schicht unterhalb ihres eigentlichen Gegenstands beginnt, hat einen konkreten Grund: Die *Surplus Area*, in der RFC 9868 seine Optionen ablegt, ist allein durch das Zusammenspiel beider Schichten definiert. Das UDP-Length-Feld markiert das Ende der UDP-Nutzdaten; wo das umgebende IP-Datagramm endet, ergibt sich dagegen ausschließlich aus dem IP-Header. Ohne die Längenangaben von IPv4 lässt sich daher weder feststellen, ob eine Surplus Area vorhanden ist, noch wie groß sie ausfällt [2, Sec. 7].

Dabei genügt der Blick auf die Version 4 des Protokolls. RFC 9868 definiert die Surplus Area und den Optionsmechanismus für IPv4 und IPv6 in gleicher Struktur. Die Versionen unterscheiden sich zum einen darin, wie die Nutzlast des IP-Datagramms abgegrenzt wird: bei IPv4 durch zwei Felder des festen Headers [2, Sec. 7], bei IPv6 durch die Längenangabe des Basis-Headers zusammen mit der Kette der Extension Header [6]. Zum anderen unterscheiden sich einzelne Randwerte: So muss ein Empfänger bei der Fragmentierungsoption von RFC 9868 wieder zusammengesetzte Datagramme von mindestens 2926 Byte bei IPv4 und 2886 Byte bei IPv6 unterstützen [2, Sec. 11.6], und allein bei IPv6 gibt es Pakete ohne eigenständige UDP-Längenangabe (UDP `Length` gleich null, etwa bei Jumbograms), in denen UDP Options nicht einsetzbar sind [2, Sec. 22]. Die Struktur des Optionsmechanismus berühren diese Unterschiede nicht; er lässt sich vollständig an IPv4 zeigen. Implementierung und Evaluation dieser Arbeit sind entsprechend auf IPv4 begrenzt (Abschnitt 1.4).

Auch IPv4 selbst wird in diesem Unterkapitel nicht vollständig erklärt und dargestellt. Dieser Abschnitt behandelt ausschließlich die Themen, Header-Elemente und Eigenschaften, die RFC 9868 und die spätere Implementierung tatsächlich

berühren: die beiden Längenangaben des Headers, das Protokollfeld, die Adressen, die Header-Prüfsumme, das Fragmentierungsverhalten sowie das Feld **Time to Live** als Messgröße der Evaluation.

Den Ausgangspunkt bildet der Header; seinen Aufbau zeigt Abbildung 2.1 [5].

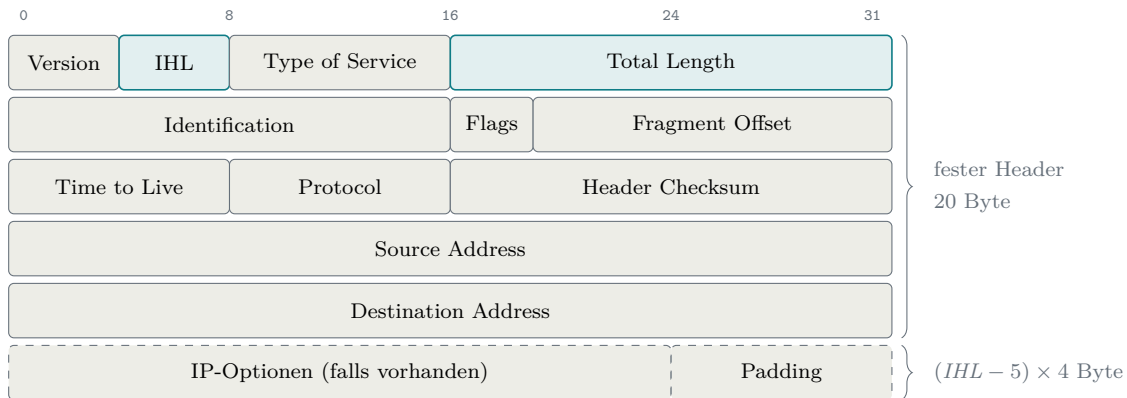


Abbildung 2.1: Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.

Im Zentrum stehen die beiden Längenangaben der ersten Zeile. **Total Length** gibt die Gesamtlänge des Datagramms in Byte an, Header und Nutzlast zusammen; als 16-Bit-Feld erlaubt es höchstens 65 535 Byte.

Die zweite Längenangabe, die **IHL** (Internet Header Length), beziffert die Länge des Headers. Sie zählt dabei nicht in Byte, sondern in 32-Bit-Wörtern zu je 4 Byte. Der Grund liegt in der Breite des Feldes: Mit seinen 4 Bit kann es höchstens den Wert 15 darstellen. In Byte gezählt würde das nicht einmal für den festen Headeranteil von 20 Byte reichen; in 32-Bit-Wörtern gezählt beschreibt derselbe Wertebereich dagegen Header bis 60 Byte [5]. Die Headerlänge ergibt sich daher als

$$\text{Headerlänge} = IHL \times 4 \text{ Byte.} \quad (2.1)$$

Der Minimalwert 5 entspricht dem optionslosen 20-Byte-Header, der Maximalwert 15 einem Header von 60 Byte. Eine IPv4-Headerlänge ist damit stets ein Vielfaches von 4; diese unscheinbare Eigenschaft kehrt bei der Ausrichtung der UDP Options wieder.

Zwischen festem Header und Nutzlast können zudem IP-Optionen liegen. Sie stellen Steuerfunktionen für Sonderfälle bereit, die die gewöhnliche Kommunikation nicht braucht, und dienen als zusätzliche Felder zur Erweiterung der IP-Header [5]: **Record Route** etwa sammelt die Adressen der durchlaufenen Router ein, **Internet Timestamp** deren Zeitstempel. In der Praxis werden solche Optionen kaum genutzt.

Wie ein Router optionstragende Pakete behandelt, hängt von seiner Architektur und Konfiguration ab: Er kann sie mit Leitungsgeschwindigkeit weiterleiten oder filtern oder die Optionen im allgemeinen Prozessor oder in Spezialhardware verarbeiten; vor allem die Verarbeitung im allgemeinen Prozessor birgt bei großen Paketmengen ein Überlastrisiko. Zugleich ist das Verwerfen optionstragender Pakete verbreitete Praxis [7, Sec. 1 und 3.1]. Zudem fasst der 40-Byte-Optionsraum bei **Record Route** höchstens neun Adressen und damit viele reale Pfade nicht [5]. Abschließend gilt, dass erst die IHL und keine Konstante den Beginn der Nutzlast festlegt.

Aus den beiden Längenangaben **Total Length** und IHL ergibt sich die Größe, auf die sich RFC 9868 durchgängig bezieht: Die Nutzlast eines IPv4-Datagramms beginnt nach $IHL \times 4$ Byte und endet bei **Total Length**. Für ein UDP-Datagramm fordert RFC 9868 deshalb [2, Sec. 7]¹

$$UDP\ Length \leq Total\ Length - Headerlänge. \quad (2.2)$$

Die rechte Seite der Ungleichung ist die Länge dieser Nutzlast, also der Platz hinter dem IP-Header. Die linke Seite ist die Länge, die das UDP-Datagramm für sich beansprucht. Die Bedingung verlangt damit, dass das UDP-Datagramm vollständig in die Nutzlast des IP-Datagramms passt; UDP darf nie mehr Platz beanspruchen, als das IP-Datagramm hergibt. Bei Gleichheit füllt das UDP-Datagramm die Nutzlast exakt aus; so arbeiten klassische Sender. Bleibt **UDP Length** dagegen zurück, ist die Differenz genau der Raum, den Abschnitt 2.4 als Surplus Area einführt.

Von den übrigen Feldern braucht die Arbeit nur wenige. **Protocol** benennt das Transportprotokoll der Nutzlast; der Wert 17 steht für UDP. Die **Header Checksum** sichert ausschließlich den Header, nicht die Nutzlast; deren Fehlererkennung bleibt der UDP-Prüfsumme überlassen (Abschnitt 2.2.1). Quell- und Zieladresse geben Absender und Empfänger des Datagramms an. Sie werden später noch einmal wichtig: Die UDP-Prüfsumme bezieht beide Adressen über einen sogenannten Pseudo-Header in ihre Berechnung ein. **Version** und **Type of Service** berühren den Optionsmechanismus nicht und werden nicht weiter vertieft.

Auch **Time to Live** spielt für die Optionen keine Rolle, kehrt in der Evaluation aber als Messgröße wieder. Jede weiterleitende Instanz verringert den Wert um mindestens eins und verwirft das Datagramm bei null [5]. Die Differenz zwischen gesendetem und empfangenem Wert entspricht deshalb nur unter der Annahme eines Dekrements von genau eins je Weiterleitung der Zahl der durchlaufenen IPv4-Router;

¹ In Ausnahmefällen liegen zwischen IPv4- und UDP-Header noch Shim-Header, etwa bei IPsec oder IPComp; das Feld **Protocol** zeigt dann nicht UDP an, und die Obergrenze verringert sich zusätzlich um die Gesamtlänge dieser Shim-Header [2, Sec. 7]. Diese Arbeit betrachtet solche Ketten nicht weiter.

Kapitel 6 nutzt sie unter dieser Annahme, um die Stabilität der Messpfade zu prüfen. Die zweite Zeile des Headers schließlich (**Identification**, **Flags**, **Fragment Offset**) gehört vollständig zur Fragmentierung, der sich der Rest dieses Abschnitts widmet.

Wie groß ein Datagramm praktisch werden kann, entscheidet allerdings nicht das 16-Bit-Längenfeld, sondern der Übertragungsweg. Jede Technik der Sicherungsschicht begrenzt die Nutzlast eines einzelnen Rahmens; diese Obergrenze heißt Maximum Transmission Unit (MTU) und liegt für Ethernet üblicherweise bei 1500 Byte. Streng genommen ist die MTU also eine Eigenschaft der Sicherungsschicht. Ihre Folgen trägt jedoch die Vermittlungsschicht, denn ein Datagramm durchquert auf dem Weg zum Ziel mehrere Netze mit möglicherweise unterschiedlichen MTUs, und IP muss festlegen, was mit einem Datagramm geschieht, das nicht in den nächsten Rahmen passt [5].

IPv4 beantwortet das mit Fragmentierung. Ist ein Datagramm größer als die MTU und das Flag **DF** (Don't Fragment) nicht gesetzt, zerlegt die IP-Schicht es in mehrere Fragmente. Das übernimmt nicht nur der Absender: Bei IPv4 darf auch jeder Router unterwegs ein Datagramm zerlegen, das nicht in sein nächstes Netz passt [5]. Jedes Fragment erhält einen eigenen IP-Header, übernimmt die **Identification** des Originals, trägt in **Fragment Offset** seine Position in Einheiten von 8 Byte und setzt das Flag **MF** (More Fragments), das nur beim letzten Fragment fehlt. Der Empfänger sammelt zusammengehörige Fragmente anhand von Quelladresse, Zieladresse, Protokoll und **Identification** und setzt das Datagramm vor der Übergabe an die Transportschicht wieder zusammen [5]. Ist **DF** dagegen gesetzt, verwirft ein Router das zu große Datagramm und meldet dies dem Absender per ICMP; auf dieser Rückmeldung baut die Path MTU Discovery auf, mit der ein Sender die kleinste MTU seines Pfades ermittelt [8, Sec. 3.2].

Für diese Arbeit ist an der Fragmentierung vor allem eine Konsequenz bedeutsam: Der UDP-Header gehört aus Sicht von IP zur Nutzlast und landet deshalb vollständig im ersten Fragment. Was das für die übrigen Fragmente bedeutet, zeigt Abbildung 2.2 am Beispiel eines in drei Teile zerlegten Datagramms.

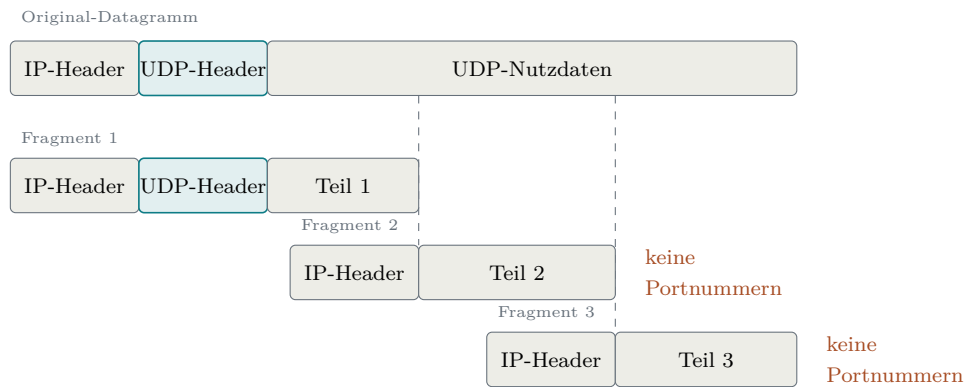


Abbildung 2.2: Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente

Aus Abbildung 2.2 geht hervor, dass nur das erste Fragment den UDP-Header enthält: Alle Folgefragmente tragen keine Portnummern. Zwischensysteme wie NATs und Firewalls, die Pakete anhand vollständiger Header zuordnen oder prüfen, können solche Fragmente nicht einordnen und verwerfen sie teils pauschal; zudem macht der Verlust eines einzelnen Fragments das gesamte Datagramm unbrauchbar. RFC 8085 wertet beides als grundsätzliches Problem der IP-Fragmentierung, das jedes Transportprotokoll trifft, und rät UDP-Anwendungen deshalb, IP-Fragmentierung zu vermeiden [8, Sec. 3.2]. Dass der Rat gerade an UDP geht, hat einen Grund: Während TCP seinen Datenstrom selbst in passende Segmente zerlegt und der IP-Fragmentierung so meist entgeht [9], erzeugt UDP je Nachricht genau ein Datagramm (Abschnitt 2.2.2); ein zu großes Datagramm kann bislang nur die IP-Schicht zerteilen. Genau diese Lücke schließt die FRAG-Option von RFC 9868: Sie verlagert die Fragmentierung auf die Transportschicht und wiederholt die Portnummern in jedem Fragment [2, Sec. 11.4]; den Verlust einzelner Fragmente behebt allerdings auch FRAG nicht. Darauf kommt Kapitel 6 zurück.

Der folgende Abschnitt wendet sich dem Protokoll zu, das RFC 9868 tatsächlich erweitert: UDP.

2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist neben dem Transmission Control Protocol (TCP) eines der beiden zentralen Transportprotokolle der Transportschicht. Im Gegensatz zu TCP verfolgt UDP ein bewusst minimalistisches Design: Es sendet Datagramme ohne vorherigen Handshake oder Zustandsabgleich mit dem Empfänger, bietet keine Garantien für Zustellung, Reihenfolge oder Duplikaterkennung und verzichtet auf jeglichen Flusskontroll- oder Staukontrollmechanismus [1]. Mit der Protokollnummer 17 im IP-Header registriert, wurde UDP in RFC 768 spezifiziert und trägt die Bezeichnung STD 6 als Internet-Standard. Bemerkenswert ist, dass

RFC 768 seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg keine formelle Aktualisierung erfahren hat. Erst RFC 9868 trägt den Header-Eintrag **Updates: 768** und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

Historisch geht UDP auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, das Cerf, Dalal und Sunshine Ende 1974 noch als monolithische Einheit aus Netzwerk- und Transportfunktionen beschrieben [10]. Mit deren Trennung in IP und TCP entstand Raum für eine schlanke, transaktionsorientierte Alternative ohne den Overhead von TCP [1]. Der UDP-Kern blieb über die folgenden Jahrzehnte unverändert, und Begleitdokumente wie die Einsatzrichtlinien von RFC 8085 klärten Randfragen, ohne den Standard formell zu ändern [8]. Die Motivation, ihn nun doch direkt zu erweitern, benennt RFC 9868 selbst: UDP zählt zu den meistgenutzten Protokollen ohne Raum für Header-Optionen, und weil inzwischen viele Protokolle UDP direkt nutzen, wäre eine Zwischenschicht oberhalb von UDP für Bestandsanwendungen kaum noch auszurollen [2, Sec. 4 und 5].

2.2.1 Protokollstruktur

Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten; den Aufbau des Headers zeigt Abbildung 2.3 [1].

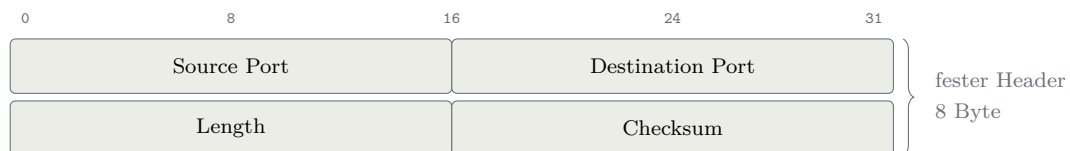


Abbildung 2.3: Der UDP-Header nach RFC 768 [1]

Aus Abbildung 2.3 geht hervor, dass der gesamte Header aus genau vier Feldern zu je 16 Bit besteht; unter ihnen das Längenfeld, für das Abschnitt 2.1 mit Gleichung 2.2 bereits die Obergrenze aufgestellt hat. Die vier Felder im Einzelnen:

- **Source Port:** Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1]. In der Praxis nutzen Anwendungen den *Source Port* zur Identifikation des Absenders, damit der Empfänger Antworten an den korrekten Port zurücksenden kann.
- **Destination Port:** Die Portnummer des Zielprozesses auf dem empfangenden Host. Zusammen mit der IP-Zieladresse ermöglicht dieses Feld die *Demultiplexierung* eingehender Datagramme an die korrekte Anwendung.
- **Length:** Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header und Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutz-

daten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte.

- **Checksum:** Eine Prüfsumme zur Fehlererkennung über einen Pseudo-Header, den UDP-Header und die Nutzdaten; ihr Verfahren und ihre Sonderwerte erläutert der folgende Absatz.

Prüfsumme. Die Prüfsumme dient der Fehlererkennung: Mit ihr erkennt der Empfänger, ob einzelne Bits des Datagramms auf dem Weg verändert wurden. Diese Prüfung auf der Transportschicht ist nötig, weil nicht jede Teilstrecke eines Pfades eigene Fehlererkennung bietet und Bits auch im Speicher eines Routers kippen können [11]. Dennoch ist die Prüfsumme optional. Für die Berechnung summiert der Sender Pseudo-Header, UDP-Header und Nutzdaten als 16-Bit-Wörter im Einerkomplement auf (ein Übertrag wird am niederwertigen Ende wieder hinzugezählt); das invertierte Ergebnis bildet den Feldwert [1, 12]. Der Empfänger summiert dieselben Wörter einschließlich des Prüfsummenfelds und erwartet lauter Einsen, also 0xFFFF; jedes andere Ergebnis zeigt einen Übertragungsfehler an [12].

Dabei hilft eine Eigenheit des Einerkomplements: Es kennt zwei Darstellungen der Null, 0x0000 und 0xFFFF. Diese Doppelrolle nutzt UDP für die Sonderwerte des Feldes: eine übertragene 0x0000 bedeutet, dass der Sender keine berechnet hat. Ergibt die Berechnung selbst den Wert null, wird stattdessen 0xFFFF übertragen. Dadurch bleibt im Einerkomplement die Zahl Null erhalten, und der Sonderwert bleibt eindeutig [1]. Dabei ist zu beachten, dass UDP Fehler nur erkennen und nicht beheben kann; ein beschädigtes Datagramm wird typischerweise verworfen [11].

Pseudo-Header. Die UDP-Prüfsumme schützt nicht nur die Transportschichtdaten, sondern bezieht über einen *Pseudo-Header* auch Informationen der Vermittlungsschicht ein. Dieser Pseudo-Header ist kein Teil des Datagramms und steht an keiner Stelle im Paket: Sender und Empfänger bauen ihn jeweils nur für die Berechnung im Speicher auf und stellen ihn dem UDP-Header dabei gedanklich voran [1]. Dieses Konzept stellt sicher, dass ein Datagramm, das durch einen IP-Fehler an den falschen Host oder das falsche Protokoll zugestellt wird, erkannt und verworfen werden kann. Seinen Aufbau zeigt Abbildung 2.4 [1].

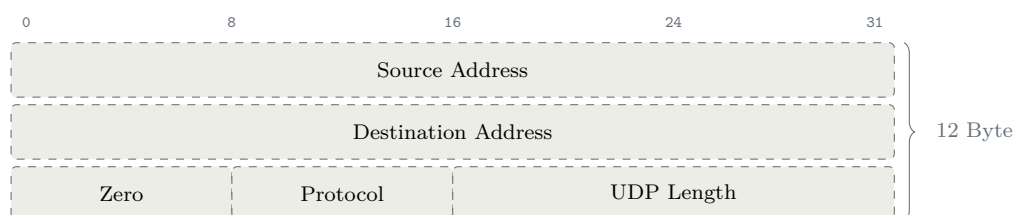


Abbildung 2.4: Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]

Aus Abbildung 2.4 geht hervor, dass der Pseudo-Header ausschließlich aus Werten besteht, die beiden Seiten ohnehin vorliegen: den beiden Adressen und dem Feld `Protocol` aus dem IP-Header (Abschnitt 2.1), einem Nullbyte als Füllung sowie einer Wiederholung des UDP-Längenfelds. Nur deshalb können Sender und Empfänger unabhängig voneinander dieselben zwölf Byte zusammensetzen; die gestrichelte Darstellung kennzeichnet, dass keines dieser Felder je übertragen wird [1].

Ein letzter Blick gilt dem Feld `Zero`: Es trägt keine Information, sondern stellt die Ausrichtung her. Die Prüfsummenarithmetik arbeitet ausschließlich auf 16-Bit-Wörtern [12], das Protokollfeld aus dem IP-Header ist aber nur 8 Bit breit; das vorangestellte Nullbyte ergänzt es zu einem vollen Wort. Nach demselben Prinzip werden auch Nutzdaten ungerader Länge für die Berechnung am Ende mit einem Nulloktett aufgefüllt [1]. Durch ein anderes Feld aus dem IP-Header, etwa `Time to Live`, lässt sich das Nullbyte dagegen nicht ersetzen: Jeder Router verringert diesen Wert beim Weiterleiten (Abschnitt 2.1); der Empfänger würde also mit einem anderen Wert rechnen als der Sender, und die Prüfung ginge selbst bei fehlerfreier Übertragung nie auf. In den Pseudo-Header gehören nur Werte, die unterwegs unverändert bleiben; ein festes Nullbyte erfüllt das von selbst.

2.2.2 Eigenschaften

Die in Abschnitt 2.2.1 beschriebene Header-Struktur verdeutlicht bereits das minimalistische Design von UDP. Aus diesem Designziel leiten sich eine Reihe funktionaler Eigenschaften ab, die maßgeblich beeinflussen, für welche Anwendungsfälle das Protokoll geeignet ist.

Nachrichtenorientierung. UDP ist ein *nachrichtenorientiertes* Protokoll: Jede Sendeoperation erzeugt genau ein Datagramm, und der Empfänger erhält jede Nachricht als atomare Einheit. TCP arbeitet dagegen *stromorientiert*: Es bietet der Anwendung einen fortlaufenden Bytestrom, den es für die Übertragung selbst in Segmente zerlegt [9, Sec. 2.2]. Sendet eine Anwendung etwa nacheinander `"hello"`, `"world"` und `"!"`, stellt UDP drei Datagramme mit genau diesen Nachrichten zu; ein TCP-Empfänger kann denselben Strom auch als `"hellowo"` und `"rld!"` erhalten [9, Sec. 3.7]. Eine Anwendung, die über TCP einzelne Nachrichten austauschen will, muss deren Grenzen deshalb selbst kennzeichnen, etwa durch Längenfelder; bei UDP entfällt das, weil jedes Datagramm genau eine Nachricht trägt.

Zustandslosigkeit. Das klassische UDP verwaltet keinen protokollspezifischen Zustand für einzelne Kommunikationsbeziehungen und hält keine Sequenznummern, Fenstergrößen oder Retransmission-Timer vor. RFC 9868 erhält dieses Grundprin-

zip, führt für die Wiederaussetzung von FRAG-Datagrammen jedoch eine begrenzte Ausnahme ein [2, Sec. 6]. Diese Ausnahme ordnet Abschnitt 2.4.5 ein.

Minimale Latenz. Da UDP verbindungslos arbeitet, entfällt jeglicher Handshake vor der Datenübertragung. Die erste Nachricht kann unmittelbar gesendet werden, ohne dass ein vorheriger Verbindungsaufbau abgewartet werden muss [8, Sec. 1].

Unabhängigkeit der Datagramme. Jedes UDP-Datagramm ist eine eigenständige, von allen anderen Datagrammen unabhängige Einheit. UDP garantiert die Zustellung nicht [1] und wiederholt verlorene Datagramme nicht; geht ein Datagramm verloren, hält UDP später eintreffende Datagramme deshalb nicht bis zu einer erneuten Übertragung zurück. Damit entfällt das sogenannte *Head-of-Line-Blocking* (HoL-Blocking): die Verzögerung, die bei einem gesicherten Bytestrom wie TCP entsteht, wenn ein verlorenes Segment die Auslieferung bereits eingetroffener Folgedaten aufhält. Anwendungen, die Zuverlässigkeit oder Reihenfolge benötigen, müssen diese oberhalb von UDP selbst herstellen [8, Sec. 3.3].

Diese Eigenschaften machen UDP zu einem attraktiven Transportprotokoll für latenzempfindliche oder verlusttolerante Anwendungen. Allerdings fehlt UDP jede Form der protokolleigenen Erweiterbarkeit. Genau diese Lücke adressiert RFC 9868 durch die Nutzung der *Surplus Area* (siehe Abschnitt 2.4), um Optionen zu transportieren, ohne den bestehenden UDP-Header zu verändern.

2.3 Erweiterungsmuster im Vergleich

Nun stellt sich die Frage, wie ein Protokoll nachträglich erweitert werden kann, dessen Header weder freie Bits noch ein Versionsfeld bietet. Das klassische Muster ist ein Optionsbereich zwischen festem Header und Nutzdaten: Ein Offsetfeld gibt an, wo die Nutzdaten beginnen, und der Raum dazwischen steht für Optionen zur Verfügung. TCP arbeitet genau so. Sein Feld `Data Offset` zählt wie die `ihl` in 32-Bit-Wörtern und ist 4 Bit breit; vom 60-Byte-Maximum bleiben nach dem festen 20-Byte-Header höchstens 40 Byte für Optionen wie Maximum Segment Size, Window Scale oder Timestamps [9]. Dasselbe Muster kennt auch IPv4: Die IP-Optionen aus Abschnitt 2.1 liegen genauso zwischen festem Header und Nutzlast. TCP und IPv4 teilen dabei sogar das Füllmuster aus den Ein-Byte-Codes `NOP` (No Operation) und `EOL` (End of Option List).

Für UDP steht dieser Weg nicht offen. Sein Header besteht aus genau vier 16-Bit-Feldern (Abschnitt 2.2.1); es gibt kein Offsetfeld, kein Versionsfeld und kein reserviertes Bit, das einen Optionsbereich ankündigen könnte. Eine Vergrößerung

oder Umdeutung des festen Headers wäre für Empfänger nach RFC 768 nicht abwärtskompatibel und scheidet damit aus.

Ein erster Ausweg verzichtet auf jede Änderung am Protokoll und legt die Optionen in die Nutzdaten selbst. Diesen Weg gehen die Erweiterungen von Teredo, einem Tunnelprotokoll, das IPv6-Pakete in UDP-Datagramme verpackt und so Hosts hinter NATs den Zugang zum IPv6-Internet öffnet [13]. Ihre Zusatzfelder stehen als *Trailer* hinter dem getunnelten Paket, noch innerhalb der UDP-Nutzdaten: Was ein Header vor den Daten ist, ist ein Trailer hinter ihnen. Weil das getunnelte Paket sein Ende selbst markiert, liest der Empfänger alles dahinter als Trailer [14]. Die Stärke dieses Ansatzes liegt auf der Hand: UDP und IP bleiben unangetastet, das Datagramm ist nach außen ein gewöhnliches Datagramm, und ein Empfänger, der nur das innere Paket auswertet, stoppt an dessen Ende und stört sich nicht an den Trailern. Die Schwäche liegt in der Voraussetzung: Das Verfahren funktioniert nur, wenn die Nutzdaten ein eigenes Längenfeld mitbringen, das ihr Ende markiert. Beliebige UDP-Anwendungen erfüllen das nicht; ihnen bliebe nur, Zusatzfelder per Konvention innerhalb der Nutzdaten von den Anwendungsdaten zu unterscheiden. Solche In-Band-Kodierung verwirft RFC 9868 jedoch, weil Altempfänger die Konvention nicht kennen und die Zusatzfelder als Nutzdaten lesen [2, Sec. 4].

Den Weg zur Surplus Area öffnet schließlich eine Eigenheit, die UDP von TCP unterscheidet: UDP trägt eine eigene Längenangabe. TCP leitet die Länge seiner Nutzlast aus der IP-Datagrammlänge ab; UDP wiederholt sie im eigenen Header [2, Sec. 4]. Damit beschreiben zwei unabhängige Angaben redundant dasselbe Ende der Nutzdaten: das UDP-Length-Feld und die IP-Längenangaben aus Abschnitt 2.1. Genau diese Redundanz überträgt den Trailer-Gedanken auf beliebige UDP-Anwendungen, denn das Längenfeld, das das Ende der Daten markiert, muss nun nicht mehr in den Nutzdaten stecken: UDP selbst bringt es mit. Bleibt UDP `Length` hinter der Nutzlast des IP-Datagramms zurück, entsteht hinter den Nutzdaten Platz, den ein Altempfänger bei herkömmlicher UDP-Verarbeitung nicht zu Gesicht bekommt, weil er nur bis UDP `Length` liest; die dokumentierten Ausnahmen behandelt Abschnitt 2.4.1. Mit den Teredo-Trailern hat dieser Bereich nichts zu tun; beide Mechanismen bleiben deshalb kompatibel [2, Sec. 4]. Warum RFC 768 das Längenfeld überhaupt vorsieht, ist bemerkenswerterweise ungeklärt: Die kursierenden Erklärungen (mehrere UDP-Pakete je IP-Datagramm; Abgrenzung der Nutzdaten von erzwungenem Null-Padding) hält RFC 9868 für unvereinbar mit früheren UDP-Entwürfen [2, Sec. 4]. Ein Feld, dessen Zweck sich nie klärte, wird damit 45 Jahre nach seiner Definition zum Erweiterungspunkt des Protokolls.

Von den drei Mustern ist für eine abwärtskompatible Erweiterung des bestehenden UDP damit allein der Trailer-Weg gangbar: Header-Optionen scheitern am

unveränderlichen Headerformat, die In-Band-Kodierung an den Altempfängern, denen die Konvention unbekannt ist. Der Trailer dagegen bleibt für Altempfänger bei gewöhnlicher Verarbeitung unsichtbar und erbt, anders als die Header-Optionen von TCP, keine feste Obergrenze für den Optionsraum, denn seine Grenze ist das Ende des IP-Datagramms. Wie dieser Bereich im Einzelnen aufgebaut ist und welche Grenzen für ihn gelten, zeigt der folgende Abschnitt.

2.4 Die Surplus Area

2.4.1 Lage und Kompatibilität

UDP `Length` umfasst nur den UDP-Header und die UDP-Nutzdaten. Die Längenangaben des IP-Headers können einen größeren Transportbereich ausweisen (Abschnitt 2.1). RFC 9868 nennt die gesamte Nutzlast hinter dem IP-Header *IP Transport Payload* und den Teil zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms *Surplus Area*; dort liegen die UDP Options. Die Lage beider Bereiche zeigt Abbildung 2.5 an einem Beispieldatagramm mit `Total Length` 60, `IHL` 5 und UDP `Length` 32 [2, Sec. 7]:

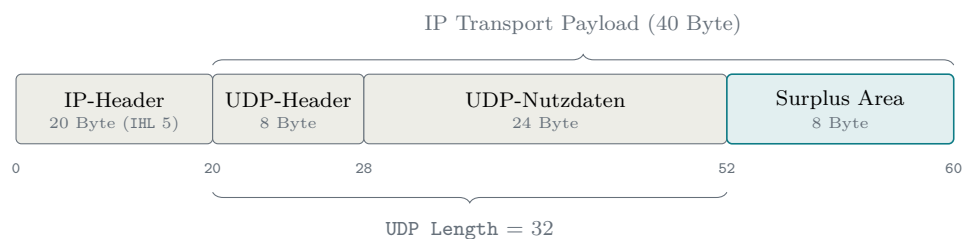


Abbildung 2.5: IP Transport Payload und Surplus Area eines Datagramms mit `Total Length` 60

Aus Abbildung 2.5 geht hervor, wie die drei Größen zusammenhängen: Die IP Transport Payload beginnt unmittelbar hinter dem IP-Header, UDP `Length` zählt ab derselben Stelle, deckt aber nur UDP-Header und Nutzdaten ab, und die Surplus Area ist der Rest bis zum Ende des IP-Datagramms; sie beginnt also bei *IP-Headerlänge* + UDP `Length`. Die Darstellung ist schematisch, alle Offsets zählen ab Byte 0 des IP-Datagramms. Im Beispiel umfasst die IP Transport Payload $60 - 20 = 40$ Byte, von denen UDP `Length` nur 32 beansprucht; die übrigen 8 Byte bilden die Surplus Area. Allgemein gilt

$$\text{Surplus-Länge} = \text{Total Length} - \text{IP-Headerlänge} - \text{UDP Length}, \quad (2.3)$$

wobei die Obergrenze aus Gleichung 2.2 bei einem gültigen Datagramm sicherstellt,

dass diese Differenz nicht negativ ist; ein Datagramm, dessen `UDP Length` die IP-Nutzlast übersteigt, ist ungültig und wird still verworfen [2, Sec. 10]. Bei gewöhnlichen UDP-Datagrammen ohne Optionen ist sie null: `UDP Length` und IP-Längenangaben enden am selben Byte, und es gibt keine Surplus Area. RFC 768 verlangt diese Gleichheit allerdings nicht; die Freiheit besteht seit seiner Veröffentlichung, RFC 9868 nutzt sie lediglich [2, Sec. 7].

Aus dieser Lage folgt unmittelbar die Kompatibilitätsidee der Erweiterung. Ein Empfänger, der UDP Options nicht kennt, verarbeitet nach RFC 768 nur die durch `UDP Length` begrenzten Nutzdaten und überliest die Surplus Area. Getestete Standardsysteme verhalten sich genau so; der RFC dokumentiert aber auch Bestandssysteme, die abweichen, etwa ein eingebettetes Gerät, das das gesamte IP-Datagramm durchreicht, und ein Erkennungssystem, das die Längendifferenz als Angriff meldet [2, Sec. 18]. Für Endsysteme bleibt die Erweiterung damit typischerweise abwärtskompatibel; das Verhalten realer Bestands- und Zwischensysteme misst die Evaluation (Kapitel 6). Auch die Vermittlungsschicht bleibt unberührt: RFC 9868 definiert keine neuen IP-Header-Felder und ändert die IPv4-Verarbeitung nicht. Für die IP-Schicht ist die Surplus Area gewöhnliche IP Transport Payload innerhalb der durch `Total Length` begrenzten Datagramm-Größe. RFC 9868 ist damit ein reines Transport-Feature, das ausschließlich die von standardkonformem UDP bisher ungenutzte Differenz zwischen `UDP Length` und dem Ende des IP-Datagramms belegt [2, Sec. 16].

Für den Beginn der Surplus Area schreibt RFC 9868 dabei bewusst keine Ausrichtung vor: Sie darf an jedem gültigen Byte-Offset anfangen, auch an einem ungeraden. Die Nutzdaten bleiben dadurch unangetastet, und das Ausrichtungsproblem wird innerhalb der Surplus Area gelöst (Abschnitt 2.4.2). Der RFC beschreibt diese Doppelrolle mit dem Begriff *trailer options offset*: `UDP Length` bleibt eine Längenangabe, ihr Endpunkt markiert aber zusätzlich den Beginn der Surplus Area [2, Sec. 7]. Genau darin liegt der Kern der Aktualisierung von RFC 768: Kein Feld ändert sein Format, ein Feld erhält eine zusätzliche Bedeutung [2, Sec. 21].

2.4.2 Aufbau und Ausrichtung

Trägt die Surplus Area Optionen, ist sie vollständig strukturiert; ungedeutete Restbytes gibt es dann nicht. Sie beginnt mit der *Option Checksum* (OCS), einem 2 Byte großen Prüfsummenfeld an der ersten 2-Byte-Grenze der Area, gemessen am Beginn des IP-Datagramms. Beginnt die Surplus Area an einem ungeraden Offset, stellt genau ein Nullbyte vor dem OCS diese Ausrichtung her (das *Pad*). Hinter dem OCS folgt die Optionsliste im TLV-Format (Type-Length-Value, Abschnitt 2.4.4); bei gewöhnlichen Datagrammen läuft sie bis zum Ende der Surplus Area oder endet vor-

zeitig mit EOL, worauf nur noch Nullbytes folgen [2, Sec. 8]. Nur beim UDP-Fragment endet sie früher, denn dort folgt hinter den Optionen noch der Fragmentdatenbereich (Abschnitt 2.4.5). Diese Anatomie zeigt Abbildung 2.6 an einem Datagramm mit 5 Byte Nutzdaten und einer einzelnen Option:

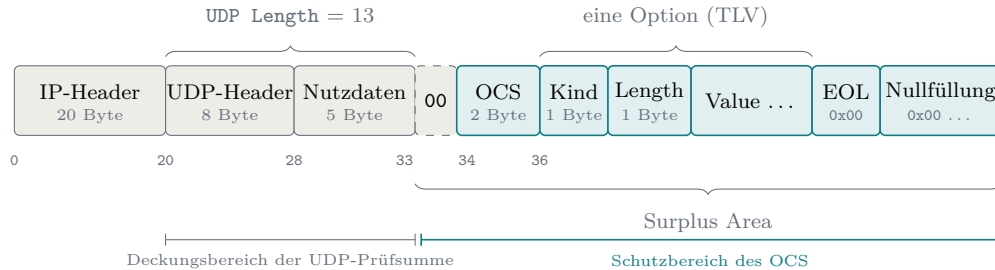


Abbildung 2.6: Anatomie eines Datagramms mit UDP Options

Aus Abbildung 2.6 geht hervor, wie die Bausteine ineinandergreifen: `UDP Length` = $8 + 5 = 13$ lässt die Surplus Area bei Byte $20 + 13 = 33$ beginnen. Dieser Offset ist ungerade, also rückt ein Pad-Byte das OCS auf die 2-Byte-Grenze bei Byte 34, und die erste Option beginnt bei Byte 36. Die unterste Ebene der Abbildung zeigt zugleich, dass sich die beiden Prüfbereiche im Datagramm lückenlos ergänzen: Die UDP-Prüfsumme endet an `UDP Length` (hinzu kommt nur ihr Pseudo-Header, Abschnitt 2.2.1), das OCS schützt genau den Teil, den die UDP-Prüfsumme nie sieht [2, Sec. 8].

Für diese Stelle formuliert RFC 9868 verbindliche Regeln [2, Sec. 8]: Der Sender darf Optionen an jedem gültigen `UDP-Length`-Offset beginnen lassen, und Ausrichtungsbytes vor dem OCS müssen null sein. Findet der Empfänger dort einen anderen Wert, greift zum ersten Mal eine Grundregel, die alle weiteren Prüfungen dieses Abschnitts teilen: Im Standardfall kostet ein Fehler in der Surplus Area nur die Optionen. Der Empfänger verwirft die Optionsliste still und stellt die Nutzdaten trotzdem zu, wie es auch ein Altempfänger täte [2, Sec. 8 und 9]. Standardfall heißt dabei: Das Datagramm selbst ist gültig, denn eine falsche `UDP Length` oder eine fehlgeschlagene UDP-Prüfsumme verwirft es komplett, noch bevor Optionen betrachtet werden, während eine zulässig ungenutzte Prüfsumme (Abschnitt 2.2.1) kein Fehler ist [2, Sec. 10 und 14]; die Anwendung hat kein strengeres Verhalten angefordert; und es ist kein UDP-Fragment, denn die Sonderwege von FRAG und UNSAFE behandelt Abschnitt 2.4.5. Die 2-Byte-Ausrichtung selbst begründet der RFC pragmatisch: Das OCS wird, wie der nächste Unterabschnitt zeigt, über 16-Bit-Wörter berechnet, und die Ausrichtung erspart dieser Rechnung das Vertauschen von Bytes [2, Sec. 8].

Ob ein Pad-Byte nötig ist, entscheidet allein die Parität des Startoffsets; es gibt nur zwei Fälle, kein Pad oder genau eines. Den Unterschied zeigt Abbildung 2.7

an zwei sonst gleichen Datagrammen, deren Nutzdaten sich um ein einziges Byte unterscheiden:

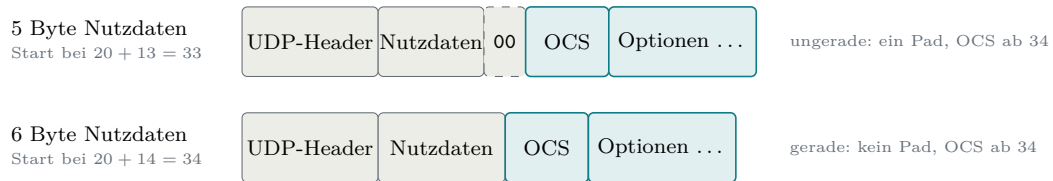


Abbildung 2.7: Paritätsentscheid für das Pad-Byte vor dem OCS

Aus Abbildung 2.7 geht hervor, dass bereits ein einziges Nutzdatenbyte über das Pad entscheidet: Bei einem IP-Header ohne Optionen liegt der Startoffset bei $20 + \text{UDP Length}$, sodass allein die Parität von `UDP Length` den Ausschlag gibt. An der Größe der Surplus Area hängt zugleich, ob sie überhaupt Optionen trägt: Nötig sind mindestens 2 Byte bei geradem und 3 Byte bei ungeradem Start, damit das ausgerichtete OCS vollständig hineinpasst. Ein kleinerer Überschuss enthält schlicht keine Optionen und ist kein Fehlerfall; passt genau das OCS hinein und sonst nichts, ist die Optionsliste leer und das Datagramm ebenfalls gültig [2, Sec. 7 und 8]. Nutzdaten braucht es dagegen keine: Auch ein Datagramm mit `UDP Length` 8, also leerem Nutzdatenteil, kann Optionen tragen [2, Sec. 8]; davon macht später die FRAG-Option Gebrauch, deren Fragmente ihre Daten vollständig in der Surplus Area transportieren.

2.4.3 OCS und Fehlerbehandlung

Das OCS selbst ist eine gewöhnliche Internet-Prüfsumme mit derselben Einerkomplementarithmetik, die Abschnitt 2.2.1 für die UDP-Prüfsumme beschreibt [12]. Zwei Eigenheiten unterscheiden die beiden. Erstens schützt das OCS genau den Bereich, den die UDP-Prüfsumme ausspart: Die Summe läuft ab dem ausgerichteten OCS-Feld über den Rest der Surplus Area, wobei das OCS-Feld selbst als null gilt; das Pad davor wird getrennt auf null geprüft und ist so mitgeschützt [2, Sec. 8 und 9]. Zweitens geht zusätzlich die Länge der gesamten Surplus Area einschließlich des Pads als vorzeichenloser 16-Bit-Summand in die Rechnung ein, obwohl dieser Wert in keinem eigenen Feld des Pakets steht [2, Sec. 9]. Das folgt demselben Gedanken wie der Pseudo-Header aus Abschnitt 2.2.1: Wie die UDP-Prüfsumme dort die `UDP Length` einbezieht, bezieht das OCS die Surplus-Länge ein, und beide Seiten können diesen Summanden unabhängig voneinander aus den Längenfeldern ableiten. Der Sender trägt das invertierte Ergebnis in das Feld ein; ergibt die Invertierung gerade `0x0000`, überträgt er das im Einerkomplement gleichwertige `0xFFFF`, denn der Nullwert bleibt der Kennzeichnung eines ungenutzten OCS vorbehalten [2, Sec. 9]. Der Empfänger summiert die Wörter der Surplus Area einschließlich des OCS-Felds

sowie den Längensummanden und erwartet wie in Abschnitt 2.2.1 lauter Einsen, also 0xFFFF [2, Sec. 9].

Ein bewusst kleines Rechenbeispiel führt diese Rechnung einmal vollständig vor: Eine Surplus Area von 6 Byte beginne an gerader Grenze und enthalte hinter dem OCS zwei NOP, ein EOL und ein Nullbyte (die beiden Ein-Byte-Codes kennt Abschnitt 2.3), als 16-Bit-Wörter gelesen also 0x0101 und 0x0000. Der Sender addiert dazu den Längensummanden 0x0006, erhält 0x0107 und trägt das invertierte Ergebnis 0xFEf8 als OCS ein. Der Empfänger summiert 0xFEf8, 0x0101, 0x0000 und 0x0006 zu 0xFFFF: Die Prüfung geht auf, obwohl die Länge in keinem eigenen Feld übertragen wurde. Die NOP-Doppelung dient dabei allein der Anschaulichkeit der Rechnung; als Füllung vor EOL rät der RFC von NOP gerade ab (Abschnitt 2.4.4), ein regulärer Sender schreibe das EOL unmittelbar hinter das OCS [2, Sec. 11.1].

Diese Bauart hat einen praktischen Hintergrund: Manche fehlerhafte Middleboxes berechnen die UDP-Prüfsumme über die gesamte IP-Nutzlast statt nur bis UDP Length. Weil das OCS über die Surplus Area und deren Länge gebildet und anschließend invertiert wird, neutralisiert es den Beitrag der fälschlich einbezogenen Surplus Area: Mit gesetztem OCS fällt die UDP-Prüfsumme in beiden Rechnungen gleich aus; an dieser fehlerhaften Ausdehnung des Prüfbereichs scheitert das Datagramm bei solchen Geräten also nicht [2, Sec. 9]. In erster Linie dient das OCS ohnehin dem Erkennen zufälliger Fehler und anderweitiger, nicht standardkonformer Nutzungen der Surplus Area; ein Schutz gegen gezielte Angriffe ist es ausdrücklich nicht [2, Sec. 9]. Wie die UDP-Prüfsumme ist das OCS zudem bedingt abschaltbar: Der Wert 0x0000 bedeutet ungenutzt, zulässig ist das nur, wenn auch die UDP-Prüfsumme null ist, und Implementierungen müssen standardmäßig ein echtes OCS setzen [2, Sec. 9]. Schlägt die Prüfung beim Empfänger fehl, gilt die Grundregel aus Abschnitt 2.4.2: Optionen still verwerfen, Nutzdaten bei bestandener oder zulässig ungenutzter UDP-Prüfsumme dennoch zustellen [2, Sec. 9].

2.4.4 TLV-Format und Empfangsentscheidung

Die Optionen hinter dem OCS folgen dem TLV-Muster, dessen Syntax RFC 9868 bewusst an TCP anlehnt (Abschnitt 2.3): Ein 1 Byte großes Feld Kind, benannt nach dem TCP-Vorbild, identifiziert die Option; das 1 Byte große Feld Length zählt die Gesamtlänge der Option einschließlich Kind und Length selbst; dahinter kann ein Wert folgen [2, Sec. 8 und 10]. Für die Größe des Werts ist das nur scheinbar eine enge Grenze: Im Standardformat fasst eine Option höchstens 254 Byte einschließlich Kind und Length; größere Optionen weichen auf ein erweitertes Format aus, in dem der Wert 255 im Length-Feld ein zusätzliches 16-Bit-Längenfeld ankündigt. Damit sind je Option höchstens 65535 Byte einschließlich der Kopffelder kodierbar

[2, Sec. 10]. Eine zusätzliche Gesamtgrenze für den Optionsraum kennt RFC 9868 dagegen bewusst nicht: Wie das Entwurfsprinzip der fehlenden Längengrenze in Abschnitt 2.4.5 festhält, gilt allein die Grenze des UDP-Pakets selbst; bei einem nicht fragmentierten Datagramm begrenzt somit der im IP-Datagramm verfügbare Platz, wie viele Optionen es trägt. Auch die beiden Ein-Byte-Codes aus Abschnitt 2.3 kehren als einzige Ausnahmen ohne Längenfeld wieder: `NOP` (Kind 1) dient mit impliziter Länge eins der Ausrichtung zwischen Optionen und darf sich dafür wiederholen, `EOL` (Kind 0) beendet die Liste vorzeitig; die Bytes dahinter muss der Sender bis zum Ende der Surplus Area beziehungsweise des Optionsbereichs eines UDP-Fragments auf null setzen [2, Sec. 8 und 11.1]. Der Empfänger darf diese Nullfüllung prüfen, muss es aber nicht; findet er dabei ein Byte ungleich null, verwirft er nach der Grundregel aus Abschnitt 2.4.2 die gesamte Optionsliste still [2, Sec. 11.1]. Variable Optionslängen und künftige Erweiterungen sind damit im Format selbst angelegt.

Für die Anzahl der Optionen gilt dasselbe Bild wie für ihre Länge: Eine feste Obergrenze gibt es nicht; ohne vorzeitiges `EOL` endet die Liste erst am Ende der Surplus Area. Begrenzend wirken stattdessen zwei Regelgruppen. Erstens die Wiederholungsregeln: Außer `FRAG`, `NOP` und den Experimentaloptionen soll jede Option höchstens einmal je Datagramm vorkommen, ein Empfänger wertet von einer solchen Option ohnehin nur die erste Instanz, und ein mehrfaches `FRAG` macht die gesamte Optionsliste ungültig [2, Sec. 10]; `NOP` wiederum soll höchstens siebenmal in Folge stehen und nicht als Ersatz für `EOL` samt Nullfüllung dienen [2, Sec. 11.2]. Zweitens die Vollständigkeitsregel: Jede Option muss ganz in die Surplus Area passen. Meldet ein `Length`-Feld eine Länge über deren Ende hinaus, gilt die gesamte Surplus Area als fehlerhaft, und der Empfänger verwirft still alle Optionen, nicht nur die angeschnittene [2, Sec. 10]; das verifizierte Erratum EID 8834 bestätigt diese Regel gegen eine ältere, weichere Formulierung in Abschnitt 14 [15]. Im Regelfall entsteht diese Lage nicht, denn der Sender muss die Surplus Area von vornherein so bemessen, dass das OCS und alle Optionen vollständig hineinpassen [2, Sec. 8].

Damit sind die Prüfschritte des Standardpfads beisammen. Abbildung 2.8 ordnet sie in der Reihenfolge, in der ein optionsfähiger Empfänger sie durchläuft, und stellt ihnen die vorgelagerte Prüfung des Datagramms selbst voran:

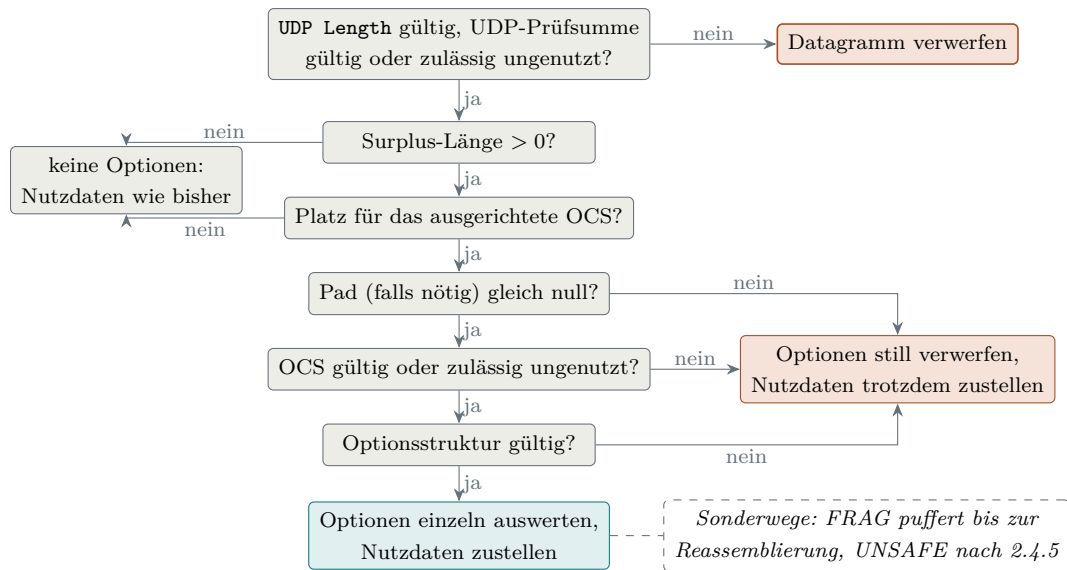


Abbildung 2.8: Empfangsentscheidung für die Surplus Area im Standardfall

Aus Abbildung 2.8 geht hervor, dass nur die vorgelagerte Prüfung des Datagramms selbst Nutzdaten kosten kann: Eine ungültige `UDP Length` oder eine falsche UDP-Prüfsumme verwirft das ganze Datagramm [2, Sec. 10 und 14]. Alle Prüfungen der Surplus Area treffen dagegen höchstens die Optionen: Die beiden Größenfragen führen auf den harmlosen Ausgang links, die drei Prüfungen dahinter auf den Fehlerausgang rechts; genau das ist die Grundregel aus Abschnitt 2.4.2 im Bild. Bei der Auswertung selbst wird einzeln ignoriert, was nur lokal stört: eine unbekannte `SAFE`-Option ebenso wie spätere Instanzen einer gewöhnlichen wiederholten Option. Die ganze Liste ist bei den strukturellen Längenfehlern aus dem vorigen Absatz und bei mehrfachem `FRAG` ungültig; daneben darf der Empfänger sie verwerfen, wenn verpflichtend zu unterstützende Optionen (im RFC *must-support*, ausgenommen `NOP` und `EOL`) nicht vor den übrigen `SAFE`-Optionen stehen, und er muss es, wenn seine freiwillige Nullfüllungsprüfung hinter `EOL` ein Byte ungleich null findet [2, Sec. 10 und 11.1]. Die gestrichelte Randnotiz markiert die Sonderwege: Ein UDP-Fragment wird erst gepuffert und nach der Reassemblierung zugestellt; `UNSAFE`-Optionen und strengere, von der Anwendung angeforderte Empfangsregeln behandelt der folgende Unterabschnitt.

2.4.5 Entwurfsprinzipien und Optionsklassen

Wie dieser Bereich genutzt werden darf, bindet RFC 9868 an sechs Entwurfsprinzipien, die ein gemeinsames Ziel haben: UDP soll durch die Optionen seinen Charakter nicht verlieren [2, Sec. 6]:

- **Zustandslos:** Nötiger Zustand gehört in die Anwendung oder in eine Bibliothek, die in ihrem Auftrag arbeitet. Die einzige, ausdrücklich begrenzte Ausnahme

ist die Zusammenführung von Fragmenten.

- **Unidirektional:** Die UDP-Schicht erzeugt nie von sich aus eine Antwort auf eine Option; ob und wann geantwortet wird, entscheidet die Anwendung oder eine Schicht beziehungsweise Bibliothek in ihrem Auftrag. Verfahren, die auf Antworten angewiesen sind, verlagert der RFC in eigene Dokumente. Als Einordnung dieser Arbeit, nicht als Begründung des RFC: Diese Zurückhaltung verhindert, dass schon der Grundmechanismus mit gefälschter Absenderadresse als Reflektor missbraucht wird; ein ausdrücklich aktiviertes Antwortverfahren muss diesen Missbrauch selbst abwehren, etwa durch die Ratenbegrenzung, die RFC 9869 für eigens als Antwort erzeugte Datagramme vorschreibt [3, Sec. 3.1].
- **Keine eigene Längengrenze:** Für den Optionsraum gilt allein die Grenze des UDP-Pakets selbst; bei einem nicht fragmentierten Paket ist das der im IP-Datagramm verfügbare Platz. Feste Grenzen werden erfahrungsgemäß überschritten; der RFC verweist auf die Erfahrung mit TCP-Optionen (40 Byte Optionsraum [9]) und IPv4-Adressen (32 Bit [5]). Grenzen darf eine Implementierung setzen, nicht die Spezifikation.
- **Kein Protokollersatz:** Optionen liefern Funktionen für den eigenen Verkehr einer Anwendung; sie sollen NTP, ICMP (insbesondere Echo) und Netzwerk-Testgeräte weder ersetzen noch nachbauen.
- **Rahmenwerk statt Protokoll:** RFC 9868 definiert Optionsformate auch dann, wenn sie noch kein vollständiges Verfahren ergeben; die Nutzung regeln andere Dokumente. Für REQ/RES übernimmt das RFC 9869 [3], für TIME bleibt sie bewusst offen. Das entspricht dem Muster von TCP, dessen Basisnorm den Optionsmechanismus festlegt, während einzelne Optionen in eigenen Dokumenten folgten.
- **Voreinstellung wie beim Altempfänger:** Empfangene Pakete werden standardmäßig auch dann zugestellt, wenn Prüfsummen-, Authentifizierungs- oder Entschlüsselungsprüfungen der Optionen scheitern; einzige Ausnahme sind UDP-Fragmente. Strengeres Verhalten muss die Anwendung ausdrücklich anfordern. Optionen sind ein Angebot, kein Filter.

Der gemeinsame Nenner der sechs Prinzipien: RFC 9868 legt die Optionsformate und ihre Grundverarbeitung fest, aber kein vollständiges Anwendungsprotokoll. UDP bleibt ein minimales Transportprotokoll, und jede zusätzliche Fähigkeit wird von der Anwendung ausdrücklich aktiviert [2, Sec. 6].

An das letzte Prinzip knüpft die zentrale Zweiteilung der Optionen an, und sie folgt unmittelbar aus der Abwärtskompatibilität: Weil ein Altempfänger die Surplus

Area überliest, muss jede Option damit rechnen, ignoriert zu werden. *SAFE*-Optionen (Kind 0 bis 191) sind genau dafür entworfen: Ein Empfänger, der sie nicht versteht, ignoriert sie, ohne dass sich die Bedeutung der Nutzdaten ändert. In diese Klasse gehören die Ein-Byte-Codes *NOP* und *EOL* ebenso wie *FRAG* [2, Sec. 10 und 11].

UNSAFE-Optionen (Kind 192 bis 255) können dagegen die Nutzdaten verändern oder eine Änderung ihrer Deutung signalisieren, etwa durch Kompression oder Verschlüsselung; ein Empfänger, der die Option nicht versteht, würde die Nutzdaten falsch deuten. RFC 9868 löst das mit einer Transportbeschränkung: *UNSAFE*-Optionen dürfen nur in UDP-Fragmenten auftreten, deren regulärer Nutzdatenteil leer ist; die transportierten Daten liegen im Fragmentdatenbereich der Surplus Area hinter der Optionsliste [2, Sec. 11.4]. Ein Altempfänger sieht dann nur ein leeres Datagramm und verwirft die Daten still, statt sie falsch zu deuten. Ein optionsfähiger Empfänger, der eine *UNSAFE*-Option nicht unterstützt, muss die reassemblierten Nutzdaten ebenso still verwerfen, und dasselbe gilt, wenn eine *UNSAFE*-Option außerhalb eines Fragment- oder Reassemblierungskontexts erscheint; zugestellt wird in beiden Fällen höchstens ein leeres Datagramm [2, Sec. 10, 12 und 14]. Die einzelnen Optionen und ihre Pflichten behandelt Kapitel 4.

Zwei Folgen dieser Prinzipien verdienen eine eigene Erwähnung. Erstens verteilt die *FRAG*-Option ein logisches Anwendungsdatagramm (im RFC *user datagram*) künftig gegebenenfalls auf mehrere IP-Datagramme. Zweitens nimmt der RFC einen ausdrücklich benannten Unterschied in Kauf: Ein Altempfänger sieht jedes UDP-Fragment als leeres Datagramm, ein optionsfähiger Empfänger das reassemblierte Datagramm mit Inhalt. Diese Zählungen gleicht der RFC nicht an, weil leere Datagramme als Lebenszeichen ohnehin unzuverlässig sind [2, Sec. 6 und 18].

2.5 Betriebssystem und Netzpfad

Ob ein Datagramm mit Surplus Area sein Ziel erreicht, entscheidet sich an zwei Stellen außerhalb des Protokolls: im Betriebssystem der beiden Endpunkte und auf dem Netzpfad dazwischen. Betrachtet wird dabei durchgehend der Linux-Netzwerkstapel für IPv4, auf dem die Referenzimplementierung dieser Arbeit läuft; andere Betriebssysteme und IPv6 bleiben ausgeklammert. Dieser Abschnitt stellt zunächst die zwei Socket-Sichten vor, mit denen der Kernel UDP-Verkehr an Anwendungen übergibt, und anschließend die Akteure des Pfades; am Ende fügt Abbildung 2.12 beides zu einem Gesamtbild zusammen. Allgemeine Netzwerkgrundlagen bleiben bewusst ausgespart: Es geht genau um die Kette, die die Messpfade dieser Arbeit tatsächlich durchlaufen.

Zwei Socket-Sichten. Der gewöhnliche Zugang zu UDP ist der *UDP-Socket*. Bei ihm übernimmt der Kernel die gesamte Protokollarbeit: Beim Empfang prüft er die Längenangaben, validiert eine vorhandene UDP-Prüfsumme, ordnet das Datagramm anhand von Zieladresse, Zielpport und weiteren Socket-Merkmalen der richtigen Anwendung zu und übergibt ihr genau die Nutzdaten bis zur Grenze von `UDP Length` [4]. Eine Surplus Area bekommt die Anwendung nie zu sehen; ein Empfänger, der nur diese Sicht nutzt, ist damit genau der Altempfänger aus Abschnitt 2.4, dessen Abschneiden an `UDP Length` RFC 9868 für getestete Standardsysteme dokumentiert [2, Sec. 18]. Beim Senden gilt das Gegenstück: Der Kernel bemisst das IP-Datagramm stets exakt auf das UDP-Datagramm, hinter den Nutzdaten bleibt kein Platz. Ein UDP-Socket kann eine Surplus Area also weder erzeugen noch lesen. Wie dieser Normalweg im Kernel aussieht, zeigt Abbildung 2.9 mit den wichtigsten Funktionen je Schicht für beide Richtungen:

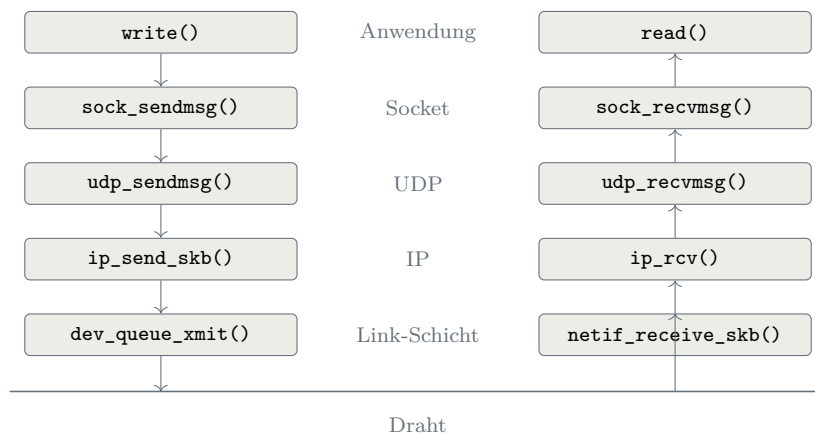


Abbildung 2.9: Sende- und Empfangsweg eines UDP-Datagramms durch den Linux-Kernel (nach [4])

Aus Abbildung 2.9 geht hervor, dass jede Schicht des Sendewegs ihr Gegenstück im Empfangsweg hat²: `udp_sendmsg()` koordiniert den Sendevorgang, den UDP-Header samt Längenfeld schreibt seine Hilfsfunktion `udp_send_skb()`, die das fertige Datagramm an die IP-Schicht übergibt [16]; `udp_rcvmsg()` liefert erst aus, nachdem der Kernel die UDP-Prüfsumme validiert hat [4]. Für diese Arbeit entscheidend ist, wo die zweite Sicht ansetzt: Der Raw-Sender speist sein fertig gebautes Datagramm eine Schicht tiefer ein, auf Höhe der IP-Schicht, und der Raw-Empfänger erhält seine Kopie, bevor der UDP-Pfad beginnt. Beide Abzweige zeichnen die folgenden Abbildungen nach.

Wer beides will, braucht die zweite Sicht: den *Raw Socket*, den Linux nur Prozessen

² Gegenüber der Vorlage (dort Abbildung 5) folgt die Abbildung dem Text der Studie und dem Kernel Quelltext: Die Socket-Schicht ist mit `sock_sendmsg()` und `sock_rcvmsg()` beschriftet, und der UDP-Sendepfad übergibt an `ip_send_skb()`, nicht an das dort gezeigte `ip_queue_xmit()` [16].

mit der Berechtigung `CAP_NET_RAW` öffnet [17]. Er arbeitet unterhalb von UDP direkt mit IP-Datagrammen: Die Anwendung schreibt den UDP-Header selbst, und die IP-Längenangabe bemisst der Kernel an der übergebenen Pufferlänge statt am UDP-Datagramm; alles, was die Anwendung hinter das UDP-Datagramm in den Puffer legt, wird so zur Surplus Area [17]. Mit der Socket-Option `IP_HDRINCL` baut sie darüber hinaus den IP-Header selbst, statt ihn vom Kernel erzeugen zu lassen [17]; diesen Weg nutzt die Referenzimplementierung, die damit das vollständige Datagramm selbst baut (Abschnitt 4.5). Die vertauschten Rollen beider Sendewege zeigt Abbildung 2.10:

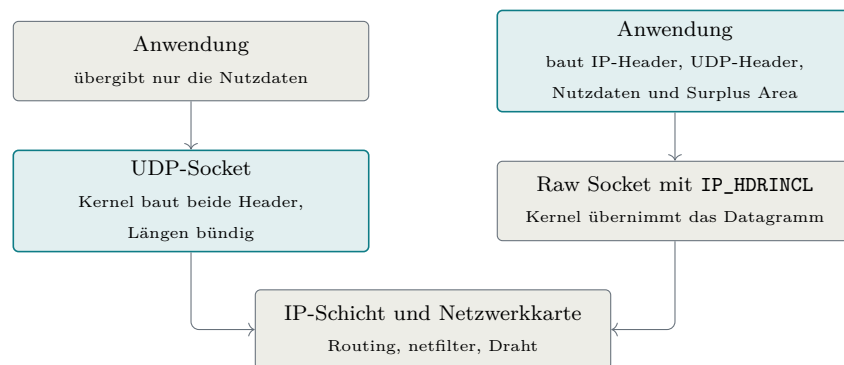


Abbildung 2.10: Die zwei Sendepfade durch den Linux-Kernel [4, 17]

Aus Abbildung 2.10 geht hervor, dass die Arbeitsteilung kippt: Am UDP-Socket baut der Kernel beide Header und bemisst die Längen bündig, am Raw Socket liefert die Anwendung das fertige Datagramm ab, und nur auf dem Raw-Socket-Weg entsteht hinter den Nutzdaten Platz für eine Surplus Area. Der Unterbau bleibt derselbe: Beide Wege münden in die IP-Schicht mit ihren netfilter-Stationen und enden an der Netzwerkkarte [4].

Ganz entlässt der Kernel den Raw-Sender allerdings nicht aus seiner Obhut, und diese Restarbeit prägt den Sendepfad der Implementierung. Routing, Nachbarauflösung und Link-Schicht bleiben Kernelsache, und im übergebenen IP-Header trägt der Kernel `Total Length` und die IP-Header-Prüfsumme stets selbst ein; UDP-Header und Surplus Area lässt er selbst dagegen unangetastet, die netfilter-Stationen dieses Wegs können das Datagramm gleichwohl noch verwerfen oder verändern [4, 17]. Zugleich fragmentiert dieser Pfad nicht: Mit `IP_HDRINCL` ist ein Datagramm auf die MTU des Interfaces begrenzt, ein größerer Sendeaufwurf schlägt fehl [17]. Für RFC 9868 ist das kein Verlust: IP-Fragmentierung sollen UDP-Anwendungen ohnehin vermeiden (Abschnitt 2.1) [8, Sec. 3.2], und für große Nutzlasten stellt der RFC stattdessen die FRAG-Option auf Transportebene bereit.

Beim Empfang entscheidet sich, an welcher Stelle ein Datagramm geprüft wird; den Weg vom Draht zu den beiden Sichten zeichnet Abbildung 2.11 nach:

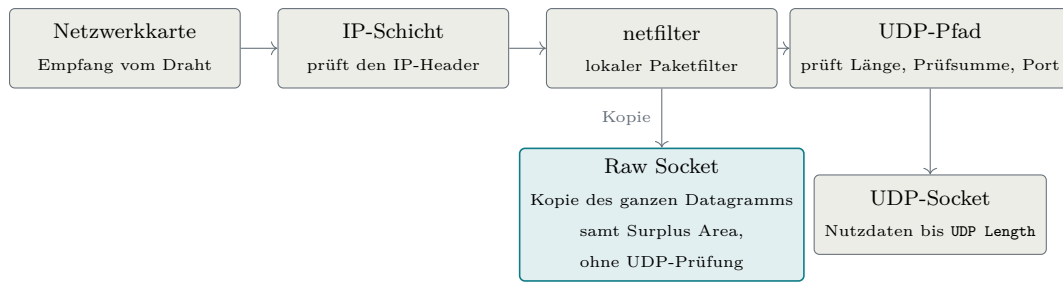


Abbildung 2.11: Der Empfangspfad durch den Linux-Kernel [4, 17]

Aus Abbildung 2.11 geht hervor, dass sich der Weg hinter dem Paketfilter gabelt: Der Raw Socket erhält eine Kopie des gesamten IP-Datagramms samt Surplus Area, und zwar bevor der UDP-Pfad des Kernels Länge, Prüfsumme und Portzuordnung geprüft hat [4, 17]. Ungeprüft heißt dabei: frei von jeder UDP-Prüfung; den IP-Header hat die IP-Schicht zu diesem Zeitpunkt dagegen bereits geprüft, wie die Abbildung zeigt. Ein Datagramm mit falscher UDP-Prüfsumme oder einer `UDP Length`, die über die IP-Nutzlast hinausweist, erreicht den Raw Socket deshalb unverändert. Die Prüfungen, die der Kernel dem UDP-Socket abnimmt, muss eine Raw-Empfangsschicht deshalb vollständig selbst ausführen, und zwar in der Reihenfolge, die RFC 9868 vorgibt: erst Mindestlänge und Längenkonsistenz des UDP-Headers, dann die UDP-Prüfsumme, erst danach Surplus-Ortung und OCS [2, Sec. 10 und 14]; wie die Implementierung das umsetzt, zeigt Kapitel 5. Zwei Arbeiten behält der Kernel dagegen auch hier. Die netfilter-Hooks bleiben beiden Sichten vorgeschaltet: Ein lokaler Paketfilter kann ein Datagramm verwerfen, bevor irgendeine Anwendung es sieht [4]. Und ankommende IP-Fragmente setzt die IP-Schicht bereits vor der Zustellung wieder zusammen, auch für Raw Sockets; eine Raw-Empfangsschicht muss die IPv4-Reassemblierung deshalb nicht nachbauen [4, 17].

Akteure auf dem Netzpfad. Zwischen den Endpunkten begegnet das Datagramm einer zweiten Gruppe von Akteuren. Der Sammelbegriff dafür lautet *Middlebox*: ein Zwischensystem, das Pakete nicht nur weiterleitet, sondern liest, verändert oder verwirft [8, Sec. 3.5]. Für die Interpretation der späteren Messungen genügt der Sammelbegriff nicht, denn die Akteure setzen an verschiedenen Stellen an und hinterlassen verschiedene Spuren. Im Zugangsnetz ersetzt die NAT eines Heimrouters oder Hotspots Adressen und Ports und bindet jeden Fluss an einen Zustandseintrag mit Zeitschranke; ihre Provider-Variante *Carrier-Grade NAT* (CGNAT) teilt dieselben öffentlichen Adressen zusätzlich unter vielen Kunden auf. Firewalls verwerfen an Netzgrenzen und Endsystemen nach Regelwerk, etwa anhand von Ports und Protokollen. Leicht zu übersehen sind die letzten beiden Akteure: die Virtualisierungsschicht eines Testrechners, deren eigener Netz- und NAT-Stack Kopffelder auf erwartete Normwerte umschreiben kann, und das Transitnetz selbst, die AS-Kette zwischen

den Zugangsnetzen, in der einzelne Netze filtern.

Ob ein einzelner Akteur UDP Options kennt, lässt sich von außen nicht feststellen; für die in dieser Arbeit beobachteten Eingriffe war eine bewusste Verarbeitung der Optionen aber nirgends nachweisbar: Was mit einer Surplus Area geschieht, erscheint durchweg als Nebenwirkung von Regeln, die für gewöhnlichen UDP-Verkehr geschrieben wurden. Wie verschieden diese Nebenwirkungen ausfallen, zeigen drei Befunde, die Kapitel 6 belegt: Der geprüfte Mobilfunk-Hotspot-Pfad verwirft Datagramme mit Surplus Area in beiden Richtungen, der NAT-Übergang einer Virtualisierungsschicht normalisiert die Kopffelder und entfernt die Surplus Area dabei, und geprüfte Transitpfade zwischen Rechenzentren transportieren sie unverändert. Zwei Eigenschaften prägen darüber hinaus die Messungen. Erstens arbeiten mehrere Akteure mit Zustand und Zeitschranken; derselbe Pfad kann sich je nach Vorgeschichte unterschiedlich verhalten [8, Sec. 3.5]. Zweitens lässt sich die Kette gezielt umgehen: Eine Kapselung, etwa in einen WireGuard-Tunnel, verbirgt das innere Datagramm vor den Akteuren zwischen den Tunnelendpunkten, die nur noch das äußere Tunnelpaket behandeln. Kapitel 6 nutzt genau das als Gegenprobe; sie belegt die Zustellung nach der Entkapselung, nicht einen nativen Transport über den offenen Pfad.

Diese Kette erklärt zugleich, warum RFC 9868 seine Optionen ausgerechnet in einem Bereich ablegt, den Altempfänger nie lesen. Weil Middleboxes bevorzugt durchlassen, was sie kennen, weichen Protokollerweiterungen in Bereiche aus, die Bestandssysteme nicht deuten; dieses Erstarren des Netzes unter seinen Zwischensystemen wird als *Ossifikation* bezeichnet. Unsichtbar ist die Surplus Area allerdings nur für die Endpunkte alter Anwendungen; für die Geräte auf dem Pfad bleibt sie ohne Verschlüsselung sichtbar [2, Sec. 16], und RFC 9868 rechnet ausdrücklich mit deren Eingriffen: Fehlerhaft rechnende Middleboxes begründen Details der OCS-Berechnung (Abschnitt 2.4) [2, Sec. 9], und UDP-Relays können eine Surplus Area beim Weiterleiten vollständig abstreifen [2, Sec. 25.6].

Alle Stationen zusammen ordnet Abbildung 2.12 zu einem Messmodell des Weges von der sendenden Anwendung bis zu dem Raw-Empfänger, der die Surplus Area auswertet; je nach Pfad können einzelne Stationen fehlen oder zusammenfallen.

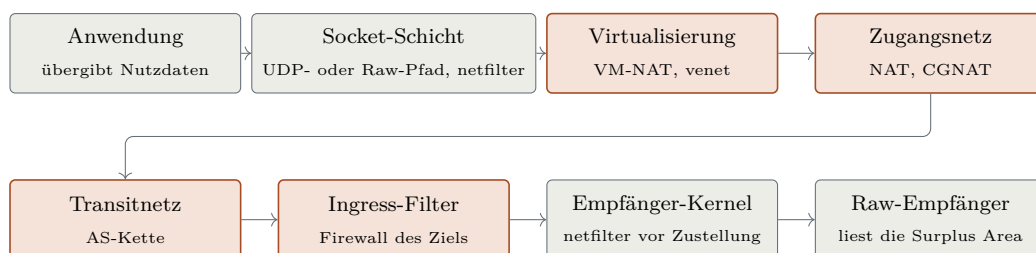


Abbildung 2.12: Messmodell des Datagrammweges

Aus Abbildung 2.12 geht hervor, dass zwischen Socket-Schicht und Empfänger-Kernel bis zu vier Stationen liegen, die das Datagramm außerhalb der Endpunkte berühren: Sie greifen teils mit Zustand ein und sind von außen nicht direkt beobachtbar. Für die Feldmessungen folgt daraus die Interpretationsregel dieser Arbeit: Geht ein optionsbehaftetes Datagramm verloren oder kommt es verändert an, ist der Befund zuerst dieser Kette zuzuordnen, bevor er der Implementierung oder dem Standard angelastet wird. Messpunkte an beiden Enden grenzen ihn dabei auf ein Intervall der Kette ein; welches Gerät eingreift, bleibt ohne zusätzliche Messpunkte oder unabhängige Evidenz offen (Kapitel 6).

Bezogen auf diese Arbeit hat der Blick auf das Werkzeugumfeld eine letzte praktische Folge. Die verbreiteten Mitschnittwerkzeuge Wireshark und tshark kennen RFC 9868 in Version 4.6 nicht: Ihre UDP-Dekodierung endet an UDP `Length` und übergeht die Surplus Area vollständig, ohne eigenes Feld, ohne Trailer-Eintrag und ohne Warnung. Für die Messkampagnen ist deshalb ein eigener Prüfer für aufgezeichnete Pakete unverzichtbar (Kapitel 6); zugleich ist der fehlende Dissektor ein Indiz dafür, wie wenig Beachtung UDP Options im Werkzeugökosystem bislang finden.

Damit sind die Grundlagen beisammen: die beteiligten Protokolle, die Erweiterungsmuster, die Surplus Area mit ihren Entwurfsprinzipien sowie Betriebssystem und Netzpfad, auf denen jede Messung aufsetzt. Bevor die Arbeit mit ihnen Implementierung und Messungen bewertet, legt Kapitel 3 die Arbeitsweise offen, mit der diese Ergebnisse entstanden sind. Denn für alle folgenden Kapitel gilt derselbe Grundsatz: Eine Aussage eines Sprachmodells ist zunächst nur ein Kandidat; ob sie gilt, entscheiden RFC, Code und Messung.

3. Methodik und KI-gestützte Entwicklung

Große Sprachmodelle unterstützten die Auswertung des RFC-Primärtexts, den Entwurf und die Implementierung, die Analyse der Messkampagnen und die Prüfung von Zwischenergebnissen. Wissenschaftlich belastbar ist dieses Vorgehen nur, wenn die Arbeit offenlegt, wie Modellausgaben geprüft werden und wer die Ergebnisse verantwortet. Diese Regeln müssen vor der Ergebnisbewertung feststehen. Deshalb steht dieses Kapitel vor der RFC-Analyse.

Die KI-gestützte Entwicklung ist Teil der Methode, aber keine eigene Forschungsfrage. Dieses Kapitel beschreibt zunächst das Forschungsdesign und die Auswertung der Norm. Danach folgen die Rollen der Beteiligten, die bekannten Schwächen der Sprachmodelle und der Prüfprozess. Abschließend ordnen externe Referenzfälle das Vorgehen ein, und die Regeln für Reproduzierbarkeit und ihre Grenzen schließen das Kapitel ab. Die Darstellung beschreibt den aktuellen Sollprozess. Sie behauptet nicht, dass jede Stufe seit dem Beginn des Projekts in dieser Form bestand.

3.1 Forschungsdesign

Die Untersuchung verbindet die Auswertung der Norm, die Referenzimplementierung und eine empirische Pfaduntersuchung. Abbildung 3.1 zeigt, wie die beiden Forschungsfragen auf diesen Teilen aufbauen:

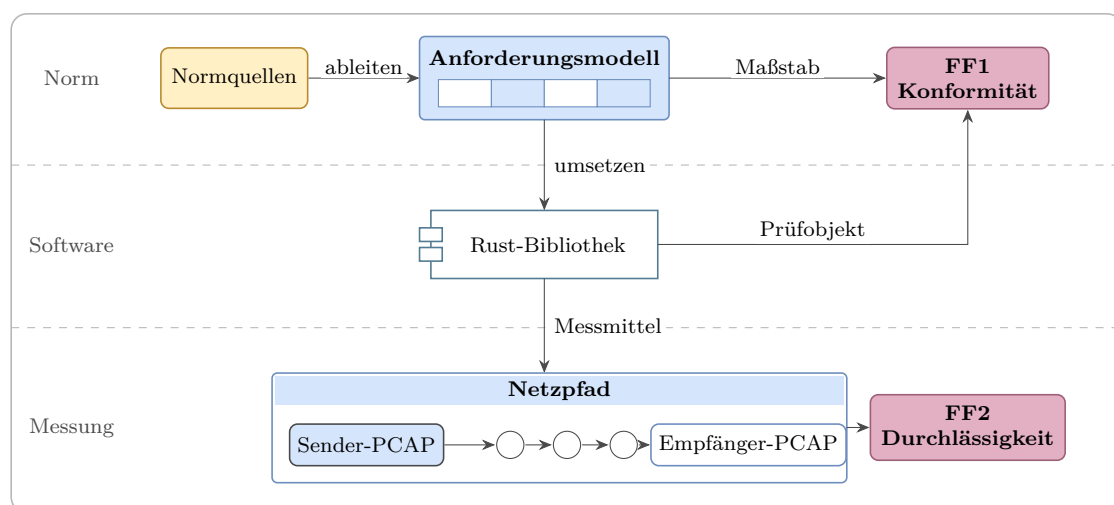


Abbildung 3.1: Aufbau der Untersuchung und Zuordnung der Forschungsfragen

Aus Abbildung 3.1 geht die Doppelrolle der Referenzimplementierung hervor:

Für FF1 ist sie der Prüfgegenstand, und das aus den Normquellen abgeleitete Anforderungsmodell bildet den Maßstab; für FF2 dient dieselbe Implementierung als Messmittel. Beide Bewertungen bleiben getrennt: Eine konforme Implementierung belegt keine Zustellbarkeit entlang realer Netzpfade, und eine unveränderte Zustellung belegt keine RFC-Konformität.

Die Forschungsfragen benötigen deshalb verschiedene Untersuchungseinheiten und Entscheidungsregeln. Tabelle 3.1 legt diese Regeln fest, bevor die Ergebnisse betrachtet werden:

Tabelle 3.1: Operationalisierung der Forschungsfragen

FF	Untersuchungs- einheit	Nachweis	Entscheidungsregel und Grenze
FF1	jede anwendbare normative Anforderung im Projektumfang	je nach Status: Normfundstelle sowie Quelltext und Prüfung oder Beleg einer technischen Grenze	Umsetzungsstand getrennt von vollständig, teilweise oder nicht erfüllbar; nicht anwendbar bleibt eine eigene Klasse
FF2	kontrolliertes Szenario; extern ein Paar aus Kontroll- und Optionsdatagramm je Pfad, Richtung und Messzeitpunkt	Paketmitschnitte an Sender und Empfänger, Sendeprotokoll und dokumentierte maschinelle Auswertung	Surplus erhalten, verändert oder entfernt, Paket verworfen oder Laufungültig; keine Übertragung auf unbeobachtete Pfade, Richtungen oder Zeitpunkte

Aus Tabelle 3.1 geht hervor, dass FF1 je Anforderung und FF2 je beobachtetem Paketpaar entschieden wird. Für FF1 wird zunächst die Anwendbarkeit bestimmt. Anwendbare **MUST**- und **SHOULD**-Aussagen gehen in die Bewertung ein; eine Abweichung von **SHOULD** benötigt eine dokumentierte Begründung. Eine **MAY**-Funktion wird nur bewertet, wenn sie für die Referenzimplementierung gewählt wurde. Nicht gewählte und außerhalb des Umfangs liegende Aussagen bleiben sichtbar, zählen aber weder als erfüllt noch als technisch unmöglich.

Der Anforderungsstatus des Implementierungs-Repositorys und die FF1-Klasse beantworten verschiedene Fragen. „Implemented“ belegt den Umsetzungsstand. „Partially implemented“ bezeichnet eine Implementierungslücke und bestimmt die technische Erfüllbarkeit noch nicht. „Partial-in-userspace“ wird als teilweise erfüllbar und „Not-feasible-in-userspace“ als nicht erfüllbar gewertet. „Delegated“ gilt nur dann als vollständig erfüllbar, wenn der RFC die Pflicht der oberen Schicht zuweist und die Schnittstelle sie als dokumentierten Vertrag abbildet; andernfalls lautet die Klasse teilweise erfüllbar. Für implementierte Funktionen dienen Quelltext

und Tests als Nachweis. Technische Grenzen werden stattdessen durch dokumentiertes Betriebssystem- oder Schnittstellenverhalten belegt. Von diesem Status je Anforderungszeile zu unterscheiden ist die Abdeckungsspalte „Covered“ der Konformitätsmatrix desselben Repositorys (Kapitel 4): Sie markiert den gewählten Umfang und ist keine FF1-Klasse.

Die kontrollierten lokalen Läufe werten jedes Szenario aus; die Szenarien sind über eigene Zielports markiert und können wie beim FRAG-Fall des RFC mehrere Datagramme enthalten. Diese Läufe validieren die Messwerkzeuge und liefern Befunde zu den lokalen Topologien, beantworten aber nicht unmittelbar FF2 zu realen Netzpfaden. Dafür vergleichen die externen Kampagnen zeitnah gesendete, gleich große Kontroll- und Optionsdatagramme auf demselben gerichteten Pfad.

Ein externer Lauf ist nur auswertbar, wenn die Senderaufzeichnung den Versand und die erwartete Surplus Area belegt. Die maschinelle Auswertung vergleicht Sender- und Empfängerzeichnung nach je Kampagne dokumentierten Zuordnungsregeln: Der lokale Referenzauswerter gruppiert die Pakete je Zielport und vergleicht Anzahl und Surplus-Inhalt, die Kampagnenauswerter ordnen die Pakete über eine im Payload mitgeführte Sequenznummer zu. Jedes zugeordnete Datagramm erhält eine Ergebnisklasse: Die Surplus Area kommt erhalten, verändert oder entfernt an, oder das Paket wird verworfen. Abweichende Paketzahlen meldet der Referenzauswerter als eigene Befundklasse; sie machen einen Lauf nur dann ungültig, wenn die Auswertung ein bestimmtes Sollergebnis erzwingt. Auch die Behandlung IP-fragmentierter Datagramme ist auswerterspezifisch: Der Referenzauswerter bricht bei IPv4-Fragmenten ab, weil er sie nicht zusammensetzt; die Kampagnenauswerter überspringen nur Folgefragmente. Einzelne Vorversuche werden als solche gekennzeichnet und nicht zu allgemeinen Aussagen über das Internet oder ein nicht beobachtetes Gerät erweitert.

Sprachmodelle sind in beiden Untersuchungen Werkzeuge, aber keine bewertende Instanz. Maßgeblich sind die Normquellen, die Messdaten, ausführbare Prüfungen und die Freigabe durch den Autor. Der nächste Abschnitt beschreibt, wie aus den Normquellen ein prüfbares Anforderungsmodell entsteht.

3.2 RFC-Auswertung und schrittweise Umsetzung

Die normativen Protokollanforderungen stammen hauptsächlich aus RFC 9868. Ergänzende Anforderungen an das UDP-Format stammen aus RFC 768; die Rechenregeln der Prüfsummen liefert als informative Referenz RFC 1071. Daneben enthält der Anforderungsbestand der Implementierung ausdrücklich gekennzeichnete Projekt-, Schnittstellen- und Plattformvorgaben. Diese Trennung verhindert, dass eine lokale Entwurfsentscheidung nachträglich als Forderung des RFC erscheint.

Normative Schlüsselwörter wie **MUST**, **SHOULD** und **MAY** werden nach BCP 14¹ gelesen. Sie sind nur in Großschreibung normativ [18, 19]. RFC 9868 stellt Absätzen mit solchen Schlüsselwörtern zusätzlich das Zeichenpaar >> voran. Die eigene Zählung ergibt 103 markierte Absätze. Diese Zahl ist eine Kontrollzahl für mögliche Anforderungen, nicht die Zahl der abschließend anwendbaren Anforderungen.

Eine erste Zählung hatte nur 96 Absätze ergeben, weil tiefer eingerückte Markierungen übersehen wurden. Ein zweites Zählverfahren deckte den Fehler auf. Kontrollzahlen werden deshalb mit zwei getrennten Verfahren ermittelt. Jede gefundene Aussage erhält außerdem eine genaue Fundstelle, eine Normstufe, eine Beurteilung der Anwendbarkeit und eine Kennzeichnung des Projektumfangs.

RFC 9868 und RFC 9869 bleiben in der Quellenarbeit getrennt. RFC 9869 beschreibt die Pfad-MTU-Bestimmung (DPLPMTUD, *Datagram Packetization Layer Path MTU Discovery*) mit UDP-Optionen und liegt außerhalb des Implementierungsumfangs dieser Arbeit [3]. Zusätzlich wird der Errata-Stand des RFC Editors berücksichtigt [15]. Das verifizierte technische Erratum EID 8834 löst den Widerspruch zwischen den Abschnitten 10 und 14 zur Behandlung überlanger Optionen. EID 8708 ist als redaktionelles Erratum für eine spätere Dokumentausgabe zurückgestellt. Der Anforderungsbestand der Implementierung verwendet diese beiden Einstufungen und dokumentiert die daraus abgeleitete Behandlung (Kapitel 4). Den Weg von den Quellen zum geprüften Bestand fasst Abbildung 3.2 zusammen:

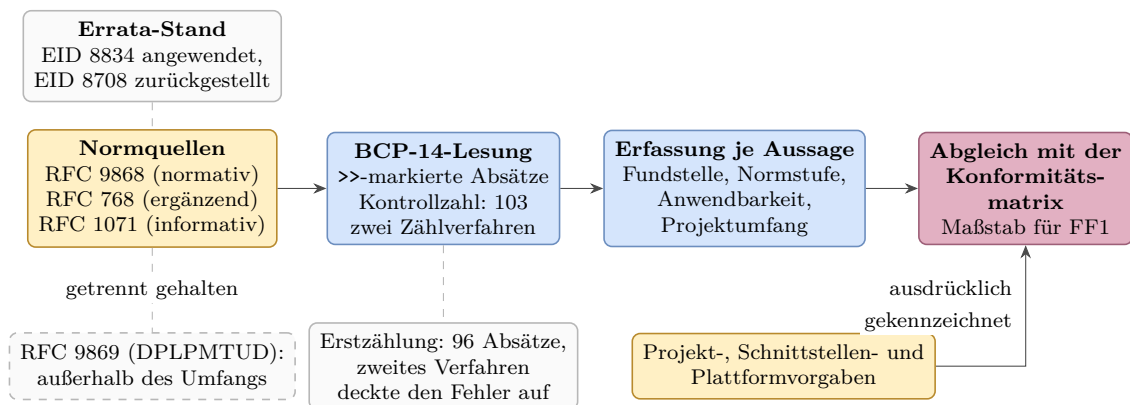


Abbildung 3.2: Von den Normquellen zum geprüften Anforderungsbestand

Aus Abbildung 3.2 geht hervor, dass der Maßstab aus zwei getrennt gehaltenen Zuflüssen entsteht: der normativen Lesung mit ihrem Errata-Stand und den ausdrücklich gekennzeichneten eigenen Vorgaben. Die Kontrollzahl sichert den normativen Zufluss mit zwei unabhängigen Zählverfahren ab; geprüft wird damit die Konformitätsmatrix des Implementierungs-Repositorys (Kapitel 4). RFC 9869 bleibt außerhalb

¹ BCP steht für Best Current Practice. BCP 14 umfasst RFC 2119 und dessen Aktualisierung RFC 8174.

des Implementierungsumfangs; der Anforderungsbestand führt seine Anwendungsfälle nur als ausdrücklich gekennzeichnete Einträge außerhalb des Umfangs, damit die Grenze des Beitrags sichtbar bleibt.

Die Umsetzung erfolgte schrittweise. Der Projektplan beginnt mit den Schritten 0 und 0.5 und reicht bis Schritt 18. Schritt 9 ging in Schritt 8 auf, Schritt 16 wurde mit der IPv6-Unterstützung aus dem Umfang entfernt. Die verbleibenden Schritte behandeln unter anderem Prüfsummen, Wire-Format, Parser, Serialisierung, typisierte Optionen, Raw Sockets, Fragmentierung, Schnittstellen und die Messumgebung. Die genaue Zuordnung und der jeweilige Abschlussnachweis stehen im Projektplan.² Diese Darstellung bildet den tatsächlichen Plan ab und ersetzt eine nachträglich vereinfachte Folge von neun Stufen. Den gelebten Kernzyklus je Schritt zeigt Abbildung 3.3:

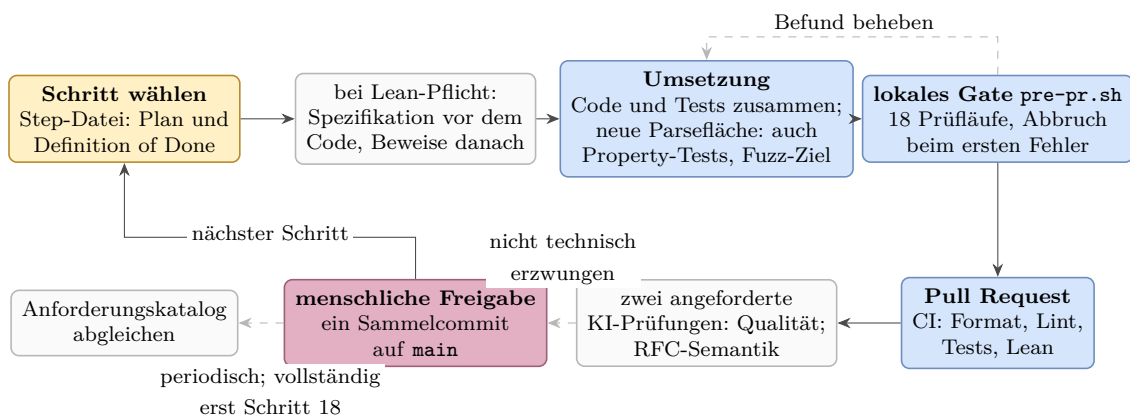


Abbildung 3.3: Kernzyklus eines Implementierungsschritts bis Schritt 18 (Ausnahmen gestrichelt)

Aus Abbildung 3.3 geht der Takt der Umsetzung hervor: Je Schritt entstehen Code, Tests und Dokumentation zusammen und erreichen den Hauptzweig als ein freigegebener Sammelcommit; die Schritte 14, 15 und 17 teilten sich einen solchen Commit. Zwei Kanten sind gestrichelt, weil sie stattfanden, aber nicht technisch erzwungen waren: Die zwei angeforderten KI-Prüfungen blockierten die Übernahme nicht, und der Abgleich des Anforderungskatalogs lief periodisch; vollständig glich ihn erst Schritt 18 ab, der als eigener Korrekturschritt auf einen nachträglichen RFC-Abgleich antwortete. Welche Rollen bei der Erzeugung und Prüfung dieser Ergebnisse wirken, klärt der folgende Abschnitt.

² Implementierungs-Repository, Commit `f1cfe52`, `docs/plan/ROADMAP.md`.

3.3 Beteiligte und ihre Rollen

Der Autor entscheidet, gewichtet, gibt frei und verantwortet das Ergebnis. Claude Code von Anthropic dient vorwiegend als führendes Werkzeug für Entwürfe, Quelltext und Analysen. Codex von OpenAI dient vorwiegend als getrennter Zweitprüfer mit dem Auftrag, Fehler und Gegenbeispiele zu suchen. Beide Programme verbinden ein Sprachmodell mit Dateizugriff, Kommandoausführung und dem Repository.

Eine Prüfung durch dasselbe Modell gilt nicht als getrennte Zweitprüfung. Auch die Prüfung durch ein Modell eines anderen Herstellers beweist keine Unabhängigkeit. Die Herstellertrennung ist eine bewusste Maßnahme gegen vollständig gleiche Fehlermuster, deren Wirkung in dieser Arbeit jedoch nicht gemessen wird. Änderungen durch den Zweitprüfer erfolgen nur in einem eigenen, vom Autor freigegebenen Arbeitsauftrag. Sie zählen dann nicht als Zweitprüfung desselben Ergebnisses.

Lean liefert maschinengeprüfte Beweise für handgeschriebene Modelle ausgewählter Invarianten des Wire-Formats, der OCS, der Fragmentierung und der Empfangslogik. Ein Prüfskript baut die Beweise und kontrolliert die verwendeten Axiome. Nicht bewiesen ist die Übereinstimmung des Rust-Codes mit diesen Modellen. Zulässig ist daher die Aussage, dass ausgewählte Invarianten in Lean-Modellen bewiesen sind. Die gesamte Implementierung ist nicht formal verifiziert. Dieses Zusammenspiel aus Erzeugung, Prüfung und Freigabe zeigt Abbildung 3.4:

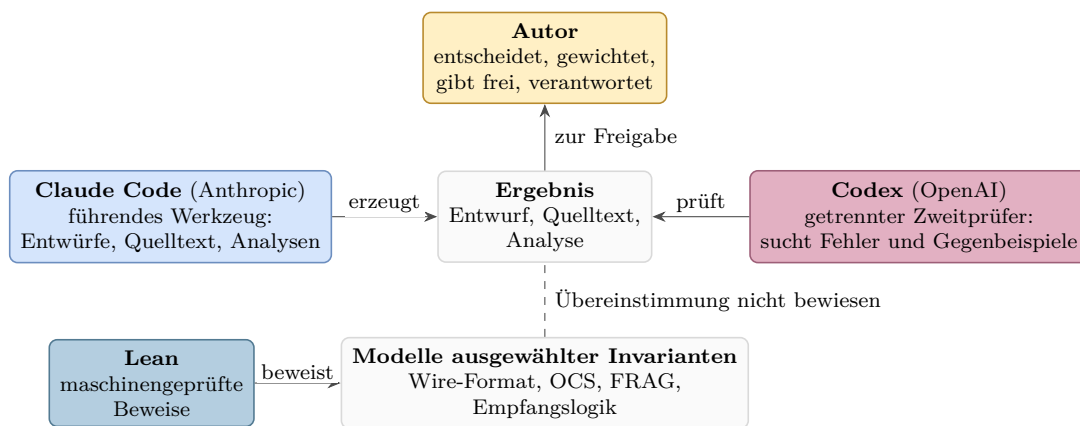


Abbildung 3.4: Rollen bei Erzeugung, Prüfung und Freigabe

Aus Abbildung 3.4 geht hervor, dass Erzeugung und Prüfung getrennt besetzt sind und jedes Ergebnis erst mit der Freigabe durch den Autor gilt. Lean wirkt dabei nur auf der Modellebene: Die gestrichelte Verbindung markiert, dass die Übereinstimmung des Rust-Codes mit den bewiesenen Modellen gerade nicht bewiesen ist.

Die Art der Prüfung richtet sich nach der Art der Aussage. Protokollaussagen werden gegen RFC-Primärtext und Errata geprüft. Aussagen über die Implementierung

werden durch Quelltext und ausführbare Prüfungen gestützt. Pfadbefunde benötigen Paketmitschnitte und Kampagnenprotokolle. Literaturbehauptungen werden an der jeweiligen Primärquelle geprüft. Ein Hashwert belegt nur die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts.

Im Umgang mit den Modellen gilt die Haltung, sie wie fähige, aber unerfahrene Entwickler zu behandeln, deren Arbeit grundsätzlich geprüft wird. Aus diesem Grund hat der Autor drei Arbeitsregeln schriftlich festgehalten:

1. Modellergebnisse werden gegen die jeweilige Primärquelle geprüft, nie gegen eigene Zusammenfassungen.
2. Aufträge werden neutral formuliert, ohne das Modell in eine gewünschte Richtung zu lenken.
3. Widerspruch wird ausdrücklich ermutigt: Ein Modell, das „nein“ sagt oder Unsicherheit meldet, ist wertvoller als eines, das gefällig zustimmt.

Diese Regeln beschreiben den Sollprozess. Welche Prüfungen tatsächlich stattfanden, wird nicht aus ihm abgeleitet, sondern aus den vorhandenen Protokollen rekonstruiert (Abschnitt 3.7). Warum diese Trennung nötig ist, zeigt der folgende Abschnitt.

3.4 Bekannte Schwächen der Sprachmodelle

Der Einsatz von Sprachmodellen bringt bekannte Schwächen mit, auf die die Methodik direkt antwortet.

Halluzination. Eine Halluzination ist ein überzeugend formulierter, aber falscher Inhalt. Kalai u. a. führen solche Fehler auf Trainings- und Bewertungsverfahren zurück, die Raten gegenüber eingestandener Unsicherheit belohnen [20].

Gefälligkeit. Sycophancy bezeichnet die Neigung, sich der erkennbaren Erwartung des Gegenübers anzuschließen. Perez u. a. beobachteten, dass größere Modelle die bevorzugte Antwort des Nutzers häufiger wiederholen; das Verhalten zeigt sich schon bei reinen Vortrainingsmodellen, und Training mit menschlicher Rückmeldung beseitigt es nicht. Cheng u. a. belegen die Verbreitung über elf aktuelle Modelle und in Experimenten messbare Schäden [21, 22]. Halluzination kann falsche Aussagen erzeugen. Gefälligkeit kann verhindern, dass ein Modell ihnen widerspricht.

Verfügbarkeit. Beide Sprachmodellwerkzeuge sind Cloud-Dienste. Sie können ausfallen, gedrosselt oder verändert werden. Die Ergebnisse liegen deshalb als Quell-

text, Tests, Protokolle und Messdaten vor. Diese Artefakte bleiben ohne die Dienste prüfbar. Der genaue Entstehungsvorgang einer Modellausgabe ist wegen wechselnder Modelle und nichtdeterministischer Antworten dagegen nicht vollständig reproduzierbar. Abbildung 3.5 stellt jeder Schwäche ihre Wirkung und die methodische Antwort gegenüber:

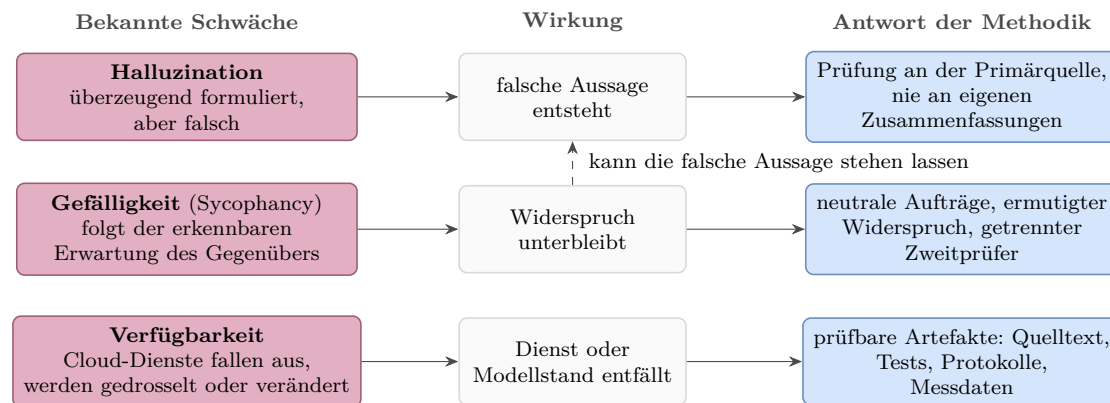


Abbildung 3.5: Schwächen der Sprachmodelle und methodische Antworten

Aus Abbildung 3.5 geht hervor, dass die ersten beiden Schwächen zusammenwirken können: Eine halluzinierte Aussage kann durch Gefälligkeit unwidersprochen bleiben. Die Antworten setzen deshalb nicht am Modell an, sondern an der Prüfinstanz und an der Form der Ergebnisse.

Die wichtigste Konsequenz ist einfach: Keine Aussage ist richtig, weil ein Modell sie behauptet. Ob die Implementierung die Norm erfüllt, wird mit der Konformitätsprüfung und der Testarchitektur beurteilt. Auch diese Prüfungen belegen nur die jeweils untersuchten Eigenschaften und keinen allgemeinen Korrektheitsbeweis.

Zwei Projektbefunde zeigen die praktische Bedeutung. Eine Gegenprüfung eigener Sekundärdokumente am RFC-Primärtext fand vier inhaltliche Abweichungen; die Konformität der Implementierung wird davon getrennt anhand von Quelltext und Tests beurteilt. Eine Kampagnennotiz nannte außerdem 15 Kontrollmessungen, tatsächlich waren es 18. Eine dreiköpfige Prüfung ließ diesen Fehler zunächst bestehen, weil die vorgelegten Prüfaussagen die Gesamtzahl nicht umfassten; eine spätere getrennte Durchsicht fand ihn. Eigene Zusammenfassungen bleiben daher Suchhilfen, und eine Prüfung belegt nur die Aussagen, die ihr tatsächlich vorgelegt wurden.

3.5 Testarchitektur und Prüfprozess

Die aktuelle Prüfkette wird lokal gestartet, verbindet aber lokale und entfernte Prüfungen. Sie ist der Sollzustand vor dem Öffnen oder Aktualisieren eines Pull

Requests. Historisch entstand diese Kette schrittweise. Aussagen über frühere Pull Requests beziehen sich deshalb nur auf die damals protokollierten Prüfungen.

Zu den deterministischen Prüfungen gehören Formatierung, statische Analyse, Dokumentationsbau, Unit-Tests und Integrationstests. Property-Tests erzeugen für jede Eigenschaft lokal 1024 Fälle. Neun zeitlich begrenzte Fuzz-Ziele suchen mit beliebigen Bytefolgen nach Abstürzen und verletzten Invarianten. Der dokumentierte Sollprozess verlangt, gefundene Gegenbeispiele zu minimieren und als Regressionstests einzuchecken. Die Lean-Prüfung baut die formalen Modelle und kontrolliert ihre Axiome.

Die systemnahen Stufen laufen auf dem entfernten Linux-Zielsystem. Dort prüfen sie die Cross-Kompilierung, die Raw-Socket-Integration mit erhöhten Rechten und das tatsächlich gesendete Paketbild. Das lokal gestartete Skript und die Reihenfolge seiner Prüfschritte sind im Implementierungs-Repository festgehalten.³ Die kontinuierliche Integration (CI) wiederholt Formatierung, statische Analyse und Rust-Tests. Nach deren Erfolg baut sie die Lean-Modelle und stellt anschließend zwei getrennte Prüfanfragen: an Claude Code zu Codequalität und Sicherheit und an Codex zur RFC-9868-Semantik.

Eine besondere Gefahr sind Fehler, die Implementierung und Test gemeinsam haben. Sender und Empfänger der Bibliothek verwenden getrennte Module für Serialisierung und Parsing, beruhen aber auf denselben projektspezifischen Formatannahmen, etwa der gemeinsamen Tabelle der Kind-Werte und der Längenkonstanten. Ein gemeinsamer Entwurfsfehler in diesen Annahmen kann deshalb jeden Round-Trip-Test bestehen. Die Wire-Prüfung verringert diese Selbstbezüglichkeit: `tcpdump` zeichnet die tatsächlich gesendeten Bytes auf. Ein getrenntes Python-Programm dekodiert IPv4, UDP und die UDP-Optionen und berechnet die Prüfsummen mit eigener Faltung nach RFC 1071 [12] und einer eigenen CRC32C-Tabelle. `tshark` bestätigt zusätzlich die IPv4- und UDP-Felder. Die im Projekt verwendete Version 4.6.4 besitzt jedoch keinen Dissektor für UDP-Optionen.

Auch diese Prüfung ist nicht vollständig unabhängig. Die erwarteten Referenzbytes stammen aus der projekteigenen Formatdokumentation. Die Unabhängigkeit gilt für den Prüfcode, nicht für den Entwurf. Dasselbe gilt für die drei handgerechneten OCS-Referenzvektoren. Sie sind nicht palindromisch und wurden getrennt nachgerechnet, damit Bytevertauschungen auffallen. RFC 9868 enthält jedoch keinen veröffentlichten Referenzwert; eine externe Bestätigung dieser Vektoren steht noch aus.

Der aktuelle Sollprozess ist in Abbildung 3.6 als Sequenzdiagramm dargestellt: Jeder Beteiligte erhält eine senkrechte Lebenslinie, die schmalen Balken zeigen

³ Implementierungs-Repository, Commit `f1cfe52`, `scripts/pre-pr.sh`.

seine aktiven Phasen, durchgezogene Pfeile sind Aufträge und gestrichelte Pfeile Rückmeldungen. Auch diese Darstellung bleibt bewusst auf die Kontrollpunkte und die Grenze ihrer technischen Erzwingung reduziert:

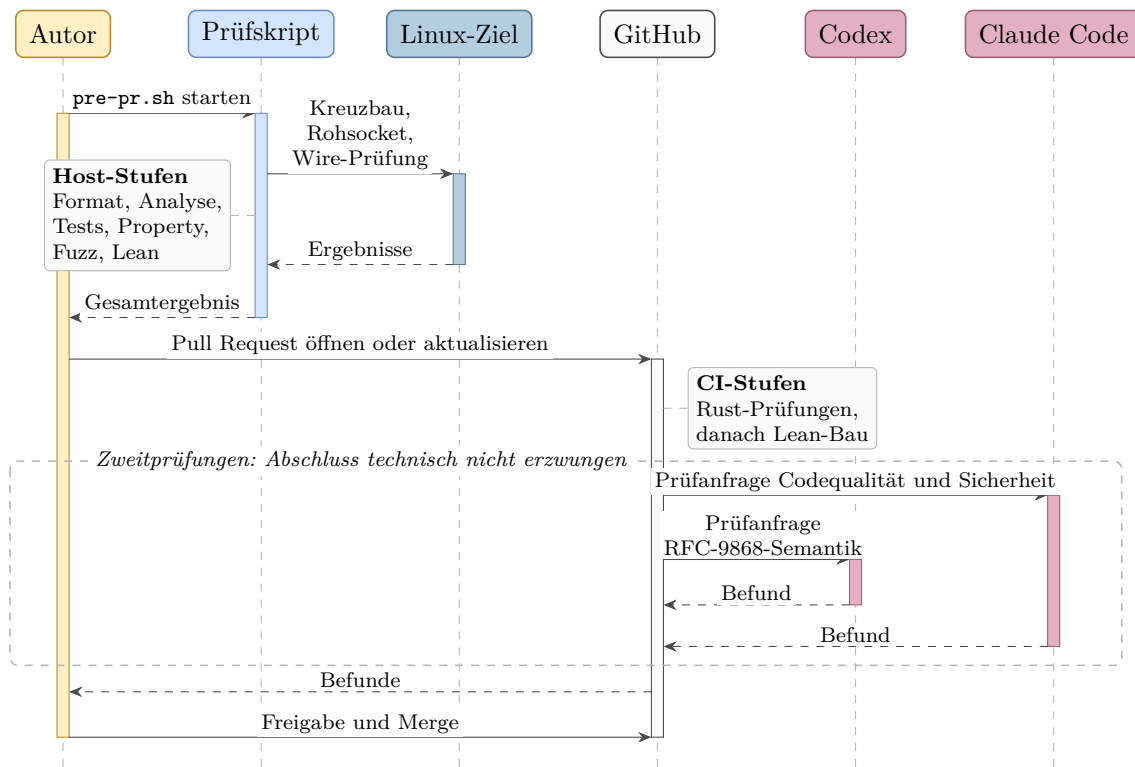


Abbildung 3.6: Prüfprozess im Sollzustand

Aus Abbildung 3.6 geht hervor, dass die Pfeilfolge nur die vorgesehene Reihenfolge vom Prüfskript über den Pull Request und die Rust/Lean-CI bis zur Freigabe durch den Autor zeigt; eine technische Merge-Bedingung ist sie nicht. Nach erfolgreicher Rust/Lean-CI werden getrennte Zweitprüfungen angefragt. Der gestrichelt umrahmte Bereich hebt hervor, dass auch deren Abschluss nicht erzwungen ist; das Thesis-Repository besitzt überhaupt keine solchen Prüfschritte. Pull Request 27 zeigt die Folge: Er wurde vor Abschluss aller Prüfungen zusammengeführt; ein Befund traf danach ein.⁴ Daher wird kein technisch erzwungenes Vier-Augen-Prinzip behauptet.

Diese Testarchitektur gehört zur Entwicklung. Die Pfaduntersuchung für FF2 verwendet die fertigen Programme, Sendeprotokolle sowie Paketmitschnitte an Sender und Empfänger. Ein bestandener Entwicklungstest sagt nichts über reale Netzpfade aus, und eine Pfadmessung ersetzt keinen Entwicklungstest. Das Projekt setzt außerdem kein systematisches Mutation-Testing am Rust-Quelltext ein; in einem dokumentierten Einzelfall wurde ein früherer Quelltextfehler gezielt wieder eingeführt, und drei Property-Tests sowie ein Fuzz-Ziel erkannten diese Mutation. Gezielte

⁴ Implementierungs-Repository, <https://github.com/ab7z/udp-transport-options/pull/27>.

Veränderungen an Paketmitschnitten prüfen nur, ob der jeweilige Auswerter bekannte Manipulationen erkennt. Der folgende Abschnitt ordnet diesen Prüfungsgedanken durch externe Vergleichsfälle ein.

3.6 Externe Referenzfälle

Diese Methodik trennt Erzeugung, Prüfung und Freigabe. Zur methodischen Einordnung dienen acht publizierte Fälle KI-gestützter Ergebnisse mit unterschiedlichen Formen der Validierung; sie bilden keine systematische Literaturübersicht, belegen keine einheitliche Prüfkette und erlauben keine Aussage über die Häufigkeit oder Wirksamkeit eines bestimmten Prüfprozesses.

Vier Fälle zeigen die Bandbreite der Prüfinstanzen: AlphaFold wurde im blinden CASP14-Vergleich an experimentell bestimmten Proteinstrukturen gemessen [23]. Bei Davies u. a. lieferte das Modell Hypothesen und Kandidatenmuster, geprüft durch Gegenbeispielsuche und den klassischen Beweis der beteiligten Mathematiker [24]. Bei FunSearch bewertet ein systematischer, ausführbarer Auswerter jeden Programmkandidaten des Sprachmodells [25], und bei AlphaProof prüft der Lean-Kern jeden gefundenen formalen Beweis maschinell [26].

Vier weitere Fälle benennen ihre Prüfinstanzen besonders klar. Bei Knuths Zyklonproblem stammte die Konstruktion vom Modell, der Beweis von Knuth und die Lean-Formalisierung von Kim Morrison aus der Lean-Community, der seine Verifikation veröffentlichte, bevor Knuth davon erfuhr [27]. Die verbesserte untere Schranke für den Anteil der Nullstellen der Riemannschen Zetafunktion auf der kritischen Linie wurde gemeinsam mit einem Anthropic-Mitarbeiter in Lean formalisiert; zwei Mathematiker von Anthropic prüften die Arbeit, und zwei externe Fachleute begutachteten sie zusätzlich [28]. Beim Gegenbeispiel zur Einheitsabstands-Vermutung erzeugte das Modell den Beweis; externe Mathematiker prüften ihn und veröffentlichten eine verdichtete, von Menschen verifizierte Fassung als Begleitpapier [29]. Der Physik-Preprint zur C-Parameter-Resummation legt im Abstract offen, dass Rechnungen, Numerik und Manuskript von Claude unter Aufsicht eines Physikers entstanden und gegen Monte-Carlo-Simulationen geprüft wurden [30]; dieser ausdrücklichen Offenlegung in der Publikation selbst folgt auch die vorliegende Arbeit (Abschnitt 1.6).

Als neunter Fall liegt die Migration der JavaScript-Laufzeit Bun von Zig nach Rust dem Softwareteil dieser Arbeit am nächsten. Das Projekt trennte Erzeugungs- und Prüfrollen, besetzte beide aber mit Modellen derselben Familie [31, 32]. Nach dem Merge wurde dennoch ein Fall undefinierten Verhaltens gemeldet; als Reaktion nahm das Projekt das Prüfwerkzeug Miri in die CI auf [33]. Daraus folgen zwei

Lehren für diese Arbeit: Die Zweitprüfung besetzt bewusst ein Modell eines anderen Herstellers, und auch eine mehrstufige Prüfung gilt nicht als Garantie.

Die neun Fälle zeigen unterschiedliche Prüfverfahren; sie belegen weder eine in allen Fällen gleiche unabhängige Prüfung mit menschlicher Freigabe noch einen Erfolg der hier gewählten Modellkombination. Das übertragbare Prinzip ist enger: Die Prüfinstanz muss zum Artefakt passen. Normtext, ausführbare Tests, formale Beweise, Messdaten und menschliche Freigabe erfüllen dabei unterschiedliche Aufgaben. Ob sie das im Projekt tatsächlich getan haben, muss dokumentiert sein; diese Dokumentation regelt der folgende Abschnitt.

3.7 Reproduzierbarkeit und Grenzen

Aussagen über den Entstehungsprozess sind nur so belastbar wie ihre Protokollierung. Maßgeblich sind die vorhandenen Arbeitsprotokolle, also die Repository-Historie, die Prüf- und Kampagnenprotokolle sowie die Messdaten. Eine Prüfung, die dort nicht belegt ist, wird nicht nachträglich als erfolgt behauptet. Ein belegter Prüfschritt bedeutet umgekehrt nicht, dass die geprüfte Aussage allgemein wahr oder vollständig ist.

Eigene Zusammenfassungen und Arbeitsnotizen dienen nur als Suchhilfe. Die Prüfung erfolgt am Repository, am Primärtext oder an den Messdaten. Pannen werden nicht geglättet: Die verlorenen Berichte dreier Kampagnenprüfer wurden aus erhaltenen Arbeitsdateien der Sitzung wiederhergestellt und als Wiederherstellung gekennzeichnet; das vollständige Sitzungstranskript ist nicht archiviert. Beim Wechsel des Prüfablaufs entstand außerdem ausnahmsweise eine doppelte Zweitprüfung; beide Läufe blieben dokumentiert. Auch Knuth berichtet, dass Zwischenstände einer Modellsitzung verloren gingen und die Fortschrittsdokumentation erneut eingefordert werden musste [27].

Technische Ergebnisse sind reproduzierbar, soweit Quellstand, Werkzeuge, Eingaben und Ausgaben archiviert sind. Die genaue Folge nichtdeterministischer Modellausgaben ist es nicht. Für frühe Projektphasen fehlen zudem vollständige Angaben zu Modellen und Konfigurationen. Diese Lücke bleibt bestehen und wird nicht rückwirkend durch den heutigen Sollprozess ersetzt.

Zum Abschluss dieses Kapitels bleiben die externe Bestätigung der OCS-Vektoren und die beschreibenden Kennzahlen der Zusammenarbeit offen. Die Wiedervorlage der Errata ist dagegen mit der Prüfung von EID 8834 und EID 8708 abgeschlossen. Kapitel 4 verwendet die hier festgelegten Quellen- und Bewertungsregeln, um aus RFC 9868 das Anforderungsmodell für FF1 abzuleiten.

4. Analyse und Anforderungsmodell

Dieses Kapitel bildet aus dem Standard das Soll-Modell der Arbeit. Kapitel 3 hat dafür die Quellen- und Bewertungsregeln festgelegt; hier werden sie angewandt: Zunächst führt Abschnitt 4.1 die Extraktion auf dem Primärtext von RFC 9868 durch; danach stellt Abschnitt 4.2 den Anforderungsbestand des Implementierungs-Repositorys vor und hält ihn gegen diesen Maßstab; Abschnitt 4.3 definiert die Bewertungskriterien für FF1. Davon getrennt legt Abschnitt 4.4 das Pfad- und Fehlermodell für FF2 fest; es stammt nicht aus dem Standard: Das Pfadmodell beruht auf der in Abschnitt 2.5 beschriebenen Kette, das Fehlermodell auf den Vorversuchen. Abschließend begründet Abschnitt 4.5 die Wahl der Werkzeuge.

Dabei gilt die in Abschnitt 3.1 festgelegte Trennung: Dieses Kapitel legt Maßstäbe fest und fällt keine Erfüllungsurteile. Ob die Implementierung eine Anforderung erfüllt und wie sich die Surplus Area auf realen Pfaden verhält, bewertet erst Kapitel 6, und zwar gegen genau die hier definierten Kriterien.

4.1 Anwendung der Extraktionsmethode

Wie in Abschnitt 3.2 festgelegt, werden die normativen Aussagen aus dem Primärtext gewonnen; die Kontrollzahl bilden die dort ermittelten 103 mit >> markierten Absätze, und unmarkierte normative Sätze bleiben einbezogen [18, 19].

Jede extrahierte Aussage wird zu einem Prüfpunkt für den Anforderungsbestand des Implementierungs-Repositorys (Abschnitt 4.2); beschreibende Definitionen bekommen keinen eigenen Prüfpunkt, sondern binden als Kontext die Aussagen, die auf ihnen aufbauen. Extrahiert wird satzgenau und nicht absatzweise, denn ein markierter Absatz kann mehrere Pflichten mit verschiedenen Adressaten bündeln; das Beispiel aus Abschnitt 8 unten zeigt genau diesen Fall. Mehrere Schlüsselwörter im selben Satz bleiben eine Aussage, wenn sie dieselbe Pflicht für denselben Adressaten ausdrücken.

Die Extraktion folgt der Leserichtung des RFC; Abbildung 4.1 zeigt, wie sich die 103 markierten Absätze auf die Abschnitte verteilen:

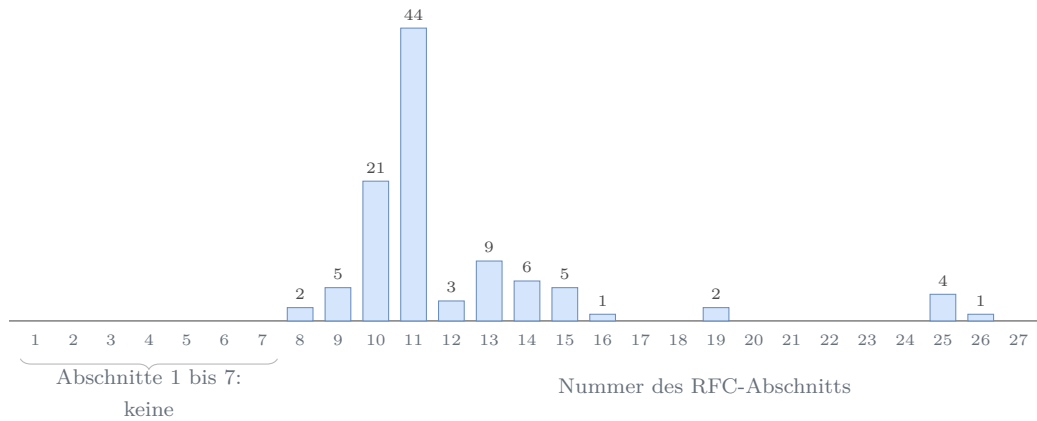


Abbildung 4.1: Verteilung der markierten Absätze über die Abschnitte von RFC 9868 (eigene Zählung)

Knapp zwei Drittel der markierten Absätze (65 von 103) liegen in den Abschnitten 10 und 11, also beim Optionsformat mit seinen Verarbeitungsregeln und bei den einzelnen Optionen; der Vorspann bis einschließlich Abschnitt 7 enthält keinen einzigen markierten Absatz.

Aus dem Vorspann stammt dennoch eine Vorgabe des Projektumfangs: Abschnitt 7 verringert die Obergrenze der `UDP Length` um vorangehende Shim-Header, und bei solchen Ketten kann das `Protocol`-Feld von 17 abweichen [2, Sec. 7]. Die Implementierung verfolgt Shim-Ketten nicht und verarbeitet nur `Protocol` 17; das ist eine ausdrückliche Umfangsvorgabe dieser Arbeit, kein Verbot des RFC.

Abschnitt 8 zeigt die satzgenaue Zerlegung an den ersten zwei markierten Absätzen des Dokuments. Der erste erlaubt als **MAY**, dass Optionen an jedem gültigen `UDP Length`-Offset beginnen. Der zweite bündelt zwei Pflichten mit verschiedenen Adressaten: Der Sender muss (**MUST**) die Ausrichtungsbytes vor dem OCS auf null setzen, der Empfänger muss (**MUST**) andernfalls alle Optionen ignorieren und die Surplus Area still verwerfen; verworfen wird die Surplus Area, nicht das Datagramm (Abschnitt 2.4.2) [2, Sec. 8]. Aus dem Absatz entstehen deshalb zwei getrennte Prüfpunkte. Wie das Implementierungs-Repository seine Anforderungen führt und wie diese Arbeit sie gegen den Maßstab hält, zeigt der folgende Abschnitt.

4.2 Anforderungskatalog und Konformitätsmatrix

Die Anforderungen der Implementierung führt das Implementierungs-Repository (Kapitel 5) in der Datei `docs/requirements.md`. Sie entstand vor dem ersten Implementierungsschritt und wurde parallel zur Umsetzung fortgeschrieben.¹ Die Datei hat

¹ Angelegt mit Commit `36cc465` am 30.05.2026, seither in 14 Commits fortgeschrieben; ausgewertet wird durchgehend der Stand `f847895` (Kapitel 5).

drei Teile: einen Katalog von 49 funktionalen Anforderungen mit stabilen Kennungen (FR-01 bis FR-50; FR-47 entfiel mit der IPv6-Umfangsentscheidung), zwölf nichtfunktionale Anforderungen (NFR-01 bis NFR-12) und eine Konformitätsmatrix mit 67 normativen Aussagen aus RFC 9868. Jede FR-Zeile nennt RFC-Quelle, Normstufe, Projektumfang, den zugehörigen Schritt des Projektplans und den Umsetzungsstand; bei den NFR-Zeilen tritt in sechs der zwölf Zeilen eine Projektbegründung an die Stelle der RFC-Quelle. Die Konformitätsmatrix führt je Aussage den RFC-Bereich, die Normstufe und die Abdeckung (**Covered**) relativ zum gewählten Umfang; 58 ihrer 67 Zeilen verweisen auf die tragenden FR- und NFR-Kennungen.

Die folgenden Zeilen zeigen einen Auszug aus der Konformitätsmatrix. Die Aussagen sind ins Deutsche gekürzt; Normstufe und Abdeckung stehen im Wortlaut der Datei:

Tabelle 4.1: Auszug aus der Konformitätsmatrix des Implementierungs-Repositorys (Stand f847895; normative Aussagen nach RFC 9868 [2])

Fund-stelle	Normative Aussage	Normstufe	Abdeckung	FR/NFR
Sec. 8	Ausrichtungsbyte vor dem OCS ist null; sonst alle Optionen ignorieren und die Surplus Area still verwerfen	MUST	yes	FR-05
Sec. 9	OCS ungleich null, wenn die UDP-Prüfsumme ungleich null ist	MUST	yes	FR-21
Sec. 10	Optionen über 254 Byte nutzen das erweiterte Format; das kleinste Format ist empfohlen	MUST/ SHOULD	yes (send); local strict receive	FR-09, FR-14
Sec. 11.4	Reassemblierungsspeicher begrenzen und nicht über Paare teilen	SHOULD	yes	FR-34, NFR-06
Sec. 11.8	TIME-Option	MUST (when implemented)	out	keine; außerhalb des Umfangs

Aus Tabelle 4.1 geht die Arbeitsteilung der Datei hervor: Die Konformitätsmatrix hält die Normseite fest, die FR- und NFR-Zeilen tragen die Umsetzung, und die Abdeckungsspalte unterscheidet erfüllte, nur differenziert erfüllte und ausdrücklich außerhalb des Umfangs gehaltene Aussagen. Die erste Zeile ist die zweite Pflicht aus dem Abschnitt-8-Beispiel in Abschnitt 4.1; die Abdeckung ist eine Selbstauskunft des Repositorys und noch kein Erfüllungsurteil dieser Arbeit.

Der Maßstab dieser Arbeit bleibt der Primärtext: Die 103 markierten Absätze aus Abschnitt 4.1 bilden das Prüfraster, gegen das die Konformitätsmatrix satzgenau gehalten wird. Dass dieses Raster nötig ist, zeigen zwei Befunde des Abgleichs: Die Matrix führt zur Lage der Surplus Area eine MUST-Zeile zu Abschnitt 7, obwohl dieser Abschnitt keinen markierten Absatz enthält und die Lage dort beschreibend

festgelegt wird, und sie stützt ihre Zeile zum Optionsbereich auf den beschreibenden Eröffnungssatz von Abschnitt 8; beide Stellen bleiben als Einstufungsabweichungen festgehalten. Ob die Abdeckungsangaben je Aussage dem Quelltext standhalten, bewertet Abschnitt 6.3. Mit welchen Kategorien Kapitel 6 dabei arbeitet, definiert der folgende Abschnitt.

4.3 FF1-Bewertungskriterien

FF1 fragt, welche Anforderungen sich unter Linux und IPv4 im Benutzerraum „vollständig, teilweise oder nicht“ erfüllen lassen (Abschnitt 1.3). Die Bewertung nutzt dafür drei Kategorien und eine Sonderklasse, die Tabelle 3.1 bereits verankert. **Vollständig** erfüllbar ist eine Anforderung, wenn sie unter den eigenen Vorgaben der Arbeit ohne Abstriche umsetzbar ist; diese Vorgaben sind die Plattform aus Abschnitt 1.3 (Linux, IPv4, Benutzerraum mit Raw Sockets) und die Protokollgrenze aus Abschnitt 4.1 (nur `Protocol 17`, keine Shim-Ketten). Quelltext und der vorgesehene Nachweis müssen das belegen. **Teilweise** erfüllbar ist sie, wenn nur ein Teil erreichbar bleibt und die Ursache der Einschränkung belegt ist. **Nicht erfüllbar** ist sie, wenn dokumentiertes Betriebssystem- oder Schnittstellenverhalten die Umsetzung im Benutzerraum grundsätzlich ausschließt. **Nicht anwendbar** bleibt als Sonderklasse sichtbar, wird aber nicht bewertet; hierhin gehören Aussagen außerhalb des Umfangs und nicht gewählte **MAY**-Funktionen.

Zwei Eigenschaften seien vorweggenommen. Erstens dürfen Kategorien leer bleiben: Findet die Bewertung keine grundsätzlich unerfüllbare Anforderung, ist das ein Ergebnis und kein Mangel des Maßstabs. Zweitens liegen die erwarteten Einschränkungen im gewählten Zugangsweg (Abschnitt 4.5.2): Die MTU-Bindung mit `EMSGSIZE` im `IP_HDRINCL`-Sendepfad und die auf die Bibliothek verlagerten UDP-Prüfungen des Raw-Empfangs sind Prüflasten, aber noch keine Teilbewertung; ob daraus eine nur teilweise erfüllbare Anforderung wird, entscheidet erst Abschnitt 6.3 je Aussage. Damit ist der Maßstab für FF1 vollständig; FF2 benötigt keinen Normmaßstab, sondern ein Modell der untersuchten Pfade und ihrer Fehler.

4.4 FF2-Pfad- und Fehlermodell

FF2 fragt, in welchem Umfang die Surplus Area auf realen Pfaden bis zur Gegenstelle erhalten bleibt (Abschnitt 1.3). Untersuchungseinheit und Ergebnisklassen stehen seit Tabelle 3.1 fest: Verglichen wird je Pfad, Richtung und Messzeitpunkt ein Paar aus Kontroll- und Optionsdatagramm; die Surplus Area kommt **erhalten**, **verändert** oder **entfernt** an, oder das Paket wird **verworfen**. Darauf bauen drei Festlegungen

auf: die Pfadstufen, die Zählregeln und das Fehlermodell; zwei Tabellen grenzen am Ende den Geltungsbereich ab.

Die Messpfade gliedern sich in fünf **Pfadstufen** mit zunehmender Netzferne. Jede Stufe ist eine eigene Klasse von Messpfaden und keine Erweiterung der vorigen Stufe; mit jeder Stufe kommen mehr fremde Akteure aus der in Abschnitt 2.5 beschriebenen Kette hinzu:

1. **Loopback:** Sende- und Empfangspfad auf demselben Host; prüft Bibliothek und Messwerkzeuge ohne fremde Akteure.
2. **Virtualisierte Topologien:** Netzpfade von virtuellen Maschinen und Containern (NAT und Bridge); die ersten Middleboxes, noch unter eigener Kontrolle.
3. **Tunnel:** Kapselung über einen realen Pfad (WireGuard); der Pfad sieht nur die Hülle und transportiert die Surplus Area im Inneren unesehen.
4. **Öffentliche Pfade in Europa:** Cloud-Endpunkte in Deutschland und Finnland sowie ein Mobilfunk-Zugangsnetz; reale Zugangs-, Transit- und Provider-Kanten.
5. **Interkontinentale Pfade:** Endpunkte in Nordamerika und Asien-Pazifik; lange Transitzketten und die Netzstrukturen großer Cloud-Anbieter.

Die Stufenfolge ordnet die Abdeckung, nicht die Ursache: Sie legt fest, welche Pfadklassen untersucht werden; einen Befund ordnet erst das Fehlermodell einem Pfadabschnitt zu. Für die Zählung gelten drei Regeln. Erstens ist die Einheit das beobachtete Paar; Befunde gelten nur für die beobachteten Pfade, Richtungen und Zeitpunkte (Tabelle 3.1). Zweitens zählen als getrennt nur Beobachtungen ohne bekannte gemeinsame Kante, denn Kampagnen und Messstrecken (Paare aus Quelle und Ziel) können dieselben Geräte durchlaufen; geteilte Kanten dienen umgekehrt der Eingrenzung eines Befunds. Drittens gelten Vorversuche ohne beidseitige Mitschnitte und Ablaufprotokolle als Piloten und werden nicht verallgemeinert; was eine vollständige Kampagne archiviert, legt das Testkonzept fest (Abschnitt 6.1).

Das **Fehlermodell** beantwortet schließlich, wo und wie eine Surplus Area auf dem Pfad scheitern kann. Es entstand schrittweise aus den Vorversuchen dieser Arbeit und dient Kapitel 6 als Klassifikationsrahmen; die Ergebnisklassen aus Tabelle 3.1 blieben davon unberührt. Sein Grundgedanke ist ein Kantenmodell: Aus Sicht der zwei Messpunkte zerfällt ein Messpfad in Kanten, und ein Befund lässt sich nur einem Intervall zwischen den Messpunkten zuschreiben, nicht einem einzelnen Gerät. Abbildung 4.2 zeigt es mit den drei erwarteten Mechanismenklassen:

Aus Abbildung 4.2 gehen die drei Mechanismenklassen hervor, mit denen das

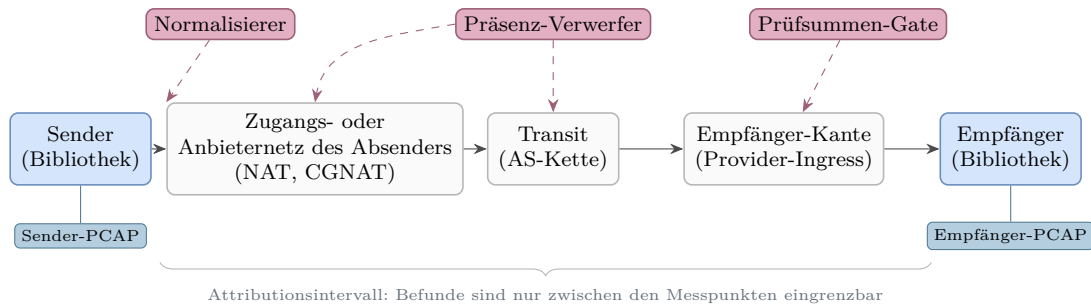


Abbildung 4.2: Kantenmodell eines Messpfads mit den erwarteten Mechanismenklassen

Modell rechnet. Ein **Normalisierer** baut Pakete aus seinem Verbindungszustand neu auf und schreibt die Längfelder ohne Surplus Area; das kann bei NAT-Stufen auftreten, die als Vermittler arbeiten, und führt zur Ergebnisklasse entfernt. Ein **Prüfsummen-Gate** rechnet die UDP-Prüfsumme fälschlich über die gesamte IP-Nutzlast statt nur bis `UDP Length` und verwirft Datagramme, deren Rechnung nicht aufgeht. Die OCS-Bauart aus Abschnitt 2.4.3 ist nach der Entwurfsabsicht des RFC genau darauf ausgelegt, diese Rechnung zu neutralisieren [2, Sec. 9]. Ein **Präsenz-Verwerfer** verwirft ein Datagramm allein deshalb, weil `UDP Length` kleiner ist als die IP-Nutzlast, unabhängig vom Inhalt der Surplus Area.

Die eingezeichneten Wirkorte sind typische Plätze: Die drei Mechanismenklassen sind nicht an die gezeigten Kanten gebunden. Befunde gelten deshalb nur für die jeweilige Messstrecke, denn Kanten können Verkehr quellabhängig behandeln. Zudem kann ein früher Eingriff einen späteren verdecken: Entfernt ein Normalisierer die Surplus Area, erreicht sie einen nachfolgenden Präsenz-Verwerfer nicht. Ob regelkonform gebaute Datagramme ein Prüfsummen-Gate auf einem konkreten Pfad tatsächlich passieren, prüft erst Kapitel 6.

Zwei Tabellen grenzen abschließend den Geltungsbereich ab: Tabelle 4.2 den Implementierungsumfang, Tabelle 4.3 die bewusst breitere Messinfrastruktur:

Aus Tabelle 4.2 geht der Kern des Artefakts hervor: Linux, IPv4, Benutzerraum und die acht *must-support*-Optionen; alles Weitere steht ausdrücklich außerhalb des Umfangs. Tabelle 4.3 zeigt, dass die Messinfrastruktur bewusst darüber hinausreicht: Der macOS-Messpunkt, die IPv6-Kontrollen und der WireGuard-Tunnel gehören zur Messrealität der untersuchten Pfade, nicht zur Bibliothek; Bewertungsgegenstand von FF2 bleiben allein die IPv4-Datagramme. Damit sind Maßstab und Messrahmen festgelegt; die Auswahl der Werkzeuge begründet der folgende Abschnitt.

Tabelle 4.2: Implementierungsumfang

Merkmal	Festlegung	außerhalb des Umfangs
Betriebssystem	Linux	andere Betriebssysteme
Vermittlungsschicht	IPv4	IPv6
Ausführungsort	Benutzerraum mit Raw Sockets	Kernelimplementierungen
Artefakt	Bibliothek mit Mess- und Beispielprogrammen	eigenständiger Dienst; Protokollautomaten für REQ/RES und TIME
Optionsumfang	die acht <i>must-support</i> -Optionen	alle übrigen SAFE- und UNSAFE-Arten (etwa TIME; reservierte wie AUTH, UCMP, UENC)

Tabelle 4.3: Messinfrastruktur

Baustein	Rolle in der Messung
Linux-Endpunkte: lokale virtuelle Maschine und Cloud-Instanzen in Europa, Nordamerika und Asien-Pazifik	Sende- und Empfangsendpunkte der Kampagnen; Mitschnitt mit <code>tcpdump</code>
Mobilfunk-Zugang über einen Hotspot	reale Zugangsnetz-Kette mit NAT und CGNAT vor dem ersten Transit
macOS-Messpunkt <code>en0</code> (<code>access_bpf</code>)	zusätzlicher Beobachtungspunkt am Zugangsnetz; keine Implementierungsplattform
native IPv6-Kontrollen	Einordnung einzelner Pfadbefunde; außerhalb der Bibliothek
WireGuard-Tunnel	Kontrollpfad mit Kapselung (Pfadstufe 3)

4.5 Technologieauswahl

Vorab ist die Form des Artefakts festzuhalten: Die Implementierung entsteht als *Bibliothek*, also als einbindbarer Baustein anderer Programme, nicht als eigenständiges Werkzeug und nicht als Kernelmodul. Der RFC sieht diese Form vor, ohne andere auszuschließen: Seine Entwurfsprinzipien verorten nötigen Zustand und jede Antwort in der Anwendung oder in einer in ihrem Auftrag arbeitenden Bibliothek (Abschnitt 2.4.5) [2, Sec. 3 und 6]. Gewählt wurde die Bibliothek, weil sie sich in die Messprogramme dieser Arbeit ebenso einbetten lässt wie in spätere Anwendungen; wenn diese Arbeit von der Implementierung spricht, meint sie diese Bibliothek samt ihren schmalen Mess- und Beispielprogrammen. Die Umsetzung verlangt vier Entscheidungen: die Wahl der Programmiersprache, den Zugang zur Surplus Area, die Werkzeuge für Mitschnitt und Auswertung sowie die Gegenproben jenseits der Tests. Dieser Abschnitt begründet sie in dieser Reihenfolge.

4.5.1 Programmiersprache

Die Wahl der Programmiersprache folgt aus vier Bedingungen. Erstens braucht die Bibliothek im Benutzerraum Zugriff auf das vollständige IP-Datagramm über die durch `UDP Length` begrenzten Nutzdaten hinaus (Abschnitt 2.5); den gewählten Zugang begründet Abschnitt 4.5.2. Zweitens verlangt das Wire-Format eine bytegenaue Kontrolle über das Datagramm, denn jede implizite Umordnung oder Auffüllung durch die Sprache würde die TLV-Kodierung und die OCS-Ausrichtung zerstören. Drittens verarbeitet der Parser Pakete, die jeder beliebige Absender im Netz erzeugt haben kann; er liegt damit an der *Vertrauensgrenze* des Systems, an der eingehende Daten als potentiell bösartig gelten müssen, und muss auch fehlerhafte oder gezielt konstruierte Pakete robust verarbeiten. Viertens soll die Bibliothek ohne eigene Laufzeitumgebung und ohne automatische Speicherbereinigung (Garbage Collector) auskommen, damit sie sich einbetten lässt und ihr Zeitverhalten vorhersagbar bleibt. Die ersten beiden Bedingungen folgen aus der Umsetzung im Benutzerraum, die letzten beiden sind Qualitätsziele dieser Arbeit.

Diese Bedingungen erfüllen *Systemprogrammiersprachen*: Sprachen mit hardware-nahem Zugriff auf Speicher und Betriebssystemschnittstellen, die ohne obligatorische Laufzeitumgebung zu nativem Maschinencode übersetzt werden; etablierte Vertreter sind C, C++, Zig und Rust. Sprachen mit Garbage Collector wie Go oder Java scheiden an genau der vierten Bedingung aus; das ist kein Eignungsurteil über diese Sprachen, sondern die Folge des bewusst gesetzten Qualitätsziels.

Alle vier Kandidaten bieten die geforderte Kontrolle; den Ausschlag gibt deshalb die dritte Bedingung. Unter *Memory Safety* wird die Abwesenheit von Speicherzu-

griffsfehlern verstanden: der Zugriff außerhalb der Grenzen eines Speicherbereichs (*Buffer Overflow*), der Zugriff auf bereits freigegebenen Speicher (*Use-after-free*) und die doppelte Freigabe (*Double Free*); das unterlassene Freigeben (*Memory Leak*) eröffnet dagegen keinen unzulässigen Zugriff und ist auch in Rust möglich. Diese Fehlerklasse ist keine Randerscheinung: Microsoft ordnet ihr rund 70 Prozent der jährlich durch die eigenen Sicherheitsupdates behobenen Schwachstellen zu, das Chromium-Projekt nennt denselben Anteil für seine Sicherheitsfehler hoher oder kritischer Schwere, und die National Security Agency (NSA) empfiehlt speichersichere Sprachen, darunter namentlich Rust [34–36].

Die entscheidungsrelevanten Eigenschaften der vier Kandidaten stellt Tabelle 4.4 stichpunktartig gegenüber:

Tabelle 4.4: Vergleich der Sprachkandidaten

Kriterium	C	C++	Zig	Rust
Speichersicherheit	nein	nein	teilweise	ja
Schutzmechanismus	keiner	optional	Laufzeit	Compiler, Laufzeit
Speicherverwaltung	manuell	manuell, RAII	Allokatoren	Ownership
Sprachfassung	ISO-Norm	ISO-Norm	vor 1.0	1.0 seit 2015

Aus Tabelle 4.4 geht hervor, dass allein Rust standardmäßige Speichersicherheit mit einer Speicherverwaltung ohne Garbage Collector verbindet: Der Compiler erzwingt die Zugriffsregeln überwiegend bereits zur Übersetzungszeit und damit für jeden Stand des Quelltexts, ergänzt um Bereichsprüfungen zur Laufzeit, die statt undefinierten Verhaltens einen definierten Fehlerzustand (*panic*) auslösen [37, 38]. C und C++ gewähren dieselbe Kontrolle ohne sprachseitige Garantien: Der C-Standard kennt gerade für Pufferüberläufe undefiniertes Verhalten [39], die Schutzmittel von C++ wie RAII und Smart Pointer bleiben optional, und die nachgelagerten Prüfwerkzeuge großer C-Projekte arbeiten nachweisend, denn sie melden nur die Fälle, die ihre Prüfläufe erreichen, und schließen die Fehlerklasse nicht aus [36, 40]. Zig prüft viele unzulässige Zugriffe erst zur Laufzeit und standardmäßig nur in den sicheren Build-Modi und hat noch keine Fassung 1.0 erreicht [41]; die praktische Relevanz dieses Unterschieds zeigt die in Abschnitt 3.6 als Referenzfall behandelte Migration der JavaScript-Laufzeit Bun von Zig nach Rust [31]. Aus diesen Gründen fiel die Wahl auf Rust.

Für die vorliegende Implementierung tragen drei Sprachmittel von Rust. Das erste sind geborgte Speichersichten: Der Parser liest die Surplus Area als geborgte Byte-Ausschnitte (*Slices*) direkt aus dem Empfangspuffer, ohne sie zu kopieren; nach dem *Ownership*-Modell borgen Referenzen einen Wert nur (*Borrowing*), und

der *Borrow Checker* prüft zur Übersetzungszeit, dass keine geborgte Sicht den Empfangspuffer überdauert, aus dem sie stammt (*Lifetime*) [38]. Das zweite sind algebraische Datentypen: Optionskennungen, Fehlerfälle und die fachlichen Ausgänge der Verarbeitung sind als `enum` modelliert, deren Fallunterscheidung der Compiler auf Vollständigkeit prüft; fehlerfähige Schnittstellen geben `Result` zurück. Das dritte ist das Schlüsselwort `unsafe`²: Operationen, deren Sicherheit der Compiler nicht selbst nachweisen kann, müssen ausdrücklich so markiert werden; die Garantien gelten damit für *Safe Rust*, also für den Code außerhalb solcher Blöcke, und sie setzen voraus, dass jeder `unsafe`-Block den Sicherheitsvertrag seiner Schnittstelle tatsächlich einhält, wofür dort die Entwicklung verantwortlich ist, nicht der Compiler. Wie wenige solche Blöcke die Bibliothek braucht und wo sie liegen, zeigt Kapitel 5. Auf externe Parser-Bibliotheken wurde bewusst verzichtet, da die Mechanik von RFC 9868 selbst Untersuchungsgegenstand ist und nicht hinter einer fertigen Abstraktion verschwinden soll.

Dem stehen Nachteile gegenüber: die Einarbeitung in das Ownership-Modell, ein gegenüber C jüngerer Ökosystem für hardwarenahe Netzprogrammierung und längere Übersetzungszeiten durch die statischen Prüfungen. Ungeachtet dieser Nachteile und aus dem Grund, dass die Speichersicherheit an der Vertrauensgrenze das ausschlaggebende Kriterium bildet, wurde die Entscheidung für Rust getroffen. Für den KI-gestützten Entwicklungsprozess aus Kapitel 3 trägt die Wahl in einer zweiten Rolle: Erzeugungsfehler werden zu sofortigen, strukturierten Übersetzungsdiagnosen, an denen das Sprachmodell vor der menschlichen Freigabe nacharbeitet, und die Speichersicherheitsprüfung konzentriert sich auf die wenigen gekapselten `unsafe`-Blöcke, sodass die Prüfkette aus Abschnitt 3.5 ihren Schwerpunkt auf die Protokolllogik legen kann. Dass dieser Schwerpunkt richtig liegt, zeigte sich früh: Der einzige in der Kernphase durch Prüfung gefundene Speicherfehler, ein Um-eins-Fehler bei ungeradem Beginn der Surplus Area, lag in sicherem Rust, überstand 23 selbstgeschriebene Tests und fiel erst der Zweitprüfung im Pull-Request auf; als Konsequenz erhielt jeder Schritt mit neuer Parsefläche verpflichtend Property-Tests und ein Fuzz-Ziel (Kapitel 5).

4.5.2 Zugang zur Surplus Area

Die zweite Entscheidung betrifft den Weg zur Surplus Area, denn ein gewöhnlicher UDP-Socket kann sie weder füllen noch auslesen (Abschnitt 2.5). Linux kennt daneben weitere Benutzerraumwege zum vollständigen Paket, namentlich Packet Sockets, virtuelle TUN/TAP-Geräte und `AF_XDP`; sie arbeiten jedoch auf der Verbindungsschicht

² Das Rust-Schlüsselwort `unsafe` ist von der `UNSAFE`-Optionsklasse aus Abschnitt 2.4.5 zu unterscheiden.

oder über eigene Netzgeräte beziehungsweise verlangen ein eigens in den Kernel geladenes Programm und damit mehr Unterbau, als die schmale Bibliothek dieser Arbeit braucht. Kernelnahe Verfahren wurden ebenso verworfen: Ein Kernelmodul hätte zwar vollen Zugriff auf den UDP-Pfad, verlagert die Implementierung aber in den privilegierten Kernelraum, bindet sie an Kernelversionen und erschwert die Tests und die Nachnutzung als gewöhnliche Bibliothek. Gewählt wurden deshalb Raw Sockets auf der IP-Schicht mit `IP_HDRINCL`, deren Mechanik Abschnitt 2.5 beschreibt. Als Betriebssystem trägt diese Entscheidung Linux: Der Weg ist dort dokumentiert [17], sein Verhalten ist im quelloffenen Kernel nachprüfbar, und dieselbe Plattform stellt alle Endpunkte der Kampagnen (Abschnitt 4.4). Der Preis ist bekannt und liegt bei den UDP-Prüfungen: Der Raw-Empfang umgeht den UDP-Pfad des Kernels, die Bibliothek übernimmt deshalb Längenprüfung, UDP-Prüfsumme und Optionsverarbeitung selbst und prüft den IPv4-Kopf zusätzlich; beim Senden ergänzt der Kernel einzelne Kopffelder wie die IP-Gesamtlänge und die Kopfprüfsumme selbst, und der Sendepfad fragmentiert nicht, sodass jedes Datagramm mit Optionen in die MTU passen muss [17]. Diese Konsequenzen prägen den Entwurf in Kapitel 5.

4.5.3 Mitschnitt und Auswertung

Die dritte Entscheidung betrifft die Messwerkzeuge. Da Wireshark in der verwendeten Version 4.6 die Surplus Area nicht dekodiert (Abschnitt 2.5), zeichnet `tcpdump` die Kampagnen skriptgesteuert auf; die PCAP-Dateien bleiben als archivierbare Beweisdateien jeder Kampagne erhalten. Die Deutung der Surplus Area übernimmt der eigene Prüfer aus Abschnitt 3.5, der OCS und Optionsstruktur aus den aufgezeichneten Bytes eigenständig nachrechnet; `tshark`, die Kommandozeilenfassung von Wireshark, dient daneben als fremde Gegenprobe für die IPv4- und UDP-Felder. Aufzeichnen, Deuten und Gegenprüfen übernehmen damit drei verschiedene Werkzeuge.

4.5.4 Fuzzing und formale Gegenprobe

Die vierte Entscheidung stellt der Testsuite zwei Gegenproben zur Seite, die beide auf das Gefahrenmodell der KI-gestützten Entwicklung (Abschnitt 3.4) antworten. Das über `cargo-fuzz` eingebundene libFuzzer-Fuzzing sucht abdeckungsgeleitet im echten Parsercode nach Abstürzen und verletzten Invarianten und mutiert dabei jenseits der von der Entwicklung vorausgewählten Beispieleingaben [42]. Das Lean-Modell aus Abschnitt 3.3 prüft ausgewählte Invarianten mit maschinengeprüften Beweisen an handgeschriebenen Modellen; Ablauf, Fuzz-Ziele und Beweisumfang beschreibt Abschnitt 3.5. Beide Gegenproben ergänzen sich spiegelbildlich: Das Fuzzing prüft den echten Code und beweist nichts; der Beweis gilt ausnahmslos, aber nur für das Modell, nicht für den Rust-Code, der es umsetzt. Dass daneben auch die Norm selbst

Prüfgegenstand bleiben muss, zeigte die Schlussphase der Implementierung: Ein nachträglicher RFC-Abgleich fand trotz grüner Prüfkette noch Konformitätslücken und wurde als eigener Korrekturschritt umgesetzt (Kapitel 5).

Damit sind die Form des Artefakts und die vier Entscheidungen begründet; Kapitel 5 baut mit ihnen das Messinstrument dieser Arbeit.

5. Entwurf und Implementierung

Dieses Kapitel baut das Messinstrument dieser Arbeit: die Bibliothek, die RFC 9868 unter den Vorgaben aus Kapitel 4 umsetzt, und die Prüfanlage, die diesen Bau absichert. Zunächst zeigt Abschnitt 5.1, wie dieser Bau entstand; danach ordnet Abschnitt 5.2 die Komponenten der Bibliothek und verankert sie in den Anforderungskennungen des Implementierungs-Repositorys (Abschnitt 4.2). Es folgen die beiden Verarbeitungswege: Abschnitt 5.3 verfolgt ein Beispieldatagramm von den Nutzdaten bis auf den Draht, Abschnitt 5.4 den umgekehrten Weg vom Draht bis zur Auslieferung, und Abschnitt 5.5 vertieft das Serialisieren und Parsen der Optionen. Ferner zeigt Abschnitt 5.6 die gebaute Prüfanlage mit ihren Nachweisspuren und Abschnitt 5.7 die Betriebs- und Sicherheitsgrenzen; abschließend benennt Abschnitt 5.8 die bewussten Grenzen des Baus.

Dabei bleibt die Trennung aus Abschnitt 3.1 bestehen: Dieses Kapitel beschreibt und begründet den Bau; die abschließende Erfüllungsbewertung und die Ergebnisse der Messkampagnen gehören zu Kapitel 6. Befunde aus Vorversuch und Prüfbetrieb erscheinen nur dort, wo sie eine Entwurfsentscheidung begründen oder die Prüfanlage absichern. Die vier Entscheidungen aus Abschnitt 4.5 werden hier angewandt und nicht neu begründet; ebenso setzt das Kapitel die Mechanik der Surplus Area aus Kapitel 2 voraus und greift sie nur dort auf, wo der Entwurf ihr eine konkrete Gestalt gibt.

5.1 Vorgehen der Implementierung

Bevor die Architektur den fertigen Bau zeigt, gehört festgehalten, wie er entstand, denn das Vorgehen ist unter KI-Beteiligung selbst ein Schutzmechanismus (Abschnitt 3.4). Die Umsetzung begann mit dem Plan, nicht mit dem Code: Der erste Commit des Repositorys legte den vollständigen Projektplan an, achtzehn Schrittdateien mit je Ziel, Anforderungen, Plan, Aufgaben und Abschlusskriterien, dazu die Modulgerüste.¹ Der Plan bezeichnet sich selbst als schrittweise abzuarbeitenden Masterplan mit einem Menschen in der Schleife; der Anforderungskatalog stand damit vor dem ersten Funktionsschritt.

Je Schritt lief der Kernzyklus aus Abbildung 3.3: ein eigener Arbeitszweig, die Umsetzung mit Tests im selben Commit, das lokale Gate mit am Ende 18 Prüfläufen,

¹ Commit `3c17252` vom 30.05.2026; die Detailpläne, der Anforderungskatalog (Abschnitt 4.2) und die Architektur- und Formatreferenzen folgten am selben Tag mit Commit `36cc465`.

der Pull-Request mit der kleineren CI-Kette und zwei angeforderten KI-Prüfungen, zuletzt die menschliche Durchsicht des Diffs und genau ein freigegebener Sammelcommit auf dem Hauptzweig; nur die Schritte 14, 15 und 17 teilten sich einen solchen Commit. Die Regeln dazu stehen versioniert im Repository selbst, einschließlich des Grundsatzes, dass jede Änderung vor der Übernahme von einem Menschen gelesen und verantwortet wird.

Der Plan blieb dabei beweglich: Ein Machbarkeits-Vorversuch (Schritt 0.5, Abschnitt 5.4) und die unabhängige Wire-Prüfung (Schritt 10.5, Abschnitt 5.6) wurden eingeschoben, Schritt 9 ging in Schritt 8 auf, und die zunächst mitgebaute IPv6-Unterstützung wurde mit Schritt 16 samt der Katalogzeile FR-47 wieder aus dem Umfang entfernt (Abschnitt 5.8). Den Abschluss bildete kein Funktionsschritt, sondern eine Prüfung: Schritt 18 hielt die Befunde eines nachträglichen RFC-Abgleichs als eigene Schrittdatei fest und arbeitete sie gesammelt ab, darunter die Behandlung unbekannter UNSAFE-Optionen vor einem FRAG und den vollständigen Statusabgleich des Anforderungskatalogs.²

Dass diese Schleifen nicht Förmerei waren, zeigen die Fänge: Die Prüfung gegen den RFC-Primärtext fand vier Fehler in der eigenen Regeldatei des Repositories, darunter eine falsch wiedergegebene Ausrichtungsregel; die Zweitprüfung im Pull-Request fing den Um-eins-Fehler der Surplus-Ortung, den 23 selbstgeschriebene Tests übersehen hatten (Abschnitt 5.6); und bei Schritt 18 musste eine erste, vom Sprachmodell vorgeschlagene Korrektur selbst noch einmal korrigiert werden, weil sie hinter der unbekannten UNSAFE-Option weiter nach FRAG suchte. Ebenso klar sind die Grenzen der Belegbarkeit: Die Sammelcommits verschmelzen die innere Reihenfolge eines Schritts, die KI-Prüfungen waren angefordert, aber nicht erzwungen, und für das Lean-Gate ist kein Fang eines echten Produktfehlers belegt; es sicherte den Modellbestand (Abschnitt 5.6). Mit diesem Vorgehen im Rücken zeigt der Rest des Kapitels den fertigen Bau.

5.2 Gesamtarchitektur

Den Rahmen des Baus legt Kapitel 4 fest: die Plattformvorgabe mit Linux, IPv4 und dem Benutzerraum mit Raw Sockets sowie die vier Entscheidungen aus Abschnitt 4.5, die dort begründet sind und hier angewandt werden. Innerhalb dieses Rahmens entsteht das Artefakt aus Tabelle 4.2: eine Bibliothek, die die Surplus Area baut, liest und prüft, samt zwei schmalen Messprogrammen für die Kampagnen.

² Commit 4c82ab0 vom 02.08.2026 mit 53 geänderten Dateien; die Schrittdatei ist `docs/plan/steps/18-rfc9868-audit-remediation.md`.

Die Bibliothek³ besteht aus acht Modulen und zwei Messprogrammen, zusammen rund 8.600 physische Rust-Zeilen⁴: Sechs Module tragen je einen fachlichen Baustein, vom Wire-Format bis zur öffentlichen Schnittstelle, zwei liegen als Querschnitt unter allen. Komponente meint in diesem Kapitel einen solchen Baustein zusammen mit seiner Quelleinheit: Der Modulname bestimmt zugleich die Quelleinheit unter `src/`, ein gleichnamiges Verzeichnis oder eine gleichnamige Rust-Datei.

Die Module, die Programme und ihre Importbeziehungen zeigt Abbildung 5.1:

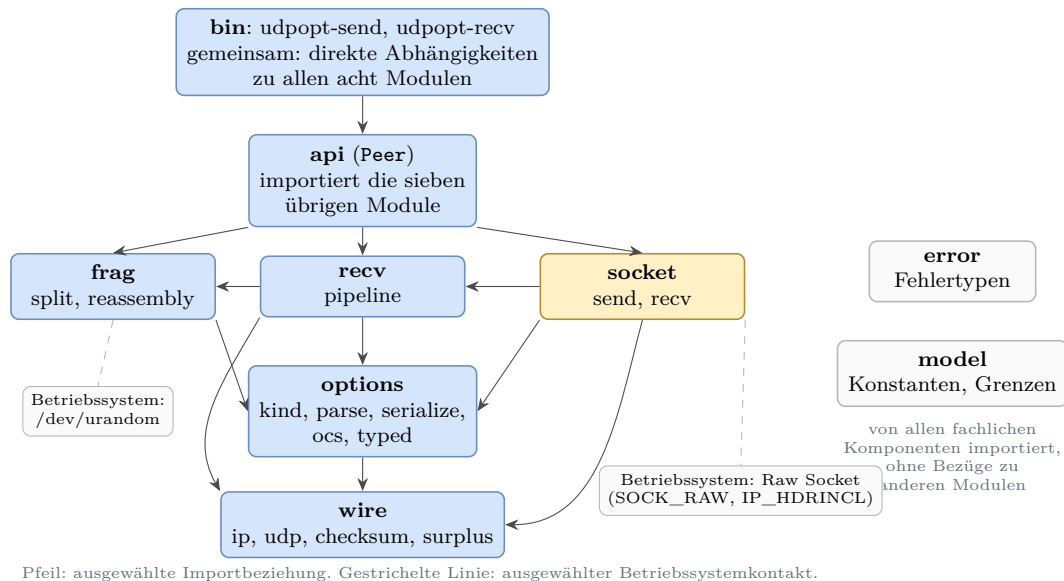


Abbildung 5.1: Komponenten der Bibliothek und ihre Abhängigkeiten

Aus Abbildung 5.1 geht zweierlei hervor. Erstens zeigen die Kanten ausgewählte tatsächliche Importe und bilden keine strikte Schichtung; so nutzt die Socket-Schicht für eine Längendiagnose einen Helfer der Empfangspipeline. Zweitens liegen die Netz-Systemaufrufe innerhalb der Bibliothek allein in `socket`, und mit ihnen die beiden `unsafe`-Blöcke in `src/` (Abschnitt 4.5.1), gekapselt hinter sicheren Schnittstellen. Daneben liest `frag` für die FRAG-Identifikation das Zufallsgerät `/dev/urandom`, `api` und `socket` fragen für Fristen die monotone Uhr ab, und die beiden Messprogramme kommen mit Konsolenausgabe hinzu; `udpopt-send` kann zusätzlich eine JSONL-Manifestdatei fortschreiben. Daraus folgt eine Eigenschaft, die die Prüfarchitektur in Abschnitt 5.6 trägt: Die reinen Protokollpfade erhalten Zeitpunkte und Identifikation als Parameter, jede Protokollentscheidung ist damit ohne Netz und ohne erhöhte Rechte prüfbar.

Unten liegt `wire`, die reine Bytesicht: IPv4-Kopf, UDP-Kopf, Prüfsummenarith-

³ Implementierungs-Repository `udp-transport-options`, Commit `f847895`; alle Datei- und Zeilenangaben dieses Kapitels beziehen sich auf diesen Stand.

⁴ Gezählt mit `wc -l` über alle `.rs`-Dateien der neun Quelleinheiten unter `src/` (sieben Verzeichnisse sowie die Einzeldateien `error.rs` und `model.rs`), ohne die 40 Zeilen von `src/lib.rs`: genau 8.606 Zeilen.

metik und die Ortung der Surplus Area. Hier wird der IPv4-Kopf geprüft und nur `Protocol 17` angenommen (Umfangsvorgabe aus Abschnitt 4.1), und hier liegt die Ortungsregel, nach der Optionen an jedem gültigen `UDP Length`-Offset beginnen dürfen [2, Sec. 8]: Aus Gesamtlänge, Kopflänge und `UDP Length` bestimmt `wire` Beginn, Pad-Bedarf und Länge der Surplus Area, ohne ein einziges Optionsbyte zu deuten.

Darüber teilen sich drei Komponenten die Optionslogik. `options` trägt den Optionskörper: Optionsverzeichnis, Parser, Serialisierer, die typisierten Optionswerte und die OCS-Rechenregeln (FR-19, FR-21). `recv` enthält mit der Pipeline die Empfangsordnung, die für jedes an sie übergebene Datagramm über Annahme oder Verwerfen entscheidet (FR-20, FR-38). `frag` zerlegt große Nachrichten in Fragmente und setzt sie unter begrenztem Speicher wieder zusammen (FR-34, NFR-06). Die acht *must-support*-Optionen ([2, Sec. 10]) verteilen sich über diese drei Komponenten; die Vertiefung folgt in Abschnitt 5.4 bis Abschnitt 5.7.

Oben bündelt `api` alles im Typ `Peer`, der Gegenstelle als Bibliothekstyp: Eine Anwendung konfiguriert einen `Peer` und erhält Senden und Empfangen als je einen Aufruf, ohne Socket-Verwaltung, Prüfreihefolge oder Optionsbau selbst zu übernehmen; Aufruferpflicht bleiben die erhöhten Rechte für Raw Sockets, die Wahl der Empfangsstrengigkeit und der anwendungsseitige Protokollzustand, etwa die Herkunft eines RES-Tokens; Rechte und Strengigkeit behandelt Abschnitt 5.7. Die zwei Programme `udpopt-send` und `udpopt-recv` sind schmale Messaufsätze für die Kampagnen. Quer zu allem liegen `error` mit den Fehlertypen und `model` mit den Protokollkonstanten sowie den Reassemblierungs- und Schutzgrenzen: von allen fachlichen Komponenten importiert, selbst ohne Abhängigkeiten zu anderen Bibliotheksmodulen. Wie ein Datagramm dieses Gefüge von oben nach unten durchläuft, zeigt der Sendepfad.

5.3 Sendepfad

Beim Senden baut die Bibliothek aus Nutzdaten und gewünschten Optionen ein vollständiges IPv4-Datagramm und übergibt es dem Raw Socket; den Rahmen setzt die Plattformvorgabe aus Kapitel 4, beschrieben wird durchgehend der IPv4-Weg unter Linux. Als roter Faden dient von hier an ein festes Beispieldatagramm, das die Darstellung bis in die Evaluation begleitet: drei Nutzdatenbytes `odd` und genau eine REQ-Option mit dem vier Byte langen Token `c0ffee01`.⁵ Unter den vorbereiteten Prüfscenarien der Bibliothek ist es das kleinste, das alle vier Bauelemente der Surplus

⁵ Prüfscenario `pad-odd` in `examples/wire_probe.rs` (Zeilen 248 bis 258); die erwarteten Bytes hält unabhängig davon das Prüfprogramm `scripts/wire-check.py` fest (Zeilen 248 bis 250 und 272).

Area zugleich enthält: das Pad-Byte, das OCS, eine TLV-Option und die Nullfüllung nach EOL.

Wie aus Nutzdaten und Optionsliste das fertige Datagramm des Beispiels entsteht, zeigt Abbildung 5.2 in vier Schritten:

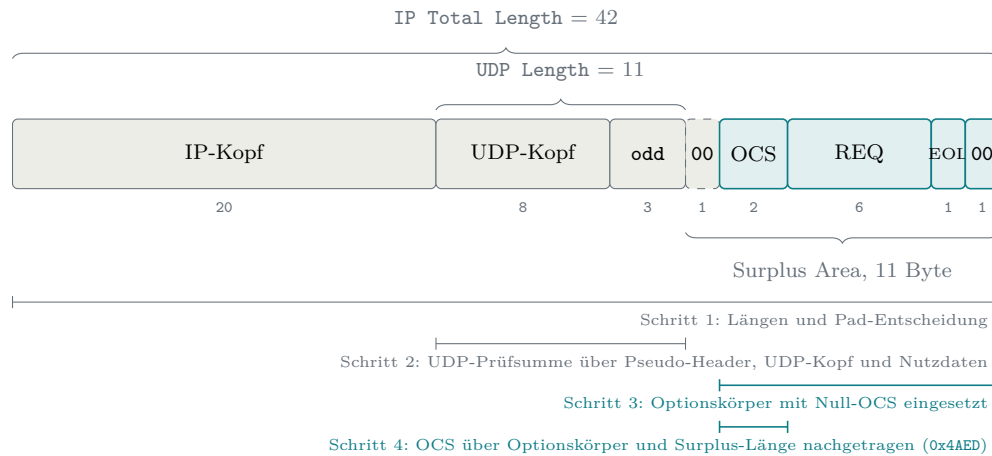


Abbildung 5.2: Entstehung des Beispieldatagramms in vier Schritten

Aus Abbildung 5.2 geht hervor, dass die Schrittfolge nicht beliebig ist: Die UDP-Prüfsumme in Schritt 2 deckt Pseudo-Header, UDP-Kopf und Nutzdaten (Abschnitt 2.2.1), das OCS in Schritt 4 den Optionskörper samt Surplus-Länge (Abschnitt 2.4.3); die beiden Bereiche ergänzen sich ohne Überschneidung, und das OCS folgt zuletzt, weil in seine Summe der fertige Optionskörper und die endgültige Surplus-Länge eingehen. Schritt 1, die Längenrechnung, zeigt der folgende Ausschnitt aus der Aufbaufunktion⁶, gekürzt um die Überlauf- und Bereichsprüfungen des Originals:

```
let udp_len = udp::HEADER_LEN + user_data.len();
let natural_start = usize::from(length::UDP_HEADER) + 20 + user_data.len();
let needs_pad = !options_body.is_empty() && natural_start % 2 == 1;
let surplus_len = if options_body.is_empty() {
    0
} else {
    usize::from(needs_pad) + options_body.len()
};
let total_len = 20 + udp_len + surplus_len;
```

Für das Beispiel ergibt die Rechnung: UDP Length $8 + 3 = 11$, und die Surplus Area beginnt hinter IP-Kopf, UDP-Kopf und Nutzdaten bei Byte $20 + 11 = 31$. Dieser Offset ist ungerade, nach der Paritätsregel aus Abschnitt 2.4.2 fällt also genau ein Pad-Byte an. Der Optionskörper aus dem Serialisierer ist zehn Byte lang (zwei Byte OCS-Platzhalter, sechs Byte REQ, EOL und ein Füllbyte auf gerade Länge; Abschnitt 5.5), die Surplus Area damit $1 + 10 = 11$ Byte und die IP Total Length

⁶ `assemble_datagram` in `src/socket/send.rs` (Zeilen 25 bis 92); der Ausschnitt gibt die Zeilen 38 bis 56 wieder.

20 + 11 + 11 = 42 Byte. Sichtbar wird hier zugleich die Senderseite der Offset-Regel aus [2, Sec. 8]: Optionen dürfen an jedem gültigen UDP `Length`-Offset beginnen, und die Bibliothek wählt stets den natürlichen Start unmittelbar hinter den Nutzdaten.

Geschrieben wird anschließend in einen mit Nullen vorbelegten Puffer der Gesamtlänge: der IP-Kopf, der UDP-Kopf mit der bereits berechneten UDP-Prüfsumme, die Nutzdaten und der Optionskörper an seiner Zielposition. Das Pad-Byte wird dabei nie eigens geschrieben: Es bleibt die Null der Puffervorbelegung und erfüllt so die Pflicht aus [2, Sec. 8], nach der Ausrichtungsbytes vor dem OCS null sein müssen (FR-05). Zum Schluss rechnet die OCS-Routine der Komponente `options` über den eingesetzten Körper und trägt das Ergebnis in den Platzhalter ein.

Bei den Prüfsummen begegnen sich zwei Nullregeln aus Abschnitt 2.2.1 und Abschnitt 2.4.3. Die UDP-Prüfsumme kennzeichnet mit einer gespeicherten Null den Verzicht auf eine Prüfsumme; eine errechnete Null wird deshalb als das im Einerkomplement gleichwertige `0xFFFF` übertragen [1]. Für das OCS verlangt der RFC einen Wert ungleich null, sobald die UDP-Prüfsumme ungleich null ist; die Null bleibt der Kennzeichnung eines unbenutzten OCS vorbehalten und ist nur bei UDP-Prüfsumme null zulässig [2, Sec. 9]. Eine ausdrückliche Senderegeln für eine errechnete OCS-Null enthält RFC 9868 nicht; weil der Nullwert die unbenutzte Form kennzeichnet, bleibt für ein verwendetes OCS im Einerkomplement aber nur die Abbildung auf `0xFFFF`. Beide Abbildungen stehen an je genau einer Stelle: für die UDP-Prüfsumme in `wire`, für das OCS in `options`.⁷

Die OCS-Rechnung selbst folgt der Regel aus Abschnitt 2.4.3: In die Einerkomplementsumme geht neben dem Optionskörper die Länge der gesamten Surplus Area als 16-Bit-Summand ein, obwohl sie in keinem Feld des Pakets steht; der Sender bestimmt sie beim Bau aus Pad-Bedarf und Optionskörperlänge, der Empfänger leitet sie aus IP- und UDP-Länge ab, und der Entwurfsgrund, die Kompensation fehlerhaft rechnender Zwischensysteme, ist ebenfalls dort beschrieben [2, Sec. 9]. Am Beispiel: Die 16-Bit-Wörter des Optionskörpers summieren sich mit dem Platzhalter null zu `0xB507`, der Längensummand 11 hebt die Summe auf `0xB512`, und invertiert entsteht das OCS `0x4AED`. Die Probe des Empfängers geht auf: `0xB507`, `0x4AED` und `0x000B` summieren sich zu `0xFFFF`.

Zwischen Puffer und Leitung steht der Kernel, denn bei `IP_HDRINCL` liefert die Bibliothek den IP-Kopf selbst an. Nach der Raw-Socket-Dokumentation füllt der Kernel dabei die IP `Total Length` und die IP-Kopfprüfsumme stets selbst aus [17]; der eigene Mitschnitt kann diese allgemeine Aussage nur für die beobachteten Felder bestätigen. Die Gegenseite der Arbeitsteilung zeigt er dafür deutlich: UDP-Kopf und

⁷ `src/wire/udp.rs` (Zeilen 85 bis 88) und `src/options/ocs.rs` (Zeilen 55 bis 58).

Surplus Area bleiben unangetastet. Im handgebaute Prüfzenario `cksum0-ocs0`, das das UDP-Prüfsummenfeld auf null lässt und den OCS-Platzhalter nie füllt, lag das Nullfeld unverändert auf der Leitung; die von RFC 9868 zugelassene Kombination aus ungenutzter UDP-Prüfsumme und ungenutztem OCS ist aus dem Benutzerraum also exakt konstruierbar [2, Sec. 9].

Derselbe Sendeweg setzt zugleich eine harte Größengrenze: Der `IP_HDRINCL`-Pfad fragmentiert nicht auf IP-Ebene, ein Datagramm oberhalb der MTU weist der Kernel mit dem Fehlercode `EMSGSIZE` („Nachricht zu lang“) ab. Die in Abschnitt 2.1 beschriebene IP-Fragmentierung steht dem Raw-Pfad damit nicht zur Verfügung; das ist ein Befund dieses Sendewegs, keine Vorgabe des RFC. Seine eigene Fragmentierungsoption begründet der RFC unabhängig davon: FRAG überträgt Nachrichten oberhalb der Empfangsgrenze `EMTU_R` und kopiert die UDP-Ports in jedes Fragment, sodass die Fragmente zustandslose NAT-Geräte durchqueren können [2, Sec. 11.4]; die Vertiefung folgt in Abschnitt 5.5. Die Bibliothek zieht im hohen Sendepfad die Konsequenz: `Peer::send` prüft gegen eine konfigurierbare Obergrenze der Datagrammlänge (Voreinstellung 1500 Byte) und zerlegt größere Nachrichten standardmäßig selbst in FRAG-Fragmente, sofern die konfigurierten MRDS- und Segmentgrenzen der Gegenstelle das zulassen; andernfalls endet der Aufruf mit `SendError::Split`.⁸ In Abschnitt 4.3 ist `EMSGSIZE` deshalb bereits als erwartete Prüflast der Sendekampagnen eingeordnet.

Am Ende des Pfads steht der Systemaufruf. Die Socket-Schicht öffnet den Raw Socket mit den Crates `socket2` und `libc` [43, 44] und schaltet `IP_HDRINCL` per `setsockopt` ein; dieser Aufruf ist der erste der beiden in Abschnitt 5.2 angekündigten `unsafe`-Blöcke in `src/` (`src/socket/send.rs`, Zeile 131). Er bleibt hinter einer sicheren Hilfsfunktion gekapselt, und ein `SAFETY`-Kommentar begründet, warum Zeiger und Längen des Aufrufs gültig sind (Abschnitt 4.5.1). Auf der Gegenseite kehrt sich die Reihenfolge um, und die Verantwortung wächst: Der Empfänger kann sich auf keine dieser Zusicherungen verlassen und prüft alles selbst.

5.4 Empfangspfad

Auf der Empfangsseite erhält der Raw Socket seine Kopie des Datagramms, bevor der UDP-Pfad des Kernels Länge, Prüfsumme und Portzuordnung geprüft hat (Abschnitt 2.5, Abbildung 2.11). Aus dieser Lage folgt die Eigenverantwortung für den UDP-Anteil: Die im Raw-Pfad ausgelassenen Kernelprüfungen und die Portauswahl muss die Bibliothek selbst leisten, bevor sie auch nur ein Byte der Surplus Area

⁸ Obergrenze und Modus mit ihren Voreinstellungen in `src/api/mod.rs` (Zeile 33 sowie Zeilen 114 bis 133); die Zerlegung übernimmt `build_outgoing_datagrams` (ab Zeile 362).

deutet.

Wie wörtlich das gemeint ist, zeigte ein Vorversuch am Beginn der Implementierung: eine Messreihe mit 97 Fällen über ein virtuelles Netzschnittstellenpaar (MTU 1500) zwischen zwei Netznamensräumen, davon 92 aus einem Kreuzprodukt von Nutzdaten- und Surplus-Größen um die MTU-Grenze, ein Fall weit oberhalb der MTU und vier gezielte Kopfanomalien.⁹ 69 Fälle erreichten den Raw-Empfang, die übrigen 28 scheiterten erwartungsgemäß schon beim Senden mit `MSGSIZE` (Abschnitt 5.3). Das zentrale Ergebnis für den Empfang sind die vier Anomalien: ein Datagramm mit UDP-Prüfsumme null, eines mit absichtlich falscher Prüfsumme, eine UDP `Length` von 500 bei 48 verfügbaren Bytes und eine UDP `Length` von 4, kürzer als der UDP-Kopf selbst. Alle vier erreichten den Raw Socket; der Versuch prüfte dabei nur die Ankunft und protokollierte Längenfelder und Surplus-Bytes, ein Byte-für-Byte-Vergleich mit dem Sendepuffer gehörte nicht zum Aufbau. Daneben bestätigte die Messreihe die Arbeitsteilung aus Abschnitt 5.3: Von den 69 empfangenen Fällen trugen 42 im Sendepuffer ein absichtlich falsches `Total Length`-Feld, und der Kernel schrieb jeden dieser Werte vor der Übertragung auf die tatsächliche Pufferlänge um. Aus dem Vorversuch folgt die Grundentscheidung des Empfangspfads: Die Bibliothek prüft Längenkonsistenz, UDP-Prüfsumme, OCS und erst dann die Optionen selbst, in der Empfangsordnung des RFC [2, Sec. 10 und 14]; die vorgelagerte Prüfung des IP-Kopfs kommt als eigene zusätzliche Kontrolle dieser Bibliothek hinzu.

Die vier Prüfungen bis zur Surplus-Ortung zeigt Listing 5.1 in gekürzter Form; ausgelassene Fehlerdetails sind mit `..` markiert:

```
1 let (ip, udp_at) = IpRepr::parse(ip_datagram)?;
2 let udp = UdpHeader::parse(&datagram[udp_at..])?;
3 if usize::from(udp.length) > ip.transport_payload_len() {
4     return Err(RecvError::UdpLengthExceedsIpPayload { .. });
5 }
6 if udp.checksum != 0 {
7     let expected = UdpHeader { checksum: 0, ..udp }
8         .compute_checksum(&ip, user_data);
9     if expected != udp.checksum {
10         return Err(RecvError::UdpChecksumMismatch { .. });
11     }
12 }
13 let Some(layout) = locate_surplus(&ip, &udp) else {
14     return deliver_without_options(false, OcsStatus::Absent);
15 };
```

Listing 5.1: Prüfreihefolge der Empfangspipeline bis zur Surplus-Ortung (gekürzt)

Zeile für Zeile entspricht das der Vorgabe: `IpRepr::parse` nimmt nur Datagramme mit `Protocol 17` an (Umfangsvorgabe) und weist dabei auch eine falsche IP-Kopfprüfsumme ab; `UdpHeader::parse` verlangt eine UDP `Length` von mindestens

⁹ Fallgenerator `cases()` in `examples/spike_client.rs` und `examples/spike_server.rs` (beide Seiten bauen dieselbe Liste); Protokoll des Vorversuchs in `docs/plan/steps/00b-spike.md`.

8; der Längenvergleich verwirft Datagramme, deren `UDP Length` über die IP-Nutzlast hinausweist, wie im Anomaliefall 500 bei 48; und die UDP-Prüfsumme wird genau dann geprüft, wenn ihr Feld nicht null ist, denn eine gespeicherte Null ist keine Prüfsumme (Abschnitt 5.3). Jeder Fehler bis hierher verwirft das ganze Datagramm als Fehlerwert der Pipeline. Damit ist die in Abschnitt 2.5 angekündigte Reihenfolge (Länge und Längenkonsistenz, dann Prüfsumme, erst danach Surplus-Ortung und OCS) im Quelltext eingelöst.¹⁰

Hinter dieser Eingangsprüfung verzweigt die Pipeline in ihre Ergebnisarten; den Lauf des reinen Verarbeitungskerns `process_datagram`, gekürzt um die Sonderpfade, zeigt Abbildung 5.3:

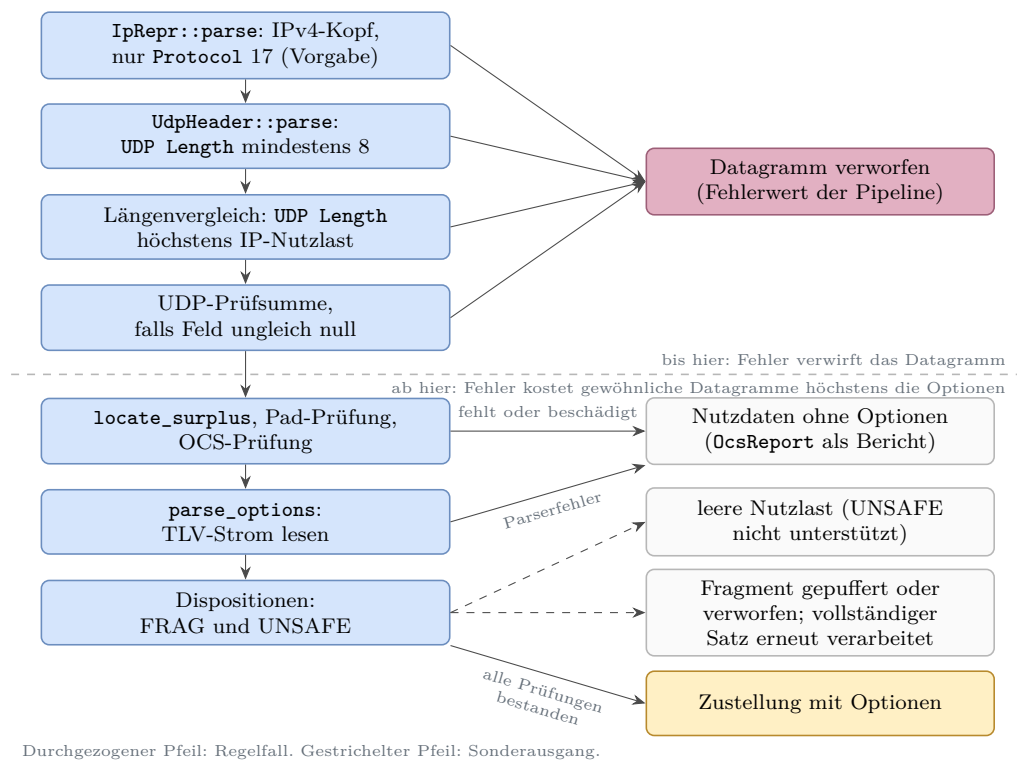


Abbildung 5.3: Empfangspipeline der Bibliothek mit ihren Ergebnisarten

Aus Abbildung 5.3 geht die Grenze der gewöhnlichen Empfangsverarbeitung hervor: Ein Fehler bis einschließlich der UDP-Prüfsumme kostet das ganze Datagramm, ein Fehler danach höchstens die Optionen; die gestrichelten Sonderausgänge für UNSAFE und FRAG vertieft Abschnitt 5.5; der Regelausgang schließt eine leere sichtbare Optionsliste ein, denn FRAG, NOP und EOL erscheinen nicht als sichtbare Optionen. Anders als Abbildung 2.8, die die Normentscheidung des RFC zeigt, steht hier die Umsetzung. Das Beispieldatagramm aus Abschnitt 5.3 nimmt den geraden Weg und liefert die drei Nutzdatenbytes samt REQ-Option.

¹⁰ Einstieg `process_datagram` in `src/recv/pipeline.rs` (Zeilen 201 bis 203); die gezeigte Prüf-
folge steht in den Zeilen 211 bis 266.

Die Surplus-Ortung selbst trägt eine Feinheit des Normtexts: „Optionen vorhanden“ verlangt mehr als einen Überschuss größer null, denn RFC 9868 bindet die Deutung der Surplus Area daran, dass das ausgerichtete OCS samt einem etwaigen Pad-Byte hineinpasst [2, Sec. 8]. Die Ortungsfunktion der Komponente `wire` setzt das um (Zeilenumbrüche angepasst):¹¹

```
pub fn locate_surplus(ip: &IpRepr, udp: &UdpHeader) -> Option<SurplusLayout> {
    let surplus = ip.transport_payload_len()
        .checked_sub(usize::from(udp.length))?;
    if surplus == 0 {
        return None;
    }
    let natural_start = ip.header_len() + usize::from(udp.length);
    let needs_pad = natural_start % 2 == 1;
    if surplus < usize::from(needs_pad) + usize::from(OCS) {
        return None;
    }
    Some(SurplusLayout { starts_at: natural_start, needs_pad, len: surplus })
}
```

Ein Überschuss von einem Byte (oder von zwei Bytes bei ungeradem Start) trägt danach keine Optionen. Ihn deshalb auch nicht als Fehler zu behandeln, ist die aus dieser Formulierung gezogene Entwurfsentscheidung der Bibliothek: Die Funktion liefert `None`, die Pipeline stellt die Nutzdaten unverändert zu, und der Bericht vermerkt mit `Absent`, dass kein nutzbarer Optionsbereich vorlag. Die Mindestgrößen dahinter hat Abschnitt 2.4.2 hergeleitet; hier entscheiden sie über den Weg im Code.

Was bei einem beschädigten Optionsbereich geschieht, regeln die Empfängerpflichten satzgenau. Weil Optionen senderseitig an jedem gültigen UDP `Length`-Offset beginnen dürfen (Abschnitt 5.3), muss der Empfänger jede solche Lage verarbeiten; verlangt sind Nullbytes vor dem OCS, und steht dort etwas anderes, sind alle Optionen zu ignorieren und die Surplus Area still zu verwerfen. Dieselbe Disposition, also derselbe Verarbeitungsausgang, gilt bei fehlgeschlagener OCS-Prüfung und bei Längenwerten, die über die Surplus Area hinausweisen (in der durch das Erratum konsolidierten Fassung) [2, Sec. 8, 9 und 10][15]. Verworfen wird dabei die Surplus Area, nicht das Datagramm: Nutzdaten mit bestandener oder zulässig ungenutzter UDP-Prüfsumme werden weiter zugestellt. In der Pipeline führt jeder dieser Wege, ebenso wie jeder andere Parserfehler im TLV-Strom, über dieselbe Hilfsfunktion `deliver_without_options`, die die leere Optionsliste liefert und den OCS-Status getrennt mitführt; der konkrete TLV-Parserfehler wird nur protokolliert. Diese Grundregel gilt für gewöhnliche Datagramme; eine nicht unterstützte UNSAFE-Option führt stattdessen zur Zustellung mit leerer Nutzlast, ein fragmentlokaler Fehler nach bereits erkanntem gültigem FRAG zum Verwerfen des Fragments (Abschnitt 5.5).

Ob der Empfänger strenger sein darf, regelt die Voreinstellung des RFC: Stan-

¹¹ `locate_surplus` in `src/wire/surplus.rs` (Zeilen 52 bis 67).

dardmäßig verhält sich ein optionsfähiger Empfänger wie ein Altempfänger und stellt auch bei fehlgeschlagener OCS-Prüfung zu; strengeres Verhalten muss die Anwendung ausdrücklich anfordern (Abschnitt 2.4.5). Die Bibliothek bildet das in der Empfangsrichtlinie `ReceivePolicy` ab, mit drei Stellschrauben: Pflichtoptionen (nur für meldefähige Optionsarten), das Ausfiltern aller optionstragenden Datagramme und eine OCS-Pflicht, die sowohl ein gültiges als auch ein zulässig ungenutztes OCS erfüllt.¹² Alle drei sind ab Werk aus. Ein Betreiber, der die OCS-Pflicht anfordert, gewinnt die Zusicherung, dass Optionen nur mit bestandenem oder zulässig ungenutztem OCS zugestellt werden, und verliert dafür Datagramme mit unterwegs beschädigter Surplus Area; das Ausfiltern aller optionstragenden Datagramme verzichtet darüber hinaus auch auf gültige Optionen. Eine Grenze bleibt: Das Prinzip des RFC nennt neben der OCS- auch die Authentifizierungs- und Entschlüsselungsprüfung; AUTH und UENC sind nicht implementiert (Tabelle 4.2), dieser Teil des Prinzips liegt außerhalb des Umfangs.

Sichtbar werden diese Entscheidungen über zwei Berichtstypen. Der `OcsReport` führt Quelle und `OcsStatus`; der Status meldet in sechs Ausprägungen, was aus der OCS-Prüfung wurde: kein nutzbarer Optionsbereich (`Absent`), gültig (`Valid`), zulässig ungenutzt (`Unused`), fehlgeschlagen (`Failed`), unzulässige Null (`InvalidZero`) und nicht beobachtet (`Unobserved`). Je sichtbarer TLV-Option meldet daneben ein `OptionReport` die Optionsart, die Quelle und den Ausgang: Erfolg, Fehlschlag oder bewusstes Übergehen; FRAG, NOP und EOL bleiben intern.¹³ Diese Sicht ist bewusst unvollständig, und drei Grenzen sind festzuhalten: Ein Pad-Fehler und eine fehlgeschlagene OCS-Rechnung fallen in derselben Ausprägung `Failed` zusammen, ein Parserfehler im TLV-Strom erzeugt keinen eigenen Optionsbericht, und `Peer::recv` bildet alle Nicht-Socket-Fehler sowie gepufferte, verworfene und gefilterte Ausgänge auf „kein Datagramm“ ab. Ob diese Sicht für die Messkampagnen ausreicht, bewertet Kapitel 6.

Eine Schwäche des Empfangswegs zeigte sich erst im Messbetrieb. Der Raw-Empfänger sieht im jeweiligen Netznamensraum jeden UDP-Verkehr unabhängig vom Zielport, die Portauswahl trifft erst der Benutzerraumfilter; in einer frühen Fassung beendete das erste fremde Paket den Empfangsaufruf still, eine laufende Kampagne endete damit ohne Befund, und als Notbehelf diente der Neustart des Empfängers. Die Behebung verlegt die Filterung in eine Schleife unter einer Gesamtfrist: Gefilterte Datagramme (fremde Ports, eigene Kopien, unlesbare Köpfe) beenden den Aufruf nicht mehr, und vor jedem Leseversuch wird die Wartezeit des Sockets auf die Restzeit der Frist gestellt, sodass stetiger fremder Verkehr das Warten nicht verlängert.¹⁴

¹² `ReceivePolicy` in `src/api/mod.rs` (Zeilen 139 bis 200).

¹³ `OcsReport`, `OcsStatus` und `OptionReport` in `src/recv/pipeline.rs` (Zeilen 128 bis 173).

¹⁴ `src/socket/recv.rs` (Zeilen 113 bis 175); Behebung in den Commits `f3bf430` und `8f8c11b`,

Seitdem bedeutet ein leeres Ergebnis genau „Frist ohne Treffer abgelaufen“; eine öffentliche Unterscheidung zwischen gefiltert und abgelaufen gibt es bewusst nicht.

In dieser Schleife steht auch der zweite und letzte `unsafe`-Block in `src/`: Nach jedem Leseerfolg fasst `std::slice::from_raw_parts` die vom Betriebssystem gefüllten Bytes als Slice zusammen (`src/socket/recv.rs`, Zeile 169), und der `SAFETY`-Kommentar hält fest, dass die Socket-Schicht genau die ersten n Bytes initialisiert hat. Damit ist die Zählung aus Abschnitt 5.2 eingelöst: genau zwei `unsafe`-Blöcke im Produktionsbaum `src/`, einer je Richtung. Was hinter der Surplus-Ortung mit dem Optionsstrom geschieht, vertieft nun Abschnitt 5.5.

5.5 Optionsverarbeitung

Die Komponente `options` trägt beide Richtungen des Optionsteils: Der Serialisierer baut aus typisierten Optionswerten den Optionskörper, den der Sendepfad in die Surplus Area kopiert (Abschnitt 5.3), und der Parser liest beim Empfang den TLV-Strom hinter dem OCS (Abschnitt 5.4). Das Format selbst hat Abschnitt 2.4.4 eingeführt; hier steht, wie der Bau es umsetzt und welche Politik er darüberlegt.

Der Rahmen ist eine bewusste Anlehnung an TCP: Die Optionen nutzen eine TLV-Syntax nach dem Vorbild der TCP-Optionen, und die zwei Ein-Byte-Optionen EOL (Kind 0) und NOP (Kind 1) übernehmen zusätzlich die Kind-Werte von TCP; beide haben eine implizite Länge von eins, und nach einem frühen EOL folgen Nullbytes bis zum Ende der Surplus Area [2, Sec. 8 und 10][9].

Auf der Sendeseite endet der gewöhnliche Optionsbau in derselben Schlussfunktion des Serialisierers; nur die Optionskörper der einzelnen FRAG-Fragmente erzeugt `frag/split.rs` getrennt (Abschnitt 5.7). Die Schlussfunktion zeigt die ganze Ordnung des Optionskörpers auf einmal:¹⁵

```
pub fn finish(self) -> Result<Vec<u8>, SerializeError> {
    let mut options = self.validated_options()?;
    options.sort_by_key(|option| canonical_rank(option.kind.to_byte()));

    let body_len = serialized_body_len(&options)?;
    patch_frag_start(&mut options, body_len)?;
    let mut out = Vec::with_capacity(body_len);
    out.extend_from_slice(&[0, 0]);

    for option in &options {
        if out.len() % 2 == 1 {
            out.push(kind::NOP);
        }
        encode_option(&mut out, option)?;
    }
}
```

beide Vorfahren von f847895.

¹⁵ `OptionsBuilder::finish` in `src/options/serialize.rs` (Zeilen 65 bis 87, unverändert); die kanonische Reihenfolge bestimmt `canonical_rank` (Zeilen 173 bis 183).

```

    }

    out.push(kind::EOL);
    if out.len() % 2 == 1 {
        out.push(0);
    }
    debug_assert_eq!(out.len(), body_len);
    Ok(out)
}

```

Der Körper beginnt mit dem Null-Platzhalter für das OCS, damit die Socket-Schicht später nur noch nachpatchen muss (Abschnitt 5.3). Danach folgen die Optionen in kanonischer Reihenfolge, also der festen Sendereihenfolge der Bibliothek: FRAG zuerst, dann APC, MDS, MRDS, REQ, RES, unbekannte Arten nach Kind-Wert (`patch_frag_start` trägt dabei in einer FRAG-Option nach, wo ihre Fragmentdaten beginnen). Beginnt eine Option an ungerader Stelle, schiebt der Serialisierer ein NOP davor; den Schluss bilden EOL und höchstens ein Nullbyte auf gerade Länge, und vor dem EOL selbst wird nie ausgerichtet, wie es der RFC empfiehlt [2, Sec. 11.1]. Für das Beispieldatagramm entsteht so genau der Körper aus Abschnitt 5.3: Platzhalter, die sechs REQ-Bytes, EOL, ein Füllbyte. Der Parser der Gegenseite liefert daraus genau eine Option, Kind 6 mit Länge 6 und dem Token `c0ffee01`; damit ist das Beispiel, das Abschnitt 5.3 gebaut und Abschnitt 5.4 geprüft hat, als Parserergebnis abgeschlossen.

Der Parser behandelt die zwei Ein-Byte-Optionen in eigenen Zweigen: NOP wird überlesen und gezählt, EOL beendet die Liste endgültig; was danach kommt, betrachtet der Parser nicht mehr.¹⁶ Genau dort liegt ein bewusst nicht genutzter Spielraum: Der RFC erlaubt dem Empfänger, die Nullfüllung nach EOL zu prüfen, verlangt es aber nicht; wer prüft und etwas anderes als Nullen findet, verwirft die Optionsliste nach der Grundregel und stellt die Nutzdaten zu [2, Sec. 11.1]. Die Implementierung verzichtet auf diese freiwillige Prüfung.

Oberhalb des Formats liegt Sendepolitik, und sie ist bewusst geschichtet. Der Serialisierer erzwingt nur die harten Bauregeln: gültige Kind-Bereiche, die festen Wertlängen der Pflichtoptionen, die Obergrenzen für Wertlänge und Gesamtlänge des Optionskörpers, die Darstellbarkeit von `Frag.Start` und höchstens ein FRAG je Körper.¹⁷ Die Empfehlung des RFC, dass gewöhnliche Optionen nicht mehrfach auftreten sollen, trägt dagegen die öffentliche Schnittstelle: Der hohe Sendepfad weist Duplikate genau der fünf meldefähigen Arten APC, MDS, MRDS, REQ und RES zurück, während die tiefe Serialisierer-Ebene Duplikate absichtlich zulässt, damit

¹⁶ `src/options/parse.rs` (Zeilen 62 bis 89).

¹⁷ `validated_options` samt Schichtungskommentar in `src/options/serialize.rs` (Zeilen 89 bis 108); die Duplikatprüfung des hohen Sendepfads in `src/api/mod.rs` (Zeilen 592 bis 595 und 613 bis 630); die Menge der meldefähigen Arten (Zeilen 704 bis 709) gehört zur Empfangsrichtlinie.

auch regelwidrige Optionsfolgen für Prüfzwecke entstehen können. Die Normseite dazu ist zweigeteilt: Mehrfache gewöhnliche Optionen sind eine Soll-Regel mit klarer Empfängerfolge (nur die erste Instanz zählt), mehrfaches FRAG dagegen macht den Optionsbereich empfängerseitig zum Fehlerfall [2, Sec. 10]. Die Sperre im Serialisierer ist damit eigene Sendepolitik und keine wörtliche RFC-Pflicht.

Eine zweite eigene Politik betrifft das erweiterte Längenformat. Es ist für Optionen ab 255 Bytes Gesamtlänge vorgesehen; der Sender muss unterhalb dieser Grenze die Kurzform verwenden und soll stets das kleinste Format wählen; für eine strukturell vollständige, aber nicht kanonische erweiterte Gesamtlänge (4 bis 254) legt der RFC keine eigene Empfängerdisposition fest, während Längen unter dem Mindestwert der jeweiligen Art ein RFC-Fehlerfall bleiben [2, Sec. 10]. Der Parser entscheidet streng: Eine erweiterte Länge unter 255 gilt als ungültige Länge und verwirft alle Optionen, genau wie ein echter Überlauf über die Surplus Area hinaus (FR-10). Der Quelltext kennzeichnet diese Strenge ausdrücklich als lokale Politik, nicht als RFC-Pflicht.¹⁸

Im Datenmodell trennt eine feste Grenze SAFE von UNSAFE: Kind-Werte 0 bis 191 gelten als SAFE, weil ein Altempfänger sie folgenlos übergehen kann, Kind-Werte 192 bis 255 als UNSAFE, weil sie die Deutung der Nutzdaten verändern können (Abschnitt 2.4.5).¹⁹ Entsprechend unterschiedlich fallen die Dispositionen aus: Eine unbekannte SAFE-Option wird still übergangen. Eine unbekannte UNSAFE-Option außerhalb eines Fragmentkontexts führt zur Zustellung mit leerer Nutzlast: Verworfen werden die Nutzdaten, nicht das Paket, denn ein Empfänger, der die Option nicht kennt, würde die Nutzdaten falsch deuten. Hat die Pipeline vor der unbekannten UNSAFE-Option bereits ein gültiges FRAG erkannt, verwirft sie den zugehörigen Reassemblierungssatz; trägt erst das wieder zusammengesetzte Datagramm eine solche Option, wird auch dieses mit leerer Nutzlast zugestellt [2, Sec. 12]. Selbst erzeugen kann die Bibliothek UNSAFE-Optionen nicht: Der Serialisierer weist jeden Kind-Wert ab 192 zurück.

Acht Optionen bilden den Pflichtkern: EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES muss jeder Endpunkt, der UDP Options unterstützt, erkennen und bei entsprechender Konfiguration auch erzeugen können; TIME gehört nicht dazu [2, Sec. 10]. Die Bibliothek führt genau diese acht als benannte Arten ihres Optionsverzeichnisses (`src/options/kind.rs`); jede andere Art läuft als `Other` mit rohem Kind-Wert durch das Datenmodell. FRAG ist unter den acht die aufwendigste und teilt sich in zwei Hälften: `frag/split.rs` zerlegt eine Nachricht in Fragmente, `frag/reassembly.rs` setzt sie unter begrenztem Speicher wieder zusammen; die

¹⁸ `src/options/parse.rs` (Zeilen 95 bis 110, Kommentar in den Zeilen 101 bis 104).

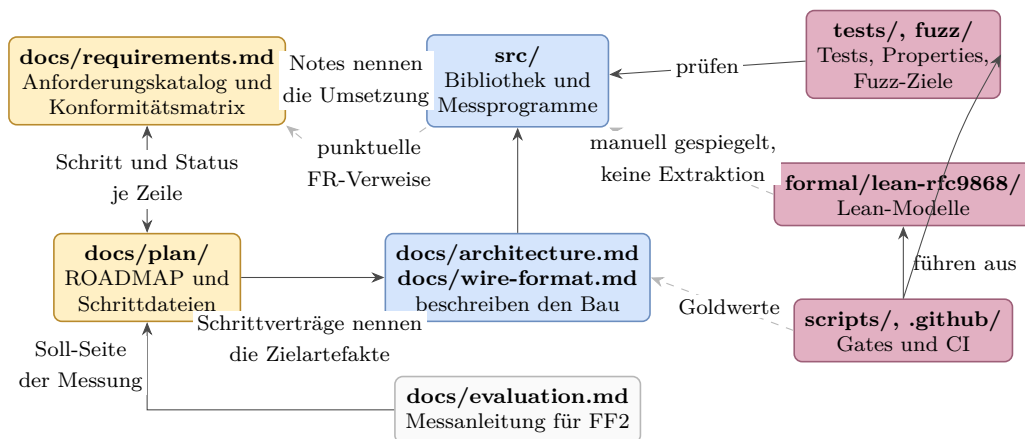
¹⁹ `UNSAFE_MIN` in `src/model.rs` (Zeilen 33 bis 37); die Dispositionen in `src/recvpipeline.rs` (Zeilen 317 bis 334).

Grenzen dieser Puffer behandelt Abschnitt 5.7.

REQ und RES zeigen schließlich den Rahmenwerk-Charakter des Standards: RFC 9868 definiert Format und Grundsemantik des Frage-Antwort-Paars; die Pfad-MTU-Ermittlung DPLPMTUD für UDP Options, die es nutzt, steht als Anwendungsfall in einem eigenen Dokument [3]. Die Bibliothek folgt diesem Zuschnitt: **Res** ist reine Datenhaltung, die Antwort auf ein empfangenes REQ löst die Anwendung aus, und die Herkunft des RES-Tokens bleibt ausdrücklich Aufruferpflicht (Abschnitt 5.7).²⁰ Ob dieser Bau hält, was die Kennungen versprechen, entscheidet nicht das Kapitel selbst, sondern die Prüfanlage, die der nächste Abschnitt zeigt.

5.6 Prüfachitektur

Den Sollprozess der Prüfung hat Abschnitt 3.5 festgelegt; dieser Abschnitt zeigt die gebaute Anlage mit ihren Zahlen am eingefrorenen Stand. Sie besteht aus sechs Ebenen: Attributtests, Property-Tests, Fuzz-Ziele, Integrationstests, Lean-Modelle und die Wire-Prüfung mit getrennten Prüfern. Wo diese Ebenen im Repository liegen und worauf sie sich stützen, zeigt Abbildung 5.4:



Durchgezogen: ausdrücklicher Verweis, Import oder Ausführung. Gestrichelt: manuelle oder teilweise Spiegelung.

Abbildung 5.4: Entwicklungs- und Nachweisartefakte des Implementierungs-Repositorys

Aus Abbildung 5.4 geht die Arbeitsteilung der Artefakte hervor: Links führt der Katalog die Anforderungen und der Plan die Schritte, in der Mitte steht der Bau mit seinen beschreibenden Referenzen, rechts prüfen Tests, Modelle und Gates; nur die gestrichelten Kanten tragen keine erzwungene Kopplung, und genau dort setzt die Bewertung in Kapitel 6 an.

Die ersten beiden Ebenen laufen bei jedem Prüflauf auf dem Entwicklungsrechner. 211 **#[test]**-Attributzeilen verteilen sich auf 156 in **src/** und 55 in **tests/**; gezählt

²⁰ **src/options/typed.rs** (Zeilen 164 bis 177).

sind Attributzeilen, nicht Prüffälle, denn aus einer Zeile können im Lauf viele Fälle entstehen. Das gilt vor allem für die zehn Property-Dateien unter `tests/`: Der Host-Prüflauf erzeugt je Eigenschaft 1.024 Zufallsfälle; ohne diese Vorgabe gilt der Standardwert 256 der Bibliothek `proptest`, etwa in der Kreuzprüfung auf dem Zielsystem.²¹

Die dritte Ebene sucht Abstürze: neun Fuzz-Ziele, je eines für Reassemblierung, Zerlegung, OCS, Serialisierer, TLV-Strom, typisierte Werte, Empfangspipeline, Datagrammaufbau und Wire-Schicht. Der Arbeitskorpus entsteht während des Laufs; eingeeckelt sind kuratierte Startwerte (20 Dateien für sechs der neun Ziele) und die Gegenbeispiele früherer Funde als gewöhnliche Regressionstests (`tests/fuzz_regressions.rs`, 94 Zeilen).

Die vierte Ebene prüft das Zusammenspiel mit dem Betriebssystem über Loopback und Raw Socket. Sieben Tests tragen `#[ignore]`: Sechs verlangen erhöhte Rechte und laufen in der Prüfmaschine, der siebte ist ein Hilfsprozess, den ein Rauschtest über eine Umgebungsvariable startet. Die sudo-Lane der Prüfskripte ist dabei eine Ausführungsart derselben Testdateien, keine eigene Testmenge.

Die fünfte Ebene modelliert die Wire-Regeln in Lean: elf Fachmodule mit zusammen 172 Theoremen, von der Prüfsummenarithmetik bis zur Empfangsordnung. Ein Prüfskript baut die Modelle und auditiert jedes Theorem auf Kernel-Ebene: Zugelassen sind nur die drei Grundaxiome von Lean; ein `sorry` oder ein `native_decide` fiele als zusätzliches Axiom auf.²² Bewiesen sind damit Modelle, nicht das Rust-Programm; diese Grenze nimmt Abschnitt 5.8 auf.

Die sechste Ebene begegnet einer Schwäche, die alle bisherigen teilen: Sender, Empfänger und Prüfungen stammen aus derselben Feder. Ein beidseitig gleicher Formatfehler überstünde jeden Roundtrip-Test; Property-Tests und Fuzz-Ziele prüfen zwar getrennt formulierte Invarianten, können aber Annahmefehler ihres eigenen Orakels teilen. Die Wire-Prüfung bricht diese Selbstbezüglichkeit mit drei getrennten Prüfkomponten: `tcpdump` zeichnet hinter dem Kernel auf, ein eigenständiges Python-Prüfprogramm ohne gemeinsamen Rust-Code rechnet die für die hinterlegten Szenarien relevanten IPv4-, UDP-, Surplus-, OCS-, TLV-, APC- und FRAG-Werte mit eigener RFC-1071-Faltung und eigener CRC32C-Tabelle nach (696 Zeilen), und `tshark` liest die IPv4- und UDP-Felder als fremder Dissektor [12]. Dass die Kette greift, zeigt eine gezielte Einzelprobe: Ein einzelnes absichtlich verändertes Surplus-Byte lässt das Prüfprogramm dreifach fehlschlagen. Eine systematische Mutationsanalyse

²¹ `PROPTTEST_CASES` in `scripts/pre-pr.sh` (Zeilen 20 und 50); die Kreuzprüfung ruft `cargo test` ohne diese Variable auf (`scripts/vm-ubuntu-server.sh`, Zeilen 130 und 141).

²² `scripts/lean-gate.sh` (Zählung Zeilen 24 bis 27, Audit Zeilen 31 bis 40); die Aggregatdatei `Rfc9868.lean` zählt nicht mit.

des Rust-Quelltexts ist das ausdrücklich nicht.

Wie nötig auch für das Prüfprogramm selbst geeignete Prüfdaten sind, zeigte ein Fund aus seiner ersten Fassung: Die Nachrechnung des ausgerichteten OCS gruppierte die 16-Bit-Wörter um ein Byte verschoben, und der Referenzfall blieb trotzdem grün, denn die damalige REQ-Option mit dem Token `deadbeef` faltet zu `0xA3A3`, einem Byte-Palindrom, dem die Vertauschung nichts anhat. Die Lehre ist doppelt: Genau gegen solche Verschiebungen richtet der RFC das OCS an der 2-Byte-Grenze aus (Abschnitt 2.4.2), und auch eine unabhängige Prüfinstanz braucht Testdaten, die ihre eigenen Fehlerklassen nicht maskieren. Die heutige REQ-Option mit dem Token `c0ffee01` faltet deshalb zu dem nicht palindromischen Wert `0xB507`.²³

Was jede Ebene leistet und was nicht, stellt Tabelle 5.1 einander gegenüber:

Tabelle 5.1: Reichweite der Prüfmittel

Prüfmittel	findet	findet nicht
Attributtests (211 Zeilen)	festgelegte Einzelfälle, Regressionen	Fehler außerhalb der gewählten Fälle
Property-Tests (10 Dateien)	verletzte Invarianten unter Zufallseingaben	beidseitig geteilte Fehlannahmen
Fuzz-Ziele (9)	Abstürze und Panics auf rohen Bytefolgen	Fehldeutungen ohne verletzte hinterlegte Invariante
Integrationstests (7 mit <code>ignore</code>)	Zusammenspiel mit Kernel und Rechten	Verhalten fremder Netzpfade
Lean-Modelle (172 Theoreme)	Widersprüche und Lücken in den modellierten Regeln	Abweichungen zwischen Modell und Rust
Wire-Prüfung (3 getrennte Prüfer)	beidseitig geteilte Formatfehler auf dem Draht	Fehler jenseits der hinterlegten Szenarien

Aus Tabelle 5.1 geht hervor, dass keine Ebene die anderen ersetzt: Jede Zeile nennt eine Fehlerklasse, die genau diese Ebene nicht sieht, und erst die Summe deckt den Raum von der Einzelregression bis zum fremdgeprüften Draht. Dass erst die Summe trägt, zeigte die Praxis zweimal: Der Um-eins-Fehler der Surplus-Ortung überstand 23 Einzelfalltests und fiel erst der Zweitprüfung im Pull-Request auf, worauf Property-Tests und Fuzz-Ziele je Parsefläche verpflichtend wurden, und der RFC-Abgleich von Schritt 18 fand trotz grüner Prüfkette weitere Konformitätslücken (Abschnitt 5.1).

Den Schluss der Klammer, die Abschnitt 5.2 geöffnet hat, bildet die Rückverfolgbarkeit. Abschnitt 5.2 bis Abschnitt 5.5 haben die tragenden Komponenten mit den Anforderungskennungen des Implementierungs-Repositorys verknüpft; die Datei `docs/requirements.md` ordnet umgekehrt jeder FR- und NFR-Zeile den umsetzen den Schritt des Projektplans und ihren Status zu (Abschnitt 4.2), und punktuelle Rückverweise im Quelltext nennen die Kennungen an tragenden Stellen, etwa FR-49

²³ Begründung der Token-Wahl im Prüfprogramm `scripts/wire-check.py` (Zeilen 244 bis 248).

an der Längenprüfung. Diese Rückverfolgbarkeit ist nicht flächendeckend: Nicht jede Quelltextstelle nennt ihre Kennung, und der Katalogabgleich lief periodisch, vollständig erst mit Schritt 18 (Abschnitt 3.2); was eine Zeile wirklich stützt, entscheidet deshalb je Nachweis erst Kapitel 6. Mit dieser Zuordnung ist das Messinstrument beisammen; es bleibt zu klären, unter welchen Rechten und Grenzen es betrieben wird.

5.7 Betriebs- und Sicherheitsgrenzen

Der Betrieb beginnt mit einem Privileg: Raw Sockets verlangen unter Linux die Fähigkeit `CAP_NET_RAW`, also das gezielt vergebbare Recht auf rohe Netzwerkzugriffe, oder gleich Root-Rechte [17, 45]. Wer die Bibliothek einsetzt, gibt dem Prozess damit weitreichenden Zugriff auf rohe IPv4-Datagramme, nicht nur auf die eigenen dieser Arbeit: Linux füllt bei `IP_HDRINCL` nur `IP Total Length` und die IP-Kopfprüfsumme stets selbst aus (Abschnitt 5.3), Quelladresse und Paketkennung dagegen nur, wenn sie null sind; ein solcher Prozess kann also insbesondere fremde Quelladressen vorgeben [17]. Die Bibliothek kapselt diesen Zugriff hinter ihrer Schnittstelle, aufheben kann sie ihn nicht; die Rechtevergabe bleibt eine Betriebsentscheidung des Aufrufers (`src/socket/mod.rs`, Zeilen 1 bis 6).

Drei Entwurfsprinzipien des RFC aus Abschnitt 2.4.5 prägen den Betrieb der Empfangsseite. Das erste betrifft die Richtung: `Peer::recv` ruft keinen Sendepfad auf, die RES-Option ist im Datenmodell reine Datenhaltung, und dass ein RES-Token wirklich aus einem empfangenen REQ stammt, bleibt ausdrückliche Aufruferpflicht (Abschnitt 5.5). Diese strikte Trennung verkleinert die Fläche für Reflexionsmissbrauch, ein allgemeiner Schutz ist sie nicht; die Sicherheitsbetrachtungen des RFC nennen REQ als die einzige Pflichtoption, die ohne eigene Sendung der Anwendung eine Antwort der Oberschicht auslösen kann [2, Sec. 25.2]. Die Verbindung dieser Beobachtung mit der Pfadtrennung des Baus ist eine eigene Ableitung dieser Arbeit.

Das zweite Prinzip betrifft den Zustand: UDP bleibt zustandslos, nötiger Zustand gehört in die Anwendung oder eine Bibliothek in ihrem Auftrag, und die Reassemblierung ist die einzige, ausdrücklich begrenzte Ausnahme. Im Bau ist der `ReassemblyCache` der einzige fachliche Empfangszustand über Paketgrenzen hinweg; sein Schlüssel besteht aus dem UDP-Vierertupel und der FRAG-Identifikation, also genau dem Identitätskriterium, mit dem der RFC ein ursprüngliches Datagramm eindeutig bestimmt. Eine Cache-Instanz je Socket-Paar fordert der RFC dagegen nicht; er empfiehlt nur, die Speichergrenzen der Reassemblierung nicht über mehrere Sockets hinweg als gemeinsame Ressource zu berechnen [2, Sec. 11.4]. Die dokumentierte Bindung jeder Instanz an genau ein Socket-Paar ist deshalb der strengere

eigene Aufruervertrag dieser Implementierung, denn die Mengengrenze gilt je Cache. Der Typ erzwingt ihn nicht, und auch die **Peer**-Ebene löst ihn nur teilweise ein: Jeder **Peer** hält genau einen Cache, filtert eingehende Datagramme aber allein über die beiden UDP-Ports, sodass Verkehr verschiedener Adresspaare mit gleichen Ports denselben Cache und dessen Mengengrenze teilt.²⁴ Sendeseitig existiert daneben nur der fortlaufende Identifikationszähler der Zerlegung, der monoton zählt und bei Erschöpfung anhält, statt Werte zu wiederholen (**src/frag/split.rs**, Zeilen 66 bis 94). Die globalen Warnzähler dienen allein der Diagnose; die Zähler im Reassemblierungszustand erzwingen dagegen die konfigurierten Grenzen.

Das dritte Prinzip legt Grenzen in die Konfiguration statt in das Format: Der Parser kennt keine Obergrenze für die Zahl der Optionen und prüft jede Option nur lokal auf Längenfehler. Mehr als sieben aufeinanderfolgende **NOP**-Optionen lösen eine Warnung aus, beenden die Verarbeitung aber nicht; Warnungen erscheinen je Kategorie beim ersten und danach bei jedem 64. Vorkommnis und werden nur sichtbar, wenn die einbettende Anwendung einen Logger installiert. Mengengrenzen trägt die Empfangskonfiguration **ReassemblyLimits** mit vier Feldern: wiederhergestellte Datagrammgröße einschließlich UDP-Kopf, Fragmentzahl je Datagramm, gleichzeitig unvollständige Datagramme je Cache und Verfallsfrist (**FR-34**, **NFR-06**). Die ersten beiden Voreinstellungen sind wörtliche **MUST**-Mindestwerte des RFC: 2.926 Byte als kleinste für IPv4 zu unterstützende Wiederherstellungsgröße (das entspricht genau zwei Fragmenten in Ethernet-Größe) und zwei Fragmente, unabhängig von der IP-Version. Die voreingestellte Verfallsfrist von zwei Minuten übernimmt eine **SHOULD**-Obergrenze für den Standardwert. Die vierte Zahl, 64 unvollständige Datagramme je Cache, ist dagegen eine eigene Schutzentscheidung: Eine Grenze für anhängige Fragmente empfiehlt der RFC nur Implementierungen, die die Fragmentierung als mögliche Gefährdung ansehen, und er nennt keinen Zahlenwert [2, Sec. 11.4, 11.6 und 25.4].²⁵ Auch hier gilt die Zweistufigkeit der Schnittstelle: Die tiefe Ebene erlaubt eigene Grenzen (**ReassemblyCache::with_limits**), wobei die wirksame Verfallsfrist bei zwei Minuten gekappt bleibt, und die hohe **Peer**-Ebene verwendet die Voreinstellungen. Die Verfallsfrist wirkt dabei nicht selbsttätig: Abgelaufener Zustand wird erst bei der nächsten Einfügung oder durch einen ausdrücklichen Bereinigungsaufwurf (**gc**) entfernt, ohne beides bleibt er gespeichert. **udpopt-recv** stellt die vier Empfangsgrenzen über Kommandozeilenargumente ein, **udpopt-send** trennt davon die Sendelänge und die angenommenen MRDS-Grenzen der Gegenstelle. Die konfigurierte Sendelänge ist eine Vorgabe des Aufrufers und keine Ermittlung

²⁴ **src/frag/reassembly.rs** (Zeilen 16 bis 25 und 241 bis 253); **Peer** mit eigenem Cache in **src/api/mod.rs** (Zeilen 263 bis 290).

²⁵ **ReassemblyLimits** mit Voreinstellungen in **src/frag/reassembly.rs** (Zeilen 216 bis 239); die Konstanten samt Herkunft in **src/model.rs** (Zeilen 73 bis 80).

der Pfad-MTU: Überschreitet ein fertiges Datagramm die MTU der Strecke, liefert Linux `EMSGSIZE`, und die Bibliothek reicht das als allgemeinen E/A-Fehler weiter (Abschnitt 5.3).

Eine letzte Betriebsgrenze zieht die Umfangsvorgabe dieser Arbeit: Angenommen wird ausschließlich `Protocol 17`; jedes andere Protokollfeld weist die Wire-Schicht mit einem eigenen Fehlerwert ab. Shim-Ketten wie IPsec oder IPComp werden damit nicht verfolgt, obwohl deren Kopflängen von der Obergrenze der UDP `Length` zusätzlich abzuziehen wären; Abschnitt 7 des RFC beschreibt diese Reduktion, trägt an dieser Stelle aber keine mit Schlüsselwörtern markierte Pflicht [2, Sec. 7]. Diese Zuschnittsentscheidung hat Abschnitt 4.1 dokumentiert.

Damit sind Rechte, Zustand und Grenzen des Betriebs benannt; was der Bau darüber hinaus bewusst nicht leistet, fasst der letzte Abschnitt zusammen.

5.8 Bewusste Grenzen

Das Kapitel hat ein Messinstrument beschrieben, keinen Allzweckbau. Damit die Evaluation seine Ergebnisse richtig einordnen kann, gehören die Grenzen dieses Baus ebenso ausdrücklich benannt wie seine Fähigkeiten.

Die erste Grenze ist die Plattform. Der Bau belegt den Benutzerraumweg unter Linux mit IPv4 und Raw Sockets, wie es die Plattformvorgabe aus Kapitel 4 vorgibt; über Kernel-Implementierungen, andere Betriebssysteme oder IPv6 sagt er nichts aus. Der macOS-Messpunkt aus Tabelle 4.3 bleibt reiner Beobachtungspunkt der Messinfrastruktur; alles Weitere würde den Rahmen dieser Arbeit sprengen.

Die zweite Grenze ist das Netz. Alle Prüfungen dieses Kapitels laufen auf dem eigenen Host oder im kontrollierten Prüfaufbau; ob die Surplus Area reale Pfade mit fremden Zwischensystemen übersteht, ist damit nicht gezeigt und auch nicht Gegenstand dieses Kapitels. Genau diese Frage untersucht Kapitel 6 mit dem hier gebauten Instrument.

Die dritte Grenze ist die Beweiskraft. Die Lean-Modelle beweisen Aussagen über Modelle der Wire-Regeln, nicht über das laufende Rust-Programm; die Verbindung zwischen beiden bleibt Prüfsache der Tests (Abschnitt 4.5.4). Fuzzing wiederum findet Fehler, beweist aber keine Abwesenheit von Fehlern. Beide Aussagen begrenzen, was „geprüft“ in dieser Arbeit bedeutet.

Die vierte Grenze ist der bewusst begrenzte Anforderungsumfang. Der Anforderungskatalog hält je Zeile fest, was abgewählt oder nur teilweise umgesetzt ist: die freiwillige Nullprüfung nach EOL (FR-18, nicht gewählt; Abschnitt 5.5), der

Verzicht auf einen Deckel für die Optionszahl (NFR-04, nicht gewählt), die erkennende, aber nicht arbeitsbegrenzende NOP-Behandlung (NFR-05, teilweise), die teilweise umgesetzte Empfangsrichtlinie je Socket-Paar (FR-44) und die im Raw-Pfad konstruktionsbedingte, als Anforderung nur teilweise belegte ICMP-Unterdrückung (FR-48); dazu kommen die aufrufergesteuerte Cache-Bereinigung und die fehlende Pfad-MTU-Ermittlung (Abschnitt 5.7). Diese Zeilen sind die erwarteten Kandidaten der Teilbewertung in Kapitel 6.

Mit diesen Grenzen ist das Messinstrument umrissen: die Bibliothek mit ihren Komponenten und Kennungen, die beiden Pfade mit dem Beispieldatagramm, die Optionsverarbeitung und die sechsstufige Prüfanlage. Das Beispieldatagramm mit dem Token `c0ffee01` geht als Referenzpaket an die Evaluation über: Kapitel 6 setzt das Instrument auf die Forschungsfragen an.

6. Evaluation und Diskussion

6.1 Test- und Messkonzept

6.2 Beobachtbarkeit und Middlebox- Verträglichkeit auf realen Pfaden

6.3 Soll-Ist-Abgleich mit RFC 9868

7. Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

7.2 Ausblick

7.3 Schlusswort

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768). URL: <https://www.rfc-editor.org/info/rfc768>.
- [2] J. Touch und C. Heard. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868). URL: <https://www.rfc-editor.org/info/rfc9868>.
- [3] G. Fairhurst und T. Jones. *Datagram Packetization Layer Path MTU Discovery (DPLPMTUD) for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869). URL: <https://www.rfc-editor.org/info/rfc9869>.
- [4] A. Stephan und L. Wüstrich. „The Path of a Packet Through the Linux Kernel“. In: *Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 23*. Technische Universität München, Chair of Network Architectures und Services, 2024, S. 89–93. DOI: [10.2313/NET-2024-04-1_16](https://doi.org/10.2313/NET-2024-04-1_16). URL: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-04-1/NET-2024-04-1_16.pdf (besucht am 13.08.2026).
- [5] J. Postel. *Internet Protocol*. RFC 791. Internet Engineering Task Force, Sep. 1981. DOI: [10.17487/RFC0791](https://doi.org/10.17487/RFC0791). URL: <https://www.rfc-editor.org/info/rfc791>.
- [6] S. Deering und R. Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200. Internet Engineering Task Force, Juli 2017. DOI: [10.17487/RFC8200](https://doi.org/10.17487/RFC8200). URL: <https://www.rfc-editor.org/info/rfc8200>.
- [7] F. Gont, R. Atkinson und C. Pignataro. *Recommendations on Filtering of IPv4 Packets Containing IPv4 Options*. RFC 7126. Internet Engineering Task Force, Feb. 2014. DOI: [10.17487/RFC7126](https://doi.org/10.17487/RFC7126). URL: <https://www.rfc-editor.org/info/rfc7126>.
- [8] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085). URL: <https://www.rfc-editor.org/info/rfc8085>.
- [9] W. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, Aug. 2022. DOI: [10.17487/RFC9293](https://doi.org/10.17487/RFC9293). URL: <https://www.rfc-editor.org/info/rfc9293>.
- [10] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675). URL: <https://www.rfc-editor.org/info/rfc675>.

- [11] J. F. Kurose und K. W. Ross. *Computer Networking: A Top-Down Approach*. 8. Aufl. Pearson, 2021.
- [12] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071). URL: <https://www.rfc-editor.org/info/rfc1071>.
- [13] C. Huitema. *Teredo: Tunneling IPv6 over UDP through Network Address Translations (NATs)*. RFC 4380. Internet Engineering Task Force, Feb. 2006. DOI: [10.17487/RFC4380](https://doi.org/10.17487/RFC4380). URL: <https://www.rfc-editor.org/info/rfc4380>.
- [14] D. Thaler. *Teredo Extensions*. RFC 6081. Internet Engineering Task Force, Jan. 2011. DOI: [10.17487/RFC6081](https://doi.org/10.17487/RFC6081). URL: <https://www.rfc-editor.org/info/rfc6081>.
- [15] RFC Editor. *RFC Errata for RFC 9868*. RFC Editor. 2026. URL: <https://www.rfc-editor.org/errata/rfc9868> (besucht am 13.08.2026).
- [16] Linux Kernel Community. *Linux 5.10: net/ipv4/udp.c*. URL: <https://elixir.bootlin.com/linux/v5.10/source/net/ipv4/udp.c> (besucht am 19.08.2026).
- [17] Linux man-pages project. *raw(7): IPv4 raw sockets*. Linux man-pages 6.18. URL: <https://man7.org/linux/man-pages/man7/raw.7.html> (besucht am 17.08.2026).
- [18] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Internet Engineering Task Force, März 1997. DOI: [10.17487/RFC2119](https://doi.org/10.17487/RFC2119). URL: <https://www.rfc-editor.org/info/rfc2119>.
- [19] B. Leiba. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. Internet Engineering Task Force, Mai 2017. DOI: [10.17487/RFC8174](https://doi.org/10.17487/RFC8174). URL: <https://www.rfc-editor.org/info/rfc8174>.
- [20] A. T. Kalai, O. Nachum, S. S. Vempala und E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. 2025. URL: <https://arxiv.org/abs/2509.04664> (besucht am 13.08.2026).
- [21] E. Perez u. a. *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv:2212.09251. 2022. URL: <https://arxiv.org/abs/2212.09251> (besucht am 13.08.2026).
- [22] M. Cheng, C. Lee, P. Khadpe u. a. „Sycophantic AI decreases prosocial intentions and promotes dependence“. In: *Science* 391.6792 (2026). Artikelkennung eaec8352. DOI: [10.1126/science.aec8352](https://doi.org/10.1126/science.aec8352).
- [23] J. Jumper, R. Evans, A. Pritzel u. a. „Highly accurate protein structure prediction with AlphaFold“. In: *Nature* 596 (2021), S. 583–589. DOI: [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).

- [24] A. Davies, P. Veličković, L. Buesing u. a. „Advancing mathematics by guiding human intuition with AI“. In: *Nature* 600 (2021), S. 70–74. DOI: [10.1038/s41586-021-04086-x](https://doi.org/10.1038/s41586-021-04086-x).
- [25] B. Romera-Paredes, M. Barekatain, A. Novikov u. a. „Mathematical discoveries from program search with large language models“. In: *Nature* 625 (2024), S. 468–475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [26] T. Hubert, R. Mehta, L. Sartran u. a. „Olympiad-level formal mathematical reasoning with reinforcement learning“. In: *Nature* 651.8106 (2026). Online veröffentlicht am 12.11.2025, S. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- [27] D. E. Knuth. *Claude’s Cycles*. Vom 28.02.2026, überarbeitet am 14.04.2026. 2026. URL: <https://cs.stanford.edu/~knuth/papers/claude-cycles.pdf> (besucht am 13.08.2026).
- [28] Anthropic. *Learning more about Claude’s mathematical capabilities*. Vom 10.08.2026. 2026. URL: <https://www.anthropic.com/research/riemann-zeta> (besucht am 13.08.2026).
- [29] OpenAI. *An OpenAI model has disproved a central conjecture in discrete geometry*. Vom 20.05.2026. 2026. URL: <https://openai.com/index/model-d-isproves-discrete-geometry-conjecture/> (besucht am 13.08.2026).
- [30] M. D. Schwartz. *Resummation of the C-Parameter Sudakov Shoulder Using Effective Field Theory*. arXiv:2601.02484, eingereicht am 05.01.2026. 2026. URL: <https://arxiv.org/abs/2601.02484> (besucht am 13.08.2026).
- [31] *Rewriting Bun in Rust*. Oven (Bun). 2026. URL: <https://bun.com/blog/bun-in-rust> (besucht am 13.08.2026).
- [32] *Rewrite Bun in Rust*. oven-sh/bun, Pull Request 30412. Gemergt am 14.05.2026; 1.009.257 hinzugefügte Zeilen. Metadaten per GitHub-API verifiziert. GitHub. 2026. URL: <https://github.com/oven-sh/bun/pull/30412> (besucht am 13.08.2026).
- [33] *PathString::slice dangling reference UB - add Miri to CI (oven-sh/bun, Issue 30719)*. Gemeldet am 14.05.2026, nach dem Merge des Rust-Ports. GitHub. 2026. URL: <https://github.com/oven-sh/bun/issues/30719> (besucht am 13.08.2026).
- [34] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*. Vortrag, BlueHat IL 2019, Tel Aviv. Microsoft Security Response Center, BlueHat IL 2019. 7. Feb. 2019. URL: <https://www.youtube.com/watch?v=PjbGojJnBZQ> (besucht am 31.05.2026).
- [35] The Chromium Project. *Memory safety*. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).

- [36] National Security Agency. *Software Memory Safety. Cybersecurity Information Sheet*. National Security Agency (NSA). 10. Nov. 2022. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 31.05.2026).
- [37] The Rust Project. *Rust Programming Language*. 2025. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [38] S. Klabnik und C. Nichols. *The Rust Programming Language*. 2025. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).
- [39] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. Frei verfügbarer Arbeitsentwurf zu ISO/IEC 9899:2018 (C17). ISO/IEC JTC 1/SC 22/WG14. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 31.05.2026).
- [40] The kernel development community. *Development tools for the kernel*. The Linux Kernel Archives. 2026. URL: <https://docs.kernel.org/dev-tools/index.html> (besucht am 20.08.2026).
- [41] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/0.16.0/> (besucht am 24.08.2026).
- [42] LLVM Project. *libFuzzer: A Library for Coverage-Guided Fuzz Testing*. LLVM Project. 2026. URL: <https://llvm.org/docs/LibFuzzer.html> (besucht am 20.08.2026).
- [43] rust-lang-Projekt. *socket2: Advanced configuration options for sockets*. URL: <https://github.com/rust-lang/socket2> (besucht am 24.08.2026).
- [44] rust-lang-Projekt. *libc: Raw bindings to platform APIs for Rust*. URL: <https://github.com/rust-lang/libc> (besucht am 24.08.2026).
- [45] Linux man-pages project. *capabilities(7): overview of Linux capabilities*. Linux man-pages 6.18. URL: <https://man7.org/linux/man-pages/man7/capabilities.7.html> (besucht am 23.08.2026).

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Quellen entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

.....

Ort, Datum

.....

Unterschrift